

UNIVERSIDAD POLITÉCNICA DE MADRID

E.T.S. DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROYECTO FIN DE GRADO

GRADO EN INGENIERÍA DE COMPUTADORES

Estudio de patologías cardíacas mediante Electrocardiograma y Vectocardiograma

Desarrollado por: Alejandro García Hidalgo

Dirigido por: Edgar Talavera

Madrid, 6 de julio de 2026



Estudio de patologías cardíacas mediante Electrocardiograma y Vectocardiograma

Desarrollado por: Alejandro García Hidalgo

Dirigido por: Edgar Talavera

Proyecto Fin de Grado, 6 de julio de 2026

E.T.S. de Ingeniería de Sistemas Informáticos

Campus Sur UPM, Carretera de Valencia (A-3), km. 7

28031, Madrid, España

Resumen

Las enfermedades cardiovasculares son la principal causa de mortalidad a nivel mundial, a menudo por fallos eléctricos imprevistos en el corazón. Tradicionalmente, el electrocardiograma (ECG) ha sido la prueba estándar para estudiar estos fallos, sin embargo, entre sus limitaciones se encuentra evaluar con claridad ciertas zonas complejas, como la cara posterior del miocardio. La vectocardiografía (VCG) resuelve esto al estudiar la electricidad del corazón en tres dimensiones, el problema es que los equipos hospitalarios de VCG son muy grandes y caros, impidiendo su uso en el día a día del paciente.

El objetivo de este proyecto es simplificar esta tecnología. Para ello, se ha diseñado e implementado un sistema digital portátil sobre una FPGA Artix-7 capaz de filtrar la señal cardíaca y medir automáticamente los intervalos RR y el RT para alertar sobre diferentes patologías cardíacas como arritmias. El sistema captura los datos y aplica la Transformada Wavelet tridimensional que separa las frecuencias de la señal. Esto permite aislar fácilmente el complejo QRS y la onda T.

A continuación, el hardware detecta el pico exacto de estas ondas basándose en máximos y mínimos dinámicos. Para evitar falsos positivos provocados por el ruido, el sistema unifica los datos de los tres ejes espaciales mediante una regla de votación por mayoría. Una vez validados los latidos, se calcula la duración de los intervalos en milisegundos y un bloque de diagnóstico evalúa si hay alguna anomalía. Finalmente, toda esta información y las alarmas médicas se empaquetan y se envían en tiempo real a un cuadro de mandos interactivo desarrollado en Python.

La validación del sistema se realizó de forma rigurosa en el simulador de Vivado, utilizando señales de pacientes reales de la base de datos internacional del MIT-BIH. Las pruebas confirmaron que la arquitectura digital es capaz de procesar las señales de forma paralela y activar las alertas de salud de forma precisa. Todo el núcleo matemático y lógico del proyecto ha quedado plenamente probado, sentando una base robusta para el futuro desarrollo de dispositivos médicos ambulatorios de telemetría tridimensional.

Palabras clave: Vectocardiografía; FPGA; Transformada Wavelet; Máquinas de Estados; Delineación Automática; Intervalo RT; Muerte Súbita; Python; Vivado; VHDL

Abstract

Cardiovascular diseases are the leading cause of global mortality, often due to unexpected electrical failures in the heart. Traditionally, the electrocardiogram (ECG) has been the standard test to study these failures; however, one of its limitations is the difficulty in clearly evaluating certain complex areas, such as the posterior wall of the myocardium. Vectorcardiography (VCG) solves this by studying the heart's electricity in three dimensions, the problem is that hospital VCG equipment is very large and expensive, preventing its use in the patient's daily life.

The goal of this project is to simplify this technology. To achieve this, a portable digital system has been designed and implemented on an Artix-7 FPGA capable of filtering the cardiac signal and automatically measuring the RR and RT intervals to warn about different cardiac pathologies such as arrhythmias. The system captures the data and applies a three-dimensional Wavelet Transform that separates the signal's frequencies. This makes it easy to isolate the QRS complex and the T wave.

Next, the hardware detects the exact peak of these waves based on dynamic maximums and minimums. To avoid false positives caused by noise, the system unifies the data from the three spatial axes using a majority voting rule. Once the heartbeats are validated, the duration of the intervals is calculated in milliseconds, and a diagnostic block evaluates if there is any anomaly. Finally, all this information and the medical alarms are packaged and sent in real time to an interactive dashboard developed in Python.

System validation was rigorously performed in the Vivado simulator, using real patient signals from the international MIT-BIH database. The tests confirmed that the digital architecture is capable of processing the signals in parallel and accurately triggering health alerts. The entire mathematical and logical core of the project has been fully proven, laying a robust foundation for the future development of ambulatory 3D telemetry medical devices.

Keywords: Vectorcardiography; FPGA; Wavelet Transform; State Machines; Automatic Delineation; RT Interval; Sudden Death; Python; VHDL; Vivado

Índice general

1	INTRODUCCIÓN	1
1.1	Motivación y Contexto	1
1.2	Estado de la Cuestión	2
1.3	Objetivos del Proyecto	3
1.4	Estructura de la memoria	4
2	FUNDAMENTOS TEÓRICOS	6
2.1	Funcionamiento Eléctrico del Corazón	6
2.2	Electrocardiografía y Vectocardiografía	8
2.3	Patologías Cardíacas de Interés	11
2.4	La Transformada Wavelet	14
3	DISEÑO E IMPLEMENTACIÓN	16
3.1	Planteamiento General	16
3.2	Especificación del Hardware	17
3.3	Módulo SPI y Recepción de la Trama	22
3.4	Unidad de Filtrado Wavelet 3D	24
3.5	Detector de Eventos	26
3.6	Módulo Puente	30
3.7	Lógica de Diagnóstico y Alarmas	33

3.8	Módulo de Comunicación UART y Protocolo de Trama	36
3.9	Interfaz Gráfica de Telemetría (Python)	39
4	SIMULACIÓN Y VALIDACIÓN	42
4.1	Base de Datos del MIT-BIH	42
4.2	Simulación en Vivado	44
4.3	Unidad de Filtrado Wavelet 3D	45
4.4	Máquina de Estados del Detector QRS	46
4.5	Máquina de Estados del Detector T	47
4.6	Módulo Puente	48
4.7	Lógica de Diagnóstico y Alarmas	48
4.8	Módulo de Comunicación UART	54
4.9	Pruebas de la Interfaz Gráfica	55
4.10	Pruebas del Módulo SPI	56
5	PRUEBAS FÍSICAS, GESTIÓN Y CONCLUSIONES	57
5.1	Pruebas Físicas con la Placa y el Sensor Real	58
5.2	Planificación Temporal	61
5.3	Presupuesto del Proyecto	61
5.4	Conclusión	62
5.5	Posibles Mejoras	63
	BIBLIOGRAFÍA	65
A	CÓDIGO FUENTE VHDL	68

B CÓDIGO FUENTE PYTHON	98
-------------------------------	-----------

C IMPACTO SOCIAL Y MEDIOAMBIENTAL	108
--	------------

Glosario	109
--------------------	-----

Siglas	110
------------------	-----

1.1. Motivación y Contexto

Las enfermedades cardiovasculares son la principal causa de muerte en el planeta, provocando la pérdida de millones de vidas cada año [17]. La gran mayoría de estos desenlaces fatales se deben a fallos imprevistos en el sistema eléctrico del corazón, que necesita recibir señales eléctricas muy precisas para saber cuándo y con qué fuerza contraerse. Cuando este circuito interno se descoordina y el ritmo natural se rompe, se producen problemas graves como las arritmias o la muerte súbita. Lo más peligroso de estas alteraciones es que suelen presentarse de forma repentina y sin síntomas previos en personas que aparentemente están sanas. Por tanto, poder vigilar de cerca y de forma constante esta actividad eléctrica es una necesidad vital para detectar cualquier anomalía a tiempo y evitar que el corazón colapse.

Tradicionalmente, se utiliza el electrocardiograma (ECG) de 12 derivaciones como la prueba estándar para estudiar estos fallos. Sin embargo, la práctica médica demuestra que el ECG tiene problemas para evaluar con claridad ciertas zonas, como la cara posterior del miocardio.

Para solucionar esto existe la vectocardiografía (VCG). En lugar de dibujar las ondas como simples líneas horizontales a lo largo del tiempo, el VCG analiza la electricidad del corazón en tres dimensiones, permitiendo un análisis más exhaustivo del órgano. A pesar de ser una técnica efectiva para observar zonas ocultas, arrastra un gran inconveniente: los equipos clínicos son grandes, caros y complejos. Debido a esto, actualmente es una tecnología poco accesible para monitorizar de forma continua a un paciente durante su vida diaria.

La motivación de este Trabajo de Fin de Grado nace de la necesidad de hacer esta tecnología más accesible, uniendo la ingeniería con la medicina preventiva. Desde el punto de vista de la ingeniería, el reto es procesar los tres ejes espaciales (X , Y , Z), lo que implica manejar una gran cantidad de información en fracciones de segundo. Plantear un microcontrolador como solución tiene varios inconvenientes; por ejemplo, que se saturaría intentando filtrar la señal y buscar sus ondas a la vez. Es aquí donde resulta ideal el uso de un chip lógico programable como la FPGA. Su capacidad para realizar procesos en paralelo permite dividir el diseño en bloques independientes que trabajan de forma simultánea sin atascarse [11]. A nivel médico, este enfoque abre las puertas a crear dispositivos Holter inteligentes, portátiles y ligeros. Equipos capaces de

calcular en tiempo real los intervalos críticos para predecir y alertar sobre cualquier patología cardíaca directamente desde el hardware.

1.2. Estado de la Cuestión

El desarrollo de sistemas para vigilar la actividad del corazón siempre ha buscado reducir el tamaño de los equipos. Tradicionalmente, las pruebas ambulatorias como, por ejemplo, el dispositivo Holter, se limitan a guardar las señales eléctricas en una memoria para después ser analizadas en el hospital. Sin embargo, gracias al uso de placas programables (FPGA) es posible ejecutar cálculos matemáticos directamente en el hardware para analizar el corazón en tiempo real mientras el paciente hace su vida normal.

Los algoritmos tradicionales suelen basarse en fórmulas matemáticas sencillas [13] o umbrales fijos para detectar los latidos. Aunque son muy fáciles de procesar para un ordenador normal, presentan un gran inconveniente: son muy sensibles al ruido. Un simple movimiento del paciente o la tensión muscular deforman la señal y comúnmente provocan diagnósticos equivocados o falsas alarmas.

Durante la investigación de proyectos con un propósito similar al que se tenía para este Trabajo de Fin de Grado, el estudio más interesante encontrado ha sido uno basado en medir la frecuencia cardíaca y el intervalo RT de forma portátil [11]. En ese trabajo demuestran que es completamente viable usar una FPGA para ejecutar un filtrado avanzado como la Transformada Wavelet. Este método limpia la señal y localiza las ondas del corazón en tiempo real, adaptándose a la forma de cada latido sin confundirse con el ruido. Además, al procesarlo en hardware, se logra reducir significativamente el consumo de energía en comparación con los procesadores convencionales.

A pesar de estos grandes descubrimientos, tras revisar la literatura médica y tecnológica, no se ha encontrado ninguna solución que se acerque a la planteada en este proyecto. Las investigaciones portátiles más avanzadas, como la mencionada anteriormente [11], trabajan únicamente con el electrocardiograma tradicional analizando un solo punto de vista. Por otro lado, los sistemas clínicos actuales que usan la vectocardiografía para analizar el corazón en 3D lo hacen siempre de forma diferida. Es decir, descargan los datos y los procesan posteriormente en ordenadores externos una vez terminada la prueba.

El motivo de no encontrar una solución parecida a la planteada se debe a que sincronizar y procesar matemáticamente tres canales espaciales al mismo tiempo en un dispositivo pequeño es algo muy complejo de diseñar. Para plantear una solución a este problema, el desarrollo de este

proyecto se apoya en los manuales de referencia técnica de los chips lógicos de última generación [9], los cuales detallan cómo construir hardware síncrono para comunicarse con sensores externos sin perder datos [12].

En conclusión, el estado de la cuestión actual justifica la necesidad de este proyecto para unificar el análisis espacial en tres dimensiones con el filtrado avanzado en hardware, creando un dispositivo portátil capaz de evaluar y prevenir problemas cardíacos de forma autónoma.

1.3. Objetivos del Proyecto

El objetivo principal de este proyecto es diseñar e implementar una arquitectura digital modular sobre una FPGA. Este sistema debe ser capaz de capturar la señal eléctrica del corazón, procesarla en tres dimensiones y calcular automáticamente en tiempo real los intervalos críticos RR y RT. El fin último es crear una herramienta de telemetría portátil para prevenir anomalías del ritmo cardíaco y problemas de conducción eléctrica.

Para definir claramente el alcance del sistema, es importante mencionar una limitación en el diseño. Inicialmente se valoró incluir un detector automatizado de infartos de miocardio, pero se descartó para garantizar la fiabilidad del prototipo. Identificar un infarto requiere analizar la morfología exacta del segmento ST, una tarea de gran complejidad. Además, la variabilidad de estas ondas entre pacientes es muy grande y existen muy pocas bases de datos verificadas para realizar pruebas concluyentes, lo que comprometía la seguridad del diagnóstico final.

Por lo tanto, centrando el esfuerzo en los componentes verificados y viables, los objetivos específicos que estructuran este trabajo son los siguientes:

- **Diseño del núcleo de adquisición y filtrado Wavelet 3D:** Programar en lenguaje VHDL un controlador síncrono para extraer los datos del sensor analógico a través del bus SPI. A continuación, implementar un banco de filtros Wavelet en cascada basado en splines cúbicos que procese en paralelo real las tres coordenadas espaciales (X, Y, Z) del corazón.
- **Lógica adaptativa y cálculo de intervalos:** Construir dos máquinas de estados independientes que trabajen en equipo para localizar el pico de la onda R y el final exacto de la onda T. La idea es emplear umbrales adaptativos que se ajusten al ritmo pertinente, calculando la duración de los intervalos RR y RT en milisegundos con una gran precisión.
- **Módulo supervisor de diagnósticos:** Diseñar un bloque de hardware que analice continuamente los intervalos de tiempo calculados. Este módulo activará banderas de alerta automática ante taquicardias, bradicardias, arritmias, asistolias o riesgo inminente de muerte súbita.

- **Procesamiento de bases de datos de referencia:** Desarrollar un script en Python para importar y limpiar los registros de la base de datos internacional de arritmias del MIT-BIH. El objetivo es generar un archivo procesable por el entorno de simulación de Vivado, con el resultado de la señal original amplificada. Permitiendo validar los algoritmos en simulación antes de pasar al laboratorio.
- **Integración física de biopotenciales:** Establecer la arquitectura de conexiones físicas entre la placa FPGA (Nexys Video) y el módulo de adquisición analógico (ADS1293EVM). Se utilizará un sistema simplificado de 5 electrodos colocados sobre el paciente para asegurar una captura fiable del vector eléctrico tridimensional.
- **Interfaz gráfica de telemetría:** Programar un cuadro de mandos interactivo en Python capaz de recibir los datos que la FPGA transmite por el puerto serie. El software deberá desempaquetar las tramas, interpretar los valores en complemento a dos (con signo) y representar en pantalla las gráficas temporales, el bucle tridimensional (VCG) y el estado de las alarmas en tiempo real.

1.4. Estructura de la memoria

Esta memoria se encuentra estructurada en cinco capítulos principales que guían al lector a través de la secuencia lógica del desarrollo del proyecto, desde el planteamiento del problema médico hasta la evaluación técnica y económica del hardware. A continuación, se resume el contenido de cada uno de ellos:

- **Capítulo 1 - Introducción:** Define el punto de partida del proyecto. Expone la motivación médica y tecnológica, analiza el estado de la cuestión para contextualizar la innovación de la arquitectura propuesta frente a soluciones previas, y establece los objetivos concretos que estructuran el trabajo.
- **Capítulo 2 - Fundamentos Teóricos:** Establece las bases fisiológicas y matemáticas necesarias para comprender el sistema. Explica el comportamiento eléctrico del miocardio, los principios de la vectocardiografía tridimensional y las patologías cardíacas objeto de estudio. También, detalla los fundamentos de la Transformada Wavelet como herramienta para el procesamiento de señales no estacionarias.
- **Capítulo 3 - Diseño e Implementación del Sistema:** Está formado por el núcleo de ingeniería del trabajo. Describe minuciosamente la selección del hardware (FPGA Nexys Video

y sensor analógico ADS1293) y el desarrollo algorítmico en VHDL de la arquitectura: la adquisición síncrona mediante bus SPI, el banco de filtros Wavelet 3D, las máquinas de estados para la delineación de ondas, la calibración de intervalos y la lógica de diagnóstico. Finaliza detallando la transmisión serie UART y el diseño del cuadro de mandos interactivo en Python.

- **Capítulo 4 - Simulación y Validación Lógica:** Se centra de manera exclusiva en la verificación digital del código. Detalla el acondicionamiento de los registros de pacientes reales extraídos de la base de datos internacional del MIT-BIH, analiza los cronogramas resultantes en los entornos de simulación de Vivado y expone las métricas cuantitativas que validan la exactitud matemática de los módulos de detección.
- **Capítulo 5 - Pruebas Físicas, Gestión y Conclusiones:** Aborda la fase de integración en el laboratorio, analizando el montaje físico de los componentes, la síntesis lógica y la depuración del hardware mediante analizadores integrados (ILA). Expone la planificación temporal, la viabilidad económica del proyecto, la conclusión y las líneas de desarrollo futuro.

2. FUNDAMENTOS TEÓRICOS

2.1. Funcionamiento Eléctrico del Corazón

La función principal del corazón es oxigenar y proporcionar sangre oxigenada a todo el cuerpo. Está dividido en cuatro partes o cavidades principales: dos aurículas en la parte superior encargadas de recibir la corriente sanguínea, y dos ventrículos en la parte inferior, responsables de impulsar la sangre para oxigenarla y distribuirla al resto del cuerpo humano. Esto conlleva una coordinación muy precisa para asegurar que la gestión de la sangre se realiza correctamente. Para ello, se generan distintos impulsos eléctricos que resultan en la contracción del miocardio. A la medición de estas variaciones de potencial se le llama electrocardiograma (ECG) [3].

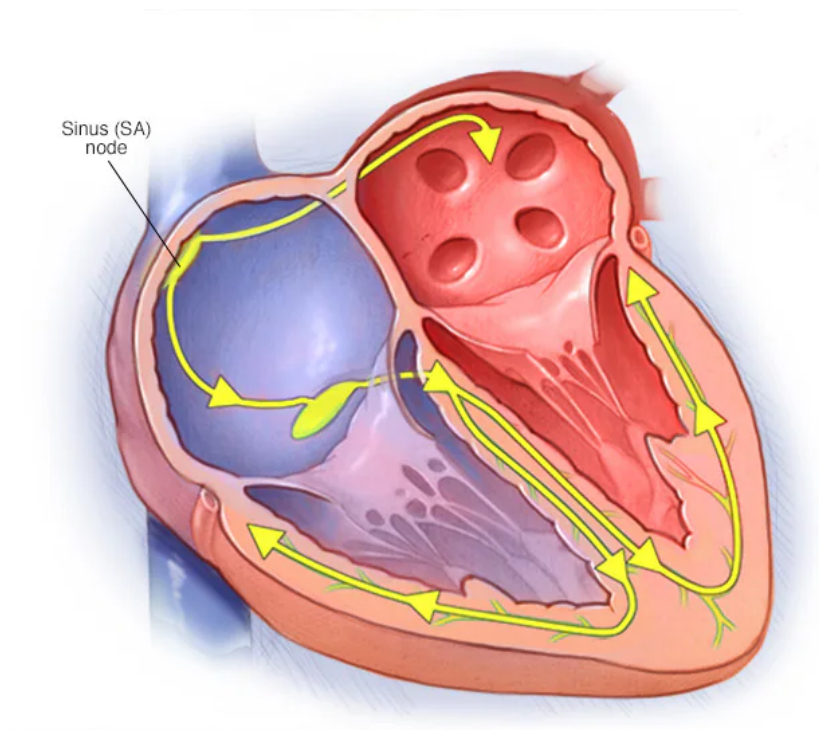


Figura 2.1. Anatomía y sistema de conducción eléctrica del corazón [2].

El ciclo cardíaco se compone de dos fases: la sístole, fase en la que el músculo se contrae para

expulsar la sangre, y la diástole, fase encargada de la relajación y destinada al llenado de las cámaras cardíacas. Este proceso se inicia de forma automática mediante un grupo de células encargadas de generar impulsos eléctricos que posteriormente contraerán el músculo. Esta corriente eléctrica se origina en la aurícula derecha, específicamente en el nódulo sinoauricular, el marcapasos natural del organismo. El impulso generado en este punto se propaga por el tejido auricular [16], provoca la contracción de las aurículas y desplaza la sangre hacia los ventrículos.

Tras recorrer las aurículas, el estímulo eléctrico alcanza el nódulo auriculoventricular. Este nodo se encarga de generar un leve retraso en el impulso eléctrico, permitiendo que las aurículas se vacíen antes de que comience la siguiente contracción ventricular [2] e, inmediatamente después, se distribuye por las paredes ventriculares. Este mecanismo asegura una contracción sincronizada de ambos ventrículos, generando la presión necesaria para impulsar la sangre hacia los pulmones y hacia el sistema circulatorio.

2.1.1. Funcionamiento a Escala Celular

A escala celular, la actividad eléctrica miocárdica está basada en las variaciones de potencial que se producen en las membranas de las células cardíacas. En estado de reposo, el interior celular posee una carga negativa de aproximadamente -90 mV [1] respecto al exterior celular. Esta diferencia de potencial permanece constante hasta la llegada de un estímulo eléctrico producido por una célula adyacente, que activa un proceso cíclico donde diversos iones, principalmente sodio (Na^+), calcio (Ca^{2+}) y potasio (K^+), se desplazan a través de la membrana celular.

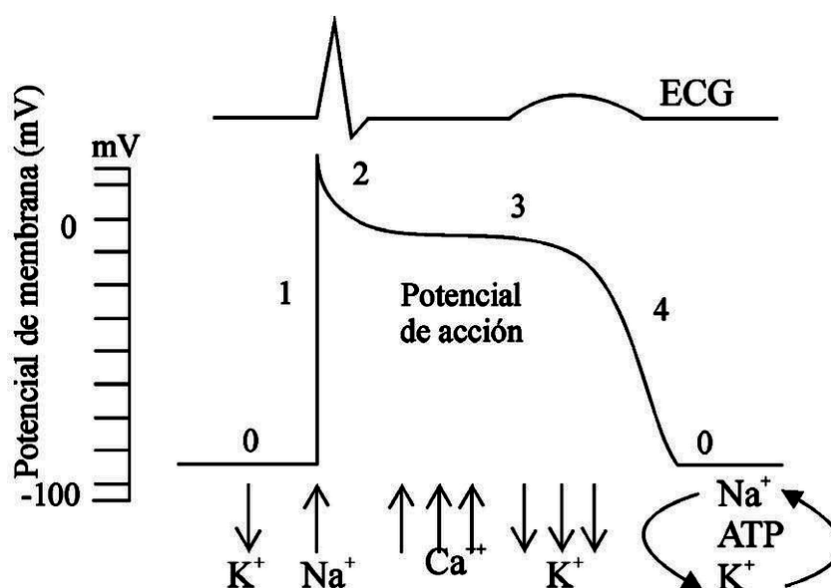


Figura 2.2. Fases del potencial de acción en una célula miocárdica [1].

Este movimiento de componentes químicos se divide en cinco etapas. La fase 0 representa la activación inicial, donde la entrada rápida de sodio invierte el voltaje de la célula para encender el latido. Las fases 1 y 2 forman un periodo de estabilidad o meseta, exclusivo del corazón, donde se inicia la salida de potasio y entra el calcio de forma equilibrada, manteniendo la energía alta el tiempo suficiente para que el músculo pueda contraerse físicamente. Inmediatamente después, la fase 3 realiza la descarga de la célula expulsando el potasio restante para recuperar la carga negativa original. Finalmente, en la fase 4 se produce el estado de reposo definitivo, donde se reajustan por completo los niveles de los tres minerales [1] y la célula queda preparada para recibir el siguiente impulso.

2.2. Electrocardiografía y Vectocardiografía

La electrocardiografía es la técnica que registra la actividad eléctrica del corazón desde la superficie de la piel. Cada vez que las células del músculo cardíaco se activan y se recuperan, generan pequeñas corrientes eléctricas que se extienden por todo el cuerpo. Al colocar electrodos en puntos concretos de los brazos, piernas y el pecho, el sensor puede medir cambios de voltaje a lo largo del tiempo. El resultado de esta medición es el electrocardiograma (ECG), una herramienta esencial para comprobar el ritmo del corazón y su estado estructural [3].

La señal de un ECG normal está compuesta por un conjunto de ondas que reflejan el orden de los movimientos eléctricos del corazón. La señal empieza con la onda P, una pequeña elevación que muestra la activación y contracción de las aurículas. Después de una breve pausa, aparece el complejo QRS, que es un conjunto de tres ondas rápidas y altas que representan la contracción de los ventrículos. Al tener una masa muscular mucho mayor, esta es la parte con más fuerza eléctrica de toda la onda. Durante este complejo también se produce la relajación de las aurículas, pero al ser una onda eléctrica muy débil y suceder al mismo tiempo, queda completamente opacada. Por último, se puede observar la onda T, una curva más lenta y suave que muestra la recuperación eléctrica de los ventrículos [15], indicando que las células se están preparando para el siguiente latido.

Para estudiar correctamente el corazón, no solo se mira la forma de estas ondas, sino también el tiempo que pasa entre ellas. El intervalo RR es el tiempo exacto que transcurre entre el punto más alto (el pico R) de un latido y el del siguiente. Esta medida sirve para calcular la frecuencia cardíaca y ayuda a ver fallos en el ritmo, como aceleraciones o paros imprevistos. Por otra parte, el intervalo RT mide el tiempo desde que empiezan a contraerse los ventrículos hasta que

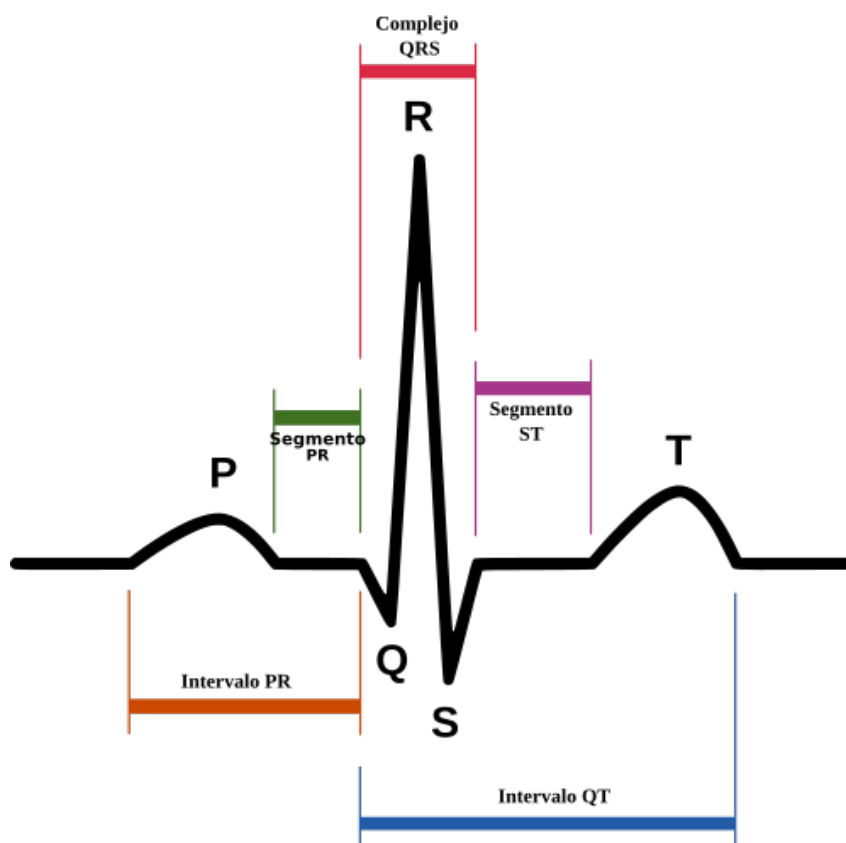


Figura 2.3. Morfología estándar de la señal electrocardiográfica [3].

terminan de recuperarse por completo. Este intervalo es muy importante, ya que si la onda T se alarga demasiado, puede provocar graves problemas de coordinación que pongan en peligro la vida de la persona. Ambos intervalos son de gran utilidad para detectar enfermedades cardíacas.

En la práctica clínica, se colocan diversos electrodos (sensores físicos) a lo largo de todo el cuerpo para captar correctamente la actividad cardíaca. Las diferencias de voltaje medidas entre estos electrodos forman lo que se conoce como derivaciones. Lo más común es utilizar un sistema de 12 derivaciones, que se consigue distribuyendo 10 electrodos por el cuerpo [3]. Estas derivaciones medirán la diferencia de electricidad entre las extremidades, compararán extremidades con una referencia media y se enfocarán en el pecho para ver el plano horizontal. Si el paciente está en cuidados intensivos o si su patología no es clara, se pueden usar sistemas de 20 o 24 derivaciones añadiendo más puntos en la espalda para obtener una imagen más completa de las zonas ocultas del corazón.

2.2.1. Principios de la Vectocardiografía

A diferencia del ECG tradicional que dibuja las ondas a lo largo del tiempo, la vectocardiografía (VCG) analiza esta misma actividad de forma espacial. Su principio básico considera que toda la electricidad del corazón se puede resumir en un único vector o flecha eléctrica en tres dimensiones que cambia de tamaño y dirección con cada latido [4]. En lugar de trazar líneas horizontales, el vectocardiograma une los puntos de este vector en un espacio de tres ejes (X , Y , Z). De esta manera, los movimientos del latido forman tres bucles o anillos cerrados en el espacio, mostrando así la actividad tridimensional exacta de cada onda (P, QRS y T).

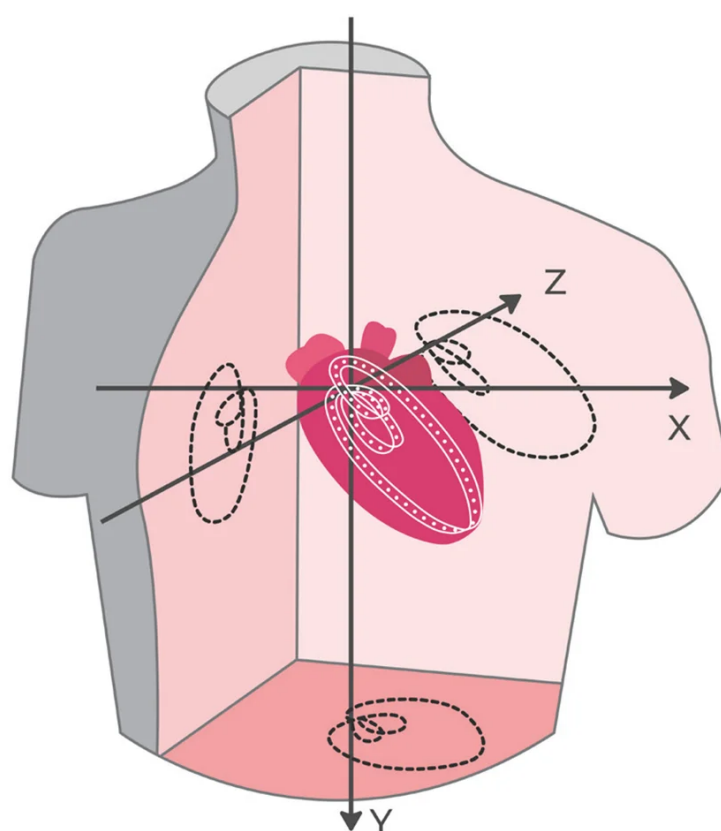


Figura 2.4. Proyección de los bucles vectorcardiográficos en los tres planos del espacio [4].

A nivel clínico, se suele necesitar una tecnología avanzada de 20 o 24 derivaciones para hacer una medición tridimensional del corazón. Sin embargo, es posible calcularlo con una configuración simplificada de solo cinco electrodos. Este método utiliza las relaciones matemáticas de las señales para deducir las tres dimensiones usando los terminales del brazo derecho (RA), brazo izquierdo (LA), pierna izquierda (LL) y un único electrodo en la espalda (V), usando la pierna derecha (RL) como la toma de tierra del sistema.

Para obtener los tres ejes espaciales con únicamente 4 electrodos activos, se necesita un punto de

referencia neutral que represente el centro eléctrico del cuerpo. Para este propósito, comúnmente se utilizan los principios de Goldberger y el terminal central de Wilson (WCT), que aplican una fórmula de promediado para combinar los voltajes de los brazos y piernas [27] y así obtener una línea de base estable.

A partir de estas referencias centrales y del resto de conexiones físicas, el hardware procesa las diferencias de potencial mediante las siguientes operaciones matemáticas:

El **eje X** registra el movimiento horizontal de la corriente del brazo izquierdo al derecho. Se calcula restando directamente el voltaje del brazo derecho (RA) al del brazo izquierdo (LA):

$$X = LA - RA \quad (2.1)$$

El **eje Y** mide el movimiento vertical de la corriente desde la parte superior a la inferior. Se consigue restando la referencia central de Goldberger al potencial de la pierna izquierda (LL), amplificando así la señal vertical en el circuito:

$$Y = LL - \text{Referencia}_{\text{Goldberger}} \quad (2.2)$$

El **eje Z** controla el sentido de la corriente hacia la parte posterior del corazón. Para calcularlo, el sistema toma la señal del electrodo del pecho (V) y le resta el terminal central de Wilson (WCT) calculado anteriormente:

$$Z = V - WCT \quad (2.3)$$

Todas estas operaciones son calculadas normalmente por los propios sensores de instrumentación, mandando las señales ya procesadas y amplificadas al sistema de adquisición.

2.3. Patologías Cardíacas de Interés

El análisis preciso de los intervalos de tiempo en la señal eléctrica no solo sirve para identificar el estado normal del corazón, sino que es la herramienta fundamental para el diagnóstico de diversas enfermedades y anomalías. A través del estudio de las variaciones en los intervalos RR y RT, se pueden detectar las anomalías más comunes.

2.3.1. Alteraciones medidas mediante el Intervalo RR

El intervalo RR es el indicador directo de la frecuencia cardíaca y de la regularidad del ritmo debido a que mide el tiempo exacto que transcurre entre latidos sucesivos [5]. Las principales

patologías de interés para este proyecto que se detectan mediante el estudio de este intervalo son las siguientes:

- **Taquicardia:** Se produce cuando la frecuencia cardíaca en reposo supera los 100 latidos por minuto [17]. En el registro eléctrico, esto se traduce en un acortamiento drástico y sostenido del intervalo RR. Debido a la alta velocidad de los latidos, las ondas de ciclos sucesivos se aproximan notablemente entre sí, reduciendo el tiempo de llenado de las cámaras cardíacas.
- **Bradicardia:** Representa la situación opuesta a la taquicardia, caracterizada por un ritmo cardíaco inferior a los 60 latidos por minuto en condiciones de reposo [15]. El sistema la detecta porque el intervalo RR se prolonga de forma considerable, mostrando un espacio excesivo y constante entre los complejos QRS de cada latido.
- **Arritmia:** Hay numerosos tipos de arritmia, pero en este caso se hace referencia a la pérdida de la regularidad del paso del tiempo entre latidos (por ejemplo, la arritmia sinusal o la fibrilación). A diferencia de las condiciones anteriores, no se identifica por intervalos cortos o largos de forma estable, sino por una variación constante o desordenada del intervalo RR de un latido a otro, lo que refleja una falta de sincronía en el ritmo natural del corazón.
- **Asistolia:** Es la ausencia completa de actividad en el corazón, lo que se traduce clínicamente en un paro cardíaco. En la monitorización, el intervalo RR deja de registrarse o se vuelve matemáticamente infinito, mostrando una línea completamente plana en el monitor sin la presencia de ondas, lo que requiere una intervención médica de extrema urgencia.

2.3.2. Alteraciones medidas mediante el Intervalo RT

Mientras que el intervalo RR se encarga de evaluar la frecuencia del latido, el intervalo RT se centra en el proceso de recuperación ventricular. Como se detalla previamente, este parámetro mide el tiempo total que pasa desde que los ventrículos comienzan a contraerse (inicio del complejo QRS) hasta que sus células se restablecen por completo (final de la onda T).

Cuando se produce un retraso en la aparición o en la finalización de la onda T, el intervalo RT se alarga de forma peligrosa. Clínicamente, a este fenómeno se le llama Síndrome de QT Largo [6]. Este retraso significa que las células miocárdicas tardan demasiado tiempo en expulsar el potasio y recuperar su carga negativa.

Durante este retraso de recuperación celular, el corazón entra en un periodo de vulnerabilidad. Si un nuevo impulso eléctrico, como el complejo QRS, llega antes de que la onda T haya concluido, se genera un cortocircuito eléctrico que puede desencadenar una arritmia ventricular caótica

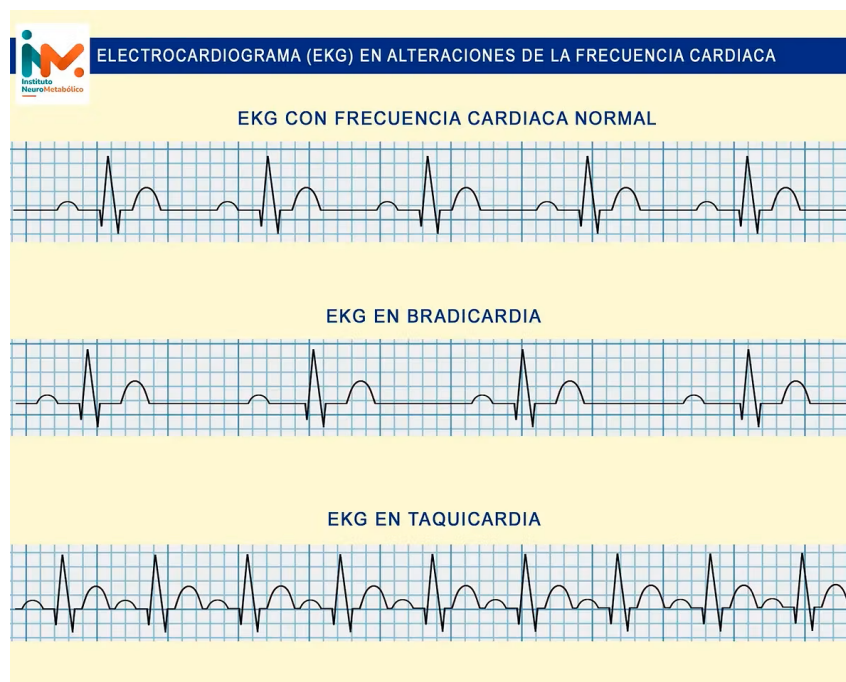


Figura 2.5. Manifestación electrocardiográfica de las alteraciones del ritmo cardíaco en el intervalo RR [5].

y letal conocida como *torsades de pointes* [18]. Esta condición es muy propensa a generar fibrilación ventricular, que en caso de no revertir espontáneamente, es sucedida por el colapso y muerte súbita del paciente.

Por lo tanto, la monitorización del intervalo RT es de vital importancia para la prevención de la muerte súbita cardíaca. Detectar a tiempo este retraso de la onda T permite alertar al paciente antes de que ocurra el fallo crítico, posibilitando prevenir esta patología aun no presentando síntomas externos visibles.

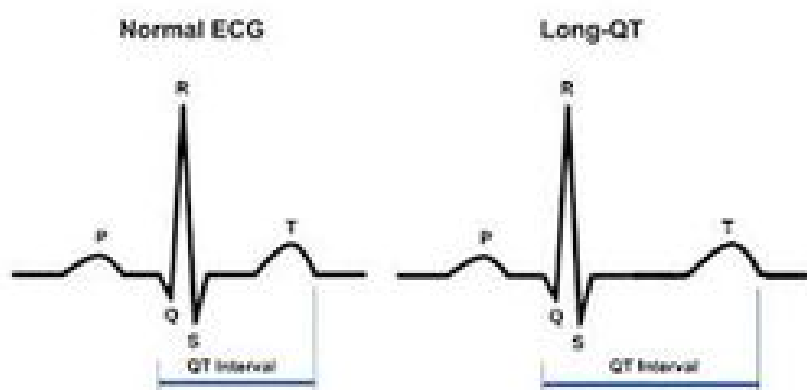


Figura 2.6. Prolongación del intervalo RT y su relación con el periodo ventricular vulnerable [6].

2.4. La Transformada Wavelet

Para poder analizar los intervalos RR y RT explicados en los apartados anteriores, no es suficiente con recibir la señal eléctrica cruda proveniente de los electrodos. Las señales biomédicas como el ECG o el VCG suelen estar acompañadas de diversos tipos de ruido, como las interferencias de la red eléctrica, los movimientos musculares del paciente o los desvíos en la línea de base causados por la propia respiración. A diferencia de las herramientas de filtrado tradicionales o la Transformada de Fourier, que analiza las frecuencias de una señal pero no tiene en cuenta en qué instante en el tiempo ocurren, la Transformada Wavelet (WT) es una herramienta matemática diseñada específicamente para analizar señales no estacionarias, es decir, señales cuyas propiedades y componentes frecuenciales cambian a lo largo del tiempo [7].

El principio básico de la transformada Wavelet consiste en descomponer la señal original en diferentes funciones llamadas *wavelets*. Estas funciones se obtienen a partir de una única función matemática principal conocida como wavelet madre. Esta función base se desplaza a lo largo de toda la señal para identificar en qué momento ocurren los cambios y, al mismo tiempo, se estira o se comprime (proceso denominado escalado) para analizar tanto las frecuencias altas como las bajas. De esta manera, en lugar de obtener un gráfico puramente temporal o puramente frecuencial, la transformada Wavelet ofrece una representación en ambos dominios[19].

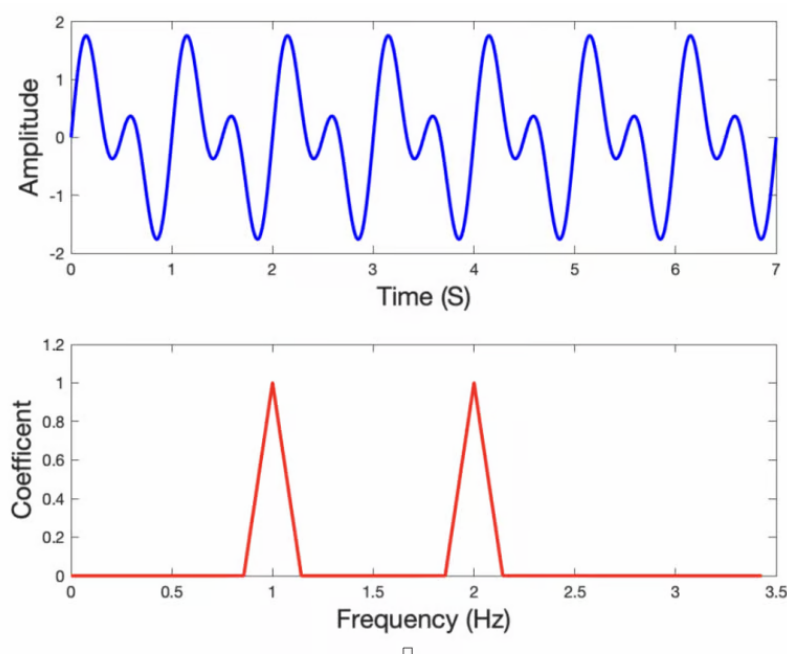


Figura 2.7. Diferencia conceptual en el análisis de señales mediante Fourier y Wavelet [7].

2.4.1. Aplicación en la Delineación y Detección de Ondas

En el contexto de este proyecto, la transformada Wavelet es una herramienta de vital importancia porque permite realizar de manera automática la delineación del ECG. Este proceso consiste en localizar con precisión milimétrica los puntos característicos de la onda, especialmente el pico de la onda R y el final de la onda T, esenciales para el cálculo de los intervalos de interés [20].

Cuando la señal del corazón se procesa mediante este método, se divide en varios niveles o escalas. Cada una de estas escalas resalta un componente específico de la señal según su forma y su velocidad de cambio:

- **El complejo QRS:** Al ser un conjunto de ondas muy rápidas y prominentes (frecuencias altas), destaca con gran nitidez en las primeras escalas (escalas más bajas). Esto permite una fácil localización del pico R, ignorando los ruidos de baja frecuencia.
- **La onda T:** Por el contrario, representa una curva más lenta y suave (frecuencias bajas), por lo que se hace mucho más visible en escalas superiores (escalas más altas). Gracias a esto, el algoritmo posterior puede delimitar con gran precisión dónde ocurre el final de la onda, a pesar de que esté deformada u oculta por el ruido muscular [8].

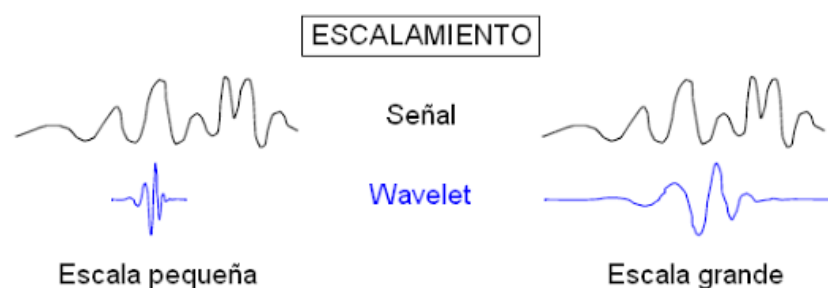


Figura 2.8. Descomposición por escalas de la señal electrocardiográfica para la detección de puntos fiduciales [8].

3.

DISEÑO E IMPLEMENTACIÓN

3.1. Planteamiento General

Una vez asentados los fundamentos médicos y matemáticos, este capítulo detalla el núcleo técnico del proyecto. El objetivo es traducir la teoría en una arquitectura de hardware real, modular y robusta, implementada sobre la placa de desarrollo Nexys Video [9]. Al desarrollar el diseño en lenguaje VHDL sobre una FPGA Artix-7, se aprovecha su capacidad de procesamiento en paralelo, lo que permite manejar los tres canales espaciales (X, Y, Z) de la señal cardíaca de forma estrictamente simultánea.

Para garantizar que el flujo de datos sea constante, rápido y no sufra cuellos de botella desde que la señal entra por el sensor hasta que el diagnóstico aparece en la pantalla del médico, la arquitectura digital se ha segmentado en los siguientes bloques funcionales encadenados:

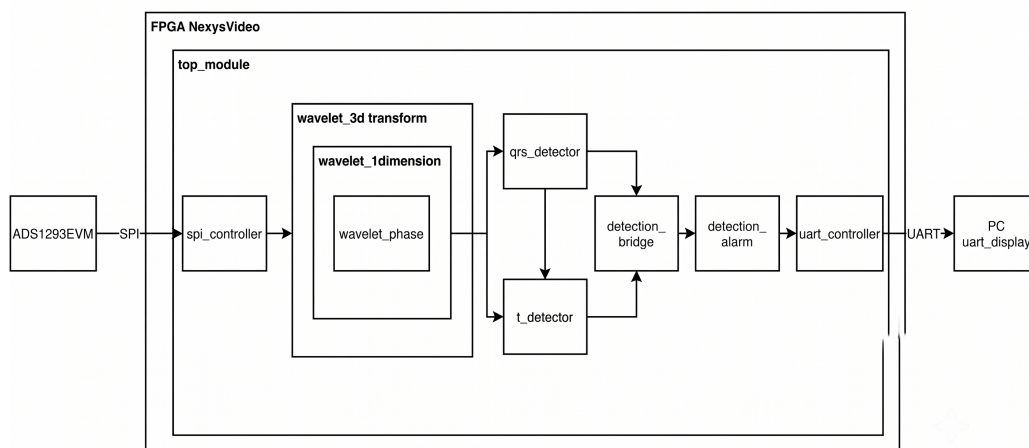


Figura 3.1. Diagrama de bloques general de la arquitectura del sistema digital en la FPGA.

Como se puede observar en el diagrama, el funcionamiento general del sistema sigue un camino de datos unidireccional y síncrono:

- **Adquisición de la Señal (Módulo SPI):** Es la puerta de entrada de los datos biomédicos. Un controlador maestro gestiona los tiempos del bus SPI a una frecuencia segura de 1 MHz para extraer las muestras digitalizadas del sensor comercial ADS1293EVM [10] de forma continua e inmune al ruido de alta frecuencia.
- **Banco de Filtros Wavelet 3D:** La señal en bruto se procesa matemáticamente aplicando el algoritmo *à trous* con filtros de *splines* cúbicos [19]. Para maximizar el rendimiento, el hardware evita el uso de multiplicadores físicos (DSPs) y utiliza desplazamientos de bits rápidos. Este bloque aísla el complejo QRS en la escala rápida 3, y la onda T en la escala lenta 8.
- **Detección Adaptativa de Ondas:** Este sistema de detección se guía por una constante de cero virtual y unos umbrales de máximo y mínimo, que esquivan pequeñas oscilaciones de fondo. Además, dos máquinas de estados localizan el pico exacto de la onda R y el final de la onda T. Estas dos máquinas recalculan automáticamente sus umbrales y ventanas de ceguera en cada latido para adaptarse a la amplitud y el ritmo real del paciente.
- **Unificación y Calibración:** Este bloque centraliza las detecciones de los tres ejes aplicando una regla de votación, es decir, que se confirma un latido si coinciden al menos dos ejes dentro de una ventana de retención predefinida de 30 ms. Tras validar el evento, el módulo compensa analíticamente los retardos propios del hardware (restando 389 ms a la onda T y sumando 83 ms al QRS) para entregar intervalos biológicos perfectos en milisegundos.
- **Supervisor Clínico de Patologías:** Analiza continuamente los intervalos calibrados. Se apoya en registros de persistencia que exigen al menos dos latidos anómalos consecutivos para erradicar falsas alarmas provocadas por el ruido o el movimiento. Si el ritmo crítico se confirma, activa banderas lógicas para alertar de taquicardias, bradicardias, arritmias, asistolias o riesgo inminente de muerte súbita.
- **Transmisión UART y Telemetría en Python:** Toda la información se empaqueta en una trama estructurada de 37 bytes y se transmite hacia el ordenador a 115200 baudios, utilizando una técnica de submuestreo controlado (*throttling*) para no saturar el canal físico. Finalmente, un cuadro de mandos en Python desempaqueta los valores en formato de complemento a dos y dibuja el bucle vectocardiográfico 3D y las alarmas médicas en tiempo real [24].

3.2. Especificación del Hardware

Para llevar a cabo la arquitectura planteada, es muy importante contar con un hardware que ofrezca una gran precisión al leer las señales del cuerpo humano y, al mismo tiempo, suficien-

te capacidad de cálculo para procesarlas en paralelo. Para asegurar la estabilidad del prototipo y poder enfocarse en el desarrollo de la lógica digital, se ha optado por integrar módulos comerciales de evaluación de alto rendimiento en lugar de diseñar los circuitos impresos desde cero.

3.2.1. Placa de Desarrollo Nexys Video

El cerebro digital del proyecto es la placa Nexys Video fabricada por Digilent. Su interior alberga una matriz de puertas lógicas programables (FPGA) de la familia Artix-7 de Xilinx (modelo XC7A200T). Este chip en particular es uno de los más potentes de su gama, contando en su interior con más de 215.000 celdas lógicas, 13 megabits de memoria RAM interna (*Block RAM*) y hasta 740 bloques físicos de procesamiento digital de señales (*DSP slices*). Además, a nivel de conectividad, la placa está diseñada para entornos exigentes de prototipado, incorporando conectores de expansión de alta densidad (FMC) y múltiples puertos PMOD, ideales para interconectar sensores médicos o periféricos personalizados mediante buses de comunicación estándar [12].



Figura 3.2. FPGA Nexys Video [9].

A nivel de diseño, el código VHDL de este proyecto se ha optimizado todo lo posible, en lugar de utilizar los multiplicadores DSP que ofrece el chip para realizar el pesado filtrado matemático, se ha reducido la lógica algorítmica a operaciones de desplazamiento binario rápido (*bit-shifting*). Gracias a esto, el sistema apenas consume una mínima fracción de los recursos físicos del chip.

Una versión más modesta y económica, como la Artix-7 35T, habría sido igual de válida y capaz de soportar todo el sistema en tiempo real. No obstante, se seleccionó la Nexys Video por su extrema robustez física, su generosa disponibilidad de puertos de entrada/salida (vitales para el cableado físico del bus SPI) y su fiabilidad garantizada en el laboratorio.

Todo el ecosistema de diseño, síntesis e implementación en hardware se ha gestionado mediante el entorno Vivado de AMD/Xilinx [12]. Este software ha permitido empaquetar los distintos bloques y garantizar que el enrutamiento interno de los cables de la FPGA cumpla estrictamente con las restricciones de tiempo (*timing constraints*). Este paso es absolutamente crítico para asegurar que el filtrado Wavelet y las máquinas de estados funcionen a altas velocidades de reloj de forma síncrona, sin sufrir retrasos lógicos ni cuellos de botella en la propagación de los datos.

3.2.2. Módulo de Adquisición Analógica ADS1293EVM

Las señales eléctricas que alcanzan la superficie de la piel son extremadamente débiles, del orden de los milivoltios, y están plagadas de ruido ambiental. Por tanto, no es físicamente viable conectarlas directamente a los pines de entrada de la placa digital. Para solucionar esto y acondicionar la señal con precisión médica, se ha utilizado como frontal analógico (*Analog Front-End*) el módulo de evaluación ADS1293EVM de Texas Instruments [10].



Figura 3.3. Foto del sensor ADS1293EVM [10].

La selección de este componente específico responde a una serie de requisitos técnicos necesarios:

- **Soporte para 5 electrodos:** Durante la fase de diseño inicial se barajó el uso de otros mó-

dulos de adquisición genéricos y de bajo coste. Sin embargo, fueron descartados porque solo ofrecían conexiones para 3 o 4 electrodos, lo cual los hacía incompatibles con el algoritmo vectocardiográfico, que exige 5 referencias físicas simultáneas. El chip ADS1293 no solo aporta las entradas exactas necesarias, sino que cuenta con amplificadores de instrumentación (INAs) de alta resolución para multiplicar la minúscula señal biológica antes de digitalizarla.

- **Generación autónoma del Terminal Central de Wilson (WCT):** Uno de los mayores atractivos técnicos de este chip es su enorme capacidad de enrutamiento interno. Para obtener el eje Z, el sistema necesita calcular el centro eléctrico del cuerpo. La gran ventaja de esta solución es que, si se configuran correctamente sus registros en la fase de inicialización, es capaz de calcular el WCT mediante hardware. Sus búferes internos promedian el voltaje de las extremidades y generan la referencia neutra de forma totalmente automática, liberando a la FPGA de procesar esta compleja carga matemática.
- **Conversión Analógico-Digital (ADC) de alta resolución:** Tras amplificar la señal, es necesario digitalizarla sin perder información biomédica. Para ello, el sensor asigna múltiples conversores analógico-digitales dedicados con una resolución médica de 24 bits a cada uno de los canales de medición. Esto garantiza que los tres ejes espaciales (X , Y , Z) se muestreen de forma estrictamente simultánea, eliminando cualquier desfase de tiempo entre ellos. Además, permite configurar su tasa de muestreo interna, entregando a la FPGA un flujo continuo de datos a una velocidad estable de 1066 Hz.
- **Fiabilidad clínica y seguridad eléctrica:** A diferencia de las soluciones genéricas descartadas, que no ofrecían garantías frente al ruido, los amplificadores de Texas Instruments integran filtros de grado médico que suprimen activamente las interferencias de la red eléctrica. Además, el módulo incorpora un diseño riguroso de aislamiento. En caso de una falla eléctrica o cortocircuito en la placa digital, el frontal analógico bloquea la corriente de retorno hacia los electrodos, garantizando que el paciente no sufra ninguna descarga y haciendo su uso completamente seguro.
- **Formato de placa de evaluación (EVM):** Manipular circuitos integrados médicos de montaje superficial (SMD) requiere soldaduras microscópicas y diseños de placa de circuito impreso (PCB) muy complejos para evitar diafonías y capacidades parásitas. Utilizar este formato comercial EVM resuelve el problema de raíz, entregando una plataforma estable y lista para operar en el laboratorio.

Nota de ingeniería: Es crucial destacar que esta placa de evaluación incorpora de fábrica un microcontrolador propio. Sin embargo, para usar la placa programable planteada, este microcontrolador se ha omitido y puenteado físicamente en el diseño. La FPGA se conecta de forma

directa a los pines del chip conversor analógico, asumiendo el rol de maestro absoluto, extrayendo las muestras puras a través del bus SPI y controlando cada parámetro del hardware.

3.2.3. Adquisición del Sensor mediante SPI

Una vez definido el hardware, el siguiente paso es establecer un canal de comunicación fiable entre la placa Nexys Video y el frontal analógico. Para ello, se ha programado en la FPGA un controlador maestro específico basado en el protocolo serie SPI (*Serial Peripheral Interface*). El objetivo de este módulo es inicializar el sensor médico tras el encendido cargando su mapa de registros y, posteriormente, extraer las lecturas tridimensionales de forma continua y sin pérdida de muestras.

A nivel de diseño físico, el controlador cuenta con un divisor de reloj ($CLK_DIV = 100$) que reduce la frecuencia nativa de la FPGA para generar una señal de sincronismo (`sclk`) de 1 MHz [12]. Aunque tanto la FPGA como el sensor soportan velocidades muy superiores, se ha fijado este límite para tener una mayor estabilidad y fiabilidad. Al interconectar ambas placas mediante cables de prototipado genéricos (cables Dupont), operar a frecuencias muy elevadas genera efectos de capacidad parásita que corrompen los datos. Trabajar a 1 MHz garantiza una transmisión inmune a las interferencias electromagnéticas que puedan darse.

3.2.4. Configuración Inicial y Mapa de Registros del Sensor

El chip ADS1293EVM no dispone de memoria no volátil. Por consiguiente, tras cada reinicio del sistema (`rst`), la FPGA está obligada a configurarlo enviándole una secuencia exacta de comandos. En el código VHDL, esta información se ha blindado dentro de un *array* optimizado de 34 bytes, distribuido en parejas lógicas de "Dirección-Valor".

Los registros críticos configurados para cumplir con los objetivos del TFG son los siguientes:

- **Enrutamiento de canales (`FLEX_CH1_CN`, `FLEX_CH2_CN`, `FLEX_CH3_CN`):** Configuran los multiplexores internos del sensor para asignar los electrodos a los canales de conversión. El canal 1 mide el eje horizontal (LA – RA); el canal 2 mide el eje vertical (LL – RA); y el canal 3 se asigna a la profundidad, registrando la diferencia entre el electrodo de la espalda y la referencia neutral.
- **Terminal Central de Wilson (`WILSON_EN1/2/3` y `WILSON_CN`):** Para obtener la tercera

dimensión (eje Z), el hardware necesita un punto neutro. Estos comandos enrutan las señales de las extremidades hacia unos *buffers* internos que promedian matemáticamente sus voltajes, generando el centro eléctrico del cuerpo que servirá de referencia al electrodo del pecho/espalda.

- **Frecuencia de muestreo (R2_RATE, R3_RATE_CH1/2/3):** Establecen el factor de diezmado de los filtros digitales internos. Se han programado para fijar una frecuencia de muestreo final (F_s) cercana a los 1066 Hz. Esta tasa permite la obtención de un volumen de datos perfectamente manejable por el algoritmo Wavelet.
- **Activación general (OSC_CN, CH_CNFG, CONFIG):** El registro OSC_CN enciende el oscilador digital del chip. CH_CNFG es vital, ya que habilita el modo de lectura por ráfaga (*Loop Readback*) para los tres canales de forma simultánea. Finalmente, CONFIG enciende los conversores analógico-digitales (ADCs) e inicia la captura continua del corazón.

Orden	Registro (Hex)	Descripción
1	[rst] Reinicio del Sistema	Reinicia chip.
2	Array Config. (34 bytes)	Carga datos FPGA.
3	FLEX_CH1_CN (0x01)	Canal 1 (LA-RA).
4	FLEX_CH2_CN (0x02)	Canal 2 (LL-RA).
5	FLEX_CH3_CN (0x03)	Canal 3 (Back-WCT).
6	WILSON_ENx/CN (0x0D-0x10)	Generar Ref. Neutral.
7	R2_RATE, R3_RATE_CHx (0x20-0x23)	Frecuencia (1066 Hz).
8	OSC_CN (0x11)	Activar oscilador.
9	CH_CNFG (0x12)	Habilitar Loop Readback.
10	CONFIG (0x13)	Activar ADCs, Conversión.

Figura 3.4. Flujo de inicialización de registros para el acondicionamiento biomédico.

3.3. Módulo SPI y Recepción de la Trama

El control de la comunicación se ha modelado íntegramente mediante una máquina de estados finita síncrona (FSM). Esta arquitectura garantiza que la transferencia de bits respete con precisión nanométrica los tiempos de retención (*hold times*) exigidos por el fabricante del sensor, evitando cualquier corrupción en el bus físico [10].

La máquina de estados se compone de seis etapas lógicas que operan en un ciclo continuo:

- **STARTUP:** Estado inicial tras el encendido. La máquina activa la selección de chip ($cs_b = '0'$) y comienza a escupir bit a bit la configuración por la línea `mosi` usando un registro de desplazamiento.

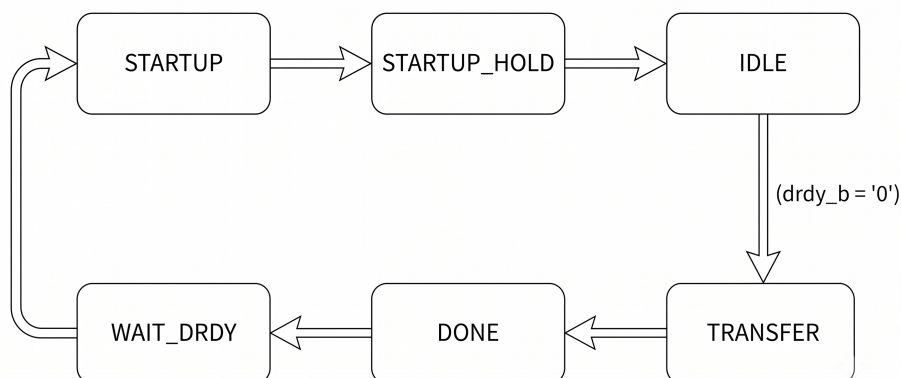


Figura 3.5. Máquina de estados síncrona para el control del bus SPI y la recepción de tramas.

- **STARTUP_HOLD:** Tras enviar un registro, el sistema levanta la línea ($cs_b = '1'$) y detiene la máquina durante 20 ciclos de reloj. Este retraso de seguridad forzado, es obligatorio para dar tiempo al chip analógico a procesar internamente el comando antes de recibir el siguiente. El ciclo se repite hasta vaciar el *array* de configuración.
- **IDLE:** Con el sensor configurado, la FPGA entra en reposo. Se dedica a monitorizar pasivamente el pin *drdy_b* (*Data Ready*). Cuando este pin cae a nivel bajo ('0'), el sensor avisa de que tiene una nueva lectura 3D lista. La máquina reacciona inmediatamente bajando *cs_b* y precargando el comando 0xB7.
- **TRANSFER:** Es el núcleo operativo. El comando 0xB7 dispara la lectura por ráfaga (*Loop Readback*). En lugar de malgastar ciclos de reloj recibiendo la dirección de cada canal byte a byte, la FPGA simplemente mantiene el reloj funcionando y el sensor escupe una trama continua de 72 bits (9 bytes). Al ocupar cada eje espacial exactamente 24 bits (3 bytes) de resolución médica, un contador clasifica la información entrante por el pin *mi_so* y la ensambla ordenadamente en los *buffers* de los canales X, Y y Z.
- **DONE:** Al capturar el noveno byte, se cierra la comunicación ($cs_b = '1'$). El hardware vuelca los *buffers* hacia los puertos de salida y emite un pulso de un ciclo de reloj en la bandera *d_valid*. Este pulso es el "disparo" que despierta al filtro Wavelet y al resto de la FPGA, indicando que hay datos frescos listos para ser operados.
- **WAIT_DRDY_HIGH:** Estado de bloqueo y seguridad. Como la FPGA procesa a 100 MHz y el sensor a 1066 Hz, la máquina se queda atrapada aquí esperando a que el pin *drdy_b* vuelva a subir a nivel alto ('1'). Esto previene que el hardware lea la misma muestra dos veces por error, retornando a IDLE de forma limpia para esperar el siguiente latido.

Para más información sobre el código fuente en VHDL de este módulo, consultar el [ANEXO A](#).

3.4. Unidad de Filtrado Wavelet 3D

Una vez que el controlador SPI extrae de forma segura los datos del sensor, estas muestras en bruto deben ser procesadas de manera eficiente. Como se detalló en el marco teórico, las señales biomédicas contienen ruidos musculares y desvíos respiratorios que empañan los puntos críticos del latido. Para solucionar esto en tiempo real, se ha diseñado en la FPGA la Unidad de Filtrado Wavelet 3D. Su propósito es recibir los tres canales de vectocardiografía (X, Y, Z) y aplicarles una descomposición matemática simultánea para aislar las frecuencias exactas asociadas al complejo QRS y a la onda T.

Para mantener la eficiencia energética y la continuidad en la arquitectura, este módulo utiliza un sistema de sincronización guiado por eventos (*data-driven*). La unidad de filtrado no está calculando constantemente en cada ciclo de reloj de la FPGA; en su lugar, permanece "dormida".^a la espera de que la bandera `d_valid` del módulo SPI se ponga en nivel alto. Al conectarse directamente a la entrada `sample_valid_in` de la Wavelet, este pulso actúa como un disparo que inicia instantáneamente el procesamiento en cadena dentro del hardware, garantizando que cada muestra médica sea filtrada en el mismo milisegundo en que llega.

3.4.1. Estructura Modular y Procesamiento en Paralelo

El código VHDL se ha estructurado siguiendo una jerarquía dividida en tres niveles de complejidad, lo que permite instanciar hasta 24 filtros matemáticos trabajando al mismo tiempo de manera ordenada:

- **Capa Superior (`wavelet_3d_transform`):** Actúa como un contenedor global. Instancia tres copias idénticas del bloque unidimensional, una para cada eje espacial. Al tratarse de hardware dedicado, estos tres ejes se procesan de forma estrictamente paralela, asegurando que no exista ni un solo nanosegundo de desfase temporal entre las coordenadas tridimensionales del corazón.
- **Capa Intermedia (`wavelet_1dimension`):** Dentro de cada eje, este bloque implementa un banco de filtros en cascada compuesto por 8 etapas consecutivas (`STAGE_1` a `STAGE_8`). Los datos suavizados de una etapa alimentan directamente a la siguiente, extrayendo la información de la señal a diferentes niveles de resolución.

- **Capa Base (wavelet_phase):** Es el motor matemático elemental. Contiene la lógica aritmética y una línea de retardo basada en registros de memoria (*delay line*) para almacenar el historial de muestras de la señal.

Un aspecto de diseño a destacar de esta arquitectura es la gestión de la sincronización de salida. Dado que los tres ejes operan bajo el mismo reloj, el sistema utiliza el canal Z como árbitro final (`if rdyz_s3 = '1'`). Cuando el eje de profundidad confirma que ha terminado sus operaciones lógicas, el módulo eleva las banderas generales, garantizando que las salidas se entreguen al siguiente bloque empaquetadas como un único vector tridimensional.

3.4.2. Escala del filtro Wavelet

El banco de filtros de cada dimensión descompone la señal en 8 niveles de detalle. Sin embargo, para cumplir con los objetivos de delineación automatizada, el código VHDL ha sido optimizado para extraer y exponer hacia el exterior únicamente dos escalas específicas, descartando el resto para simplificar la lógica posterior [19]:

- **Escala 3 (d_x_s3, d_y_s3, d_z_s3):** Se obtiene al finalizar la tercera etapa de filtrado (STAGE_3). Esta escala aísla las frecuencias altas. Fisiológicamente, coincide a la perfección con la descarga rápida de energía del complejo QRS. Al aislar esta escala, el sistema ignora por completo los ruidos lentos de la respiración y resalta el pico R con una gran nitidez.
- **Escala 8 (d_x_s8, d_y_s8, d_z_s8):** Se extrae en la última etapa del banco (STAGE_8). Tras pasar por ocho filtros consecutivos, los componentes rápidos de la señal se han eliminado, dejando únicamente frecuencias muy bajas. Esta escala dibuja una curva limpia y sumamente suave, ideal para monitorizar el relieve de la onda T sin que el ruido muscular confunda a la máquina de estados.

3.4.3. Optimización Aritmética y Algoritmo à trous

Uno de los grandes desafíos del módulo `wavelet_phase` es implementar matemáticas complejas sin penalizar el rendimiento de la FPGA. El filtro ejecuta una función polinomial de suavizado conocida como *B-Spline* cúbico, cuyos coeficientes fijos son [1, 4, 6, 4, 1] [19]. Además, extrae el nivel de detalle restando la señal suavizada a la señal original diferida (`regs(0) - regs(4*m)`).

En lugar de utilizar multiplicadores físicos dentro del chip (bloques DSP), elevando el consumo de potencia, el diseño aprovecha la aritmética binaria:

- **Desplazamiento de bits (*Bit-Shifting*):** La operación de multiplicar por 4, se optimiza desplazando el bus de datos dos posiciones a la izquierda (`shift_left`), y la operación de multiplicar por 6 se descompone sumando un desplazamiento de dos posiciones y uno de una posición ($4 + 2$).
- **Protección y Normalización:** Antes de sumar, el hardware expande dinámicamente el bus de 24 bits a 29 bits (`resize`) para evitar desbordamientos aritméticos (*overflows*). Tras la suma, la división final por 16 para normalizar la señal se realiza truncando los bits inferiores directamente en el bus de asignación y ahorrando el uso de operaciones más complejas o DSP. (`sum_h(27 downto 4)`).

Finalmente, para que este mismo filtro $[1, 4, 6, 4, 1]$ pueda analizar ondas cada vez más anchas en las etapas superiores, se emplea el parámetro genérico m (factor de escala), implementando el **algoritmo Wavelet à trous (con huecos)** [19].

En lugar de aplicar el filtro sobre muestras consecutivas, la lógica espacia las lecturas multiplicando el índice de la memoria por m . En la etapa 1 ($m = 1$), analiza datos contiguos; pero en la etapa 8 ($m = 8$), el filtro lee las posiciones espaciadas 0, 8, 16, 24 y 32. Esta decisión es la razón por la cual la línea de retardo del VHDL tiene un tamaño exacto de 33 posiciones (`array (0 to 32)`). Gracias a este algoritmo, la ventana temporal del filtro se expande exponencialmente para capturar la lenta onda T, consiguiendo utilizar exactamente el mismo consumo de memoria RAM y el mismo número de sumadores en la etapa 1 que en la etapa 8.

Para más información sobre el código fuente en VHDL de este módulo, consultar el [ANEXO A](#).

3.5. Detector de Eventos

Con la señal cardíaca tridimensional ya filtrada y descompuesta en sus respectivas escalas Wavelet, el sistema necesita un bloque de decisión para identificar el inicio, el pico y el final exacto de las ondas. Esta responsabilidad recae sobre la Unidad de Detector de Eventos, el núcleo crítico encargado de procesar la señal en tiempo real para extraer los intervalos temporales RR y RT.

El mayor desafío de este bloque no es la velocidad, sino la naturaleza caótica de la biología. El corazón de un paciente no produce impulsos idénticos: la respiración, el movimiento o leves

desvíos en los electrodos hacen que la amplitud de la señal fluctúe constantemente. Si se programaran umbrales fijos de detección, el sistema fallaría a los pocos segundos. Por este motivo, el hardware incorpora una lógica de umbrales dinámicos y adaptativos, recalculándose automáticamente en cada latido para perseguir el ritmo real del paciente [22].

Para implementar esta adaptabilidad, el sistema maneja dos registros de memoria paralelos: un umbral máximo positivo (`mem_pmax`) y un umbral mínimo negativo (`mem_pmin`). Utilizar esta doble barrera simultáneamente es estrictamente necesario porque el vector cardíaco se mueve en tres dimensiones y, dependiendo del eje espacial analizado (X, Y, Z) o de la morfología del paciente, la onda eléctrica puede manifestarse como un pico que se dispara hacia arriba o como un valle profundo que cae hacia abajo. Al rastrear el histórico en ambas direcciones, el hardware garantiza capturar la energía del latido independientemente de su polaridad. Además, funcionan de manera dinámica, dividiendo estos umbrales a la mitad de forma instantánea si detecta que la fuerza de la señal del paciente ha decaído. En caso de que la señal del paciente se vuelva más débil, este sistema dinámico se ajusta a la nueva intensidad de la señal.

Otro aspecto fundamental del diseño es la constante de "Cero Virtual" (`VIRTUAL_ZERO`). En el mundo digital, debido a los mínimos restos de ruido de fondo, una señal matemática casi nunca descansa en el cero absoluto. Al definir una franja de tolerancia cercana a cero, el hardware puede sentenciar con total certidumbre cuándo una deflexión eléctrica ha concluido su recorrido y ha regresado a la línea de base, esquivando así falsos disparos por oscilaciones residuales.

3.5.1. Máquina de Estados para la Detección del QRS

El módulo `qrs_detector` opera sobre la escala rápida 3 de la Transformada Wavelet. Para asegurar que el hardware no pierda la polaridad del latido (es decir, si la descarga eléctrica apunta hacia arriba o hacia abajo en el espacio 3D), la máquina monitoriza simultáneamente un umbral histórico positivo (`mem_pmax`) y uno negativo (`mem_pmin`).

Para proporcionar robustez al sistema frente a caídas drásticas de señal, el bloque incluye un contador de seguridad (`cnt_rr`). Si transcurren 3 segundos completos (`TIME_3S`) sin detectar ningún latido, el hardware asume que la señal se ha debilitado o existe una anomalía extrema. En ese instante, aplica un desplazamiento binario a la derecha (`shift_right`) dividiendo a la mitad sus umbrales y forzando una mayor sensibilidad para captar ondas débiles y retomar la búsqueda.

El comportamiento secuencial para cazar el complejo QRS se divide en cinco etapas (FSM):

- **Etapa 1 (Búsqueda por umbral):** El sistema limpia sus registros y compara la muestra entrante con los umbrales adaptativos. Si la señal rompe el límite positivo o negativo, el hardware memoriza la dirección del impacto (`qrs_n`) y avanza al siguiente estado.
- **Etapa 2 (Cruce por cero inicial y cierre del RR):** La máquina realiza el seguimiento de esta primera curva y almacena su pico máximo. La etapa concluye en el milisegundo exacto en que la curva invierte su pendiente y atraviesa el "Cero Virtual". Esta transición vuelca el contador temporal hacia la salida `time_rr`, cerrando la medición del latido anterior y reiniciando el reloj para el siguiente.
- **Etapa 3 (Búsqueda del rebote simétrico):** Por la naturaleza matemática del filtro Wavelet, un complejo QRS puro no es una simple "montaña", sino una onda bifásica que genera un rebote en el espectro opuesto. En este estado, el sistema espera a que la señal cruce al lado contrario y supere el umbral inverso, confirmando que es un latido real y no un pico de ruido muscular.
- **Etapa 4 (Cruce final y recalibración):** Se vigila el segundo pico. Cuando la señal pierde fuerza y entra de forma definitiva en el "Cero Virtual", el latido se da por concluido. En este instante, el hardware recalcula los umbrales adaptativos para el próximo ciclo dividiendo los picos máximo y mínimo capturados a la mitad; operación que se ejecuta aplicando un desplazamiento binario de una posición a la derecha (`shift_right`). En el mismo ciclo de reloj, dispara la bandera `qrs_detected` para avisar al resto de la FPGA.
- **Etapa 5 (Ceguera refractaria):** Tras la enorme descarga de energía de un QRS, la señal sufre ecos eléctricos. Para evitar que el sistema se auto-engañe detectando el mismo latido dos veces, la máquina entra en un bloqueo ciego de 40 milisegundos (`TIME_40MS`). Al superar este margen de seguridad, retorna a la Etapa 1.

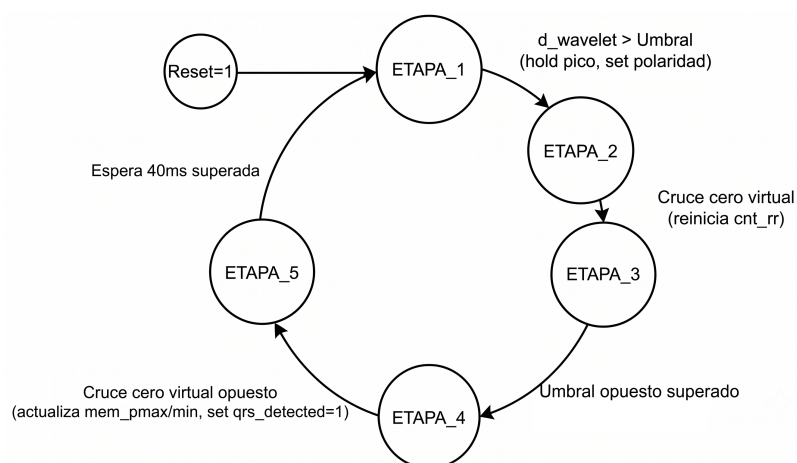


Figura 3.6. Máquina de estados síncrona para la detección y actualización del complejo QRS.

3.5.2. Máquina de Estados para la Detección de la Onda T

El objetivo del módulo `t_detector` es localizar el final matemático de la onda T operando sobre la escala lenta 8. La complejidad de ello reside en que la onda es de carácter ancha, suave y propensa a difuminarse. Además, su posición no es fija, si el paciente está en reposo, la onda T se encuentra más alejada del complejo QRS. Si está haciendo deporte, por norma general se encuentra más cerca del complejo QRS.

Para resolver este problema sin colapsar la placa con divisiones complejas, el módulo implementa una **ventana de espera dinámica** basada en desplazamientos de bits. Al recibir el disparo del QRS, el hardware toma el tiempo total del latido anterior (`rr_interval`) y suma matemáticamente su octava y su dieciseisava parte ($RR/8 + RR/16$) mediante la operación `shift_right` de 3 y 4 posiciones. Esto crea un periodo de ceguera de un 18,75 % del latido, que representa un porcentaje seguro que se estira y encoge proporcionalmente al ritmo cardíaco real del sujeto (manteniendo un suelo de seguridad de 170 ms para taquicardias extremas) [21].

La máquina encargada de delinear la onda T se mueve por estas cinco etapas:

- **Etap 1 (Espera proporcional):** La FSM se mantiene congelada hasta que el contador supera el límite dinámico (`limit_window`). Esto asegura que el sistema ignore por completo los ruidos iniciales del segmento ST, empezando a detectar únicamente cuando la onda T está emergiendo físicamente en el miocardio.
- **Etap 2 (Captura del pico primario):** Se vigila la escala 8. El sistema busca que la señal supere el 50 % de su umbral histórico registrado (`shift_right` de `mem_pmax`). Al superarlo, se guarda el pico y se espera a que la curva caiga al "Cero Virtual".
- **Etap 3 (Captura del rebote lento):** Al igual que el QRS, la onda T filtrada refleja un suave rebote en el espectro opuesto. En este caso es necesario que la señal cruce el cero y alcance un 25 % de profundidad en el umbral contrario, garantizando que se está siguiendo la morfología completa y no una deriva respiratoria. Este valor del 25 % se debe a que al analizar diversas bases de datos y aplicar la escala 8, se ha apreciado que la onda que rebota en la onda T acostumbra a tener la mitad de fuerza.
- **Etap 4 (Delineación exacta del RT y recalibración):** Este es el milisegundo más crítico del módulo. La máquina monitoriza el ascenso de esa última curva. En el instante exacto en que la pendiente se aplan y se estabiliza dentro del "Cero Virtual", significa médicamente que las células han recuperado su carga. El tiempo se congela, cerrando la medición del intervalo RT. Simultáneamente, el hardware recalcula los umbrales para el latido siguiente dividiendo los picos capturados a la mitad mediante un desplazamiento binario (`shift_right`). A

diferencia del QRS, en esta onda de baja amplitud se aplica un límite mínimo de seguridad (150.000); si el nuevo umbral calculado es inferior a esta barrera matemática, el sistema lo satura a dicho valor para evitar volverse hipersensible al ruido de fondo antes de dar paso a la etapa 5 [14].

- **Etapa 5 (Validación cruzada):** Como protección final, el hardware cruza datos entre máquinas. Si el ruido ha hecho que los umbrales de la onda T crezcan hasta superar la mitad del umbral del QRS (algo fisiológicamente imposible), la máquina los resetea forzosamente a un valor seguro. Finalmente, se activa la señal `t_detected <= '1'`, indicando que el intervalo RT está listo para ser diagnosticado. Después de esto, se reinicia la maquina de estados, regresando a la etapa 1 para esperar nuevamente la detección de la onda R.

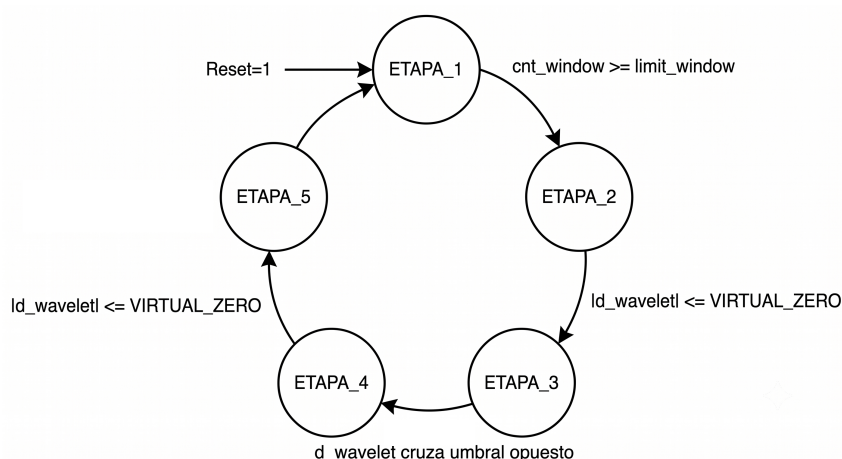


Figura 3.7. Máquina de estados para la delineación adaptativa y localización del final de la onda T.

Para analizar la estructura a nivel de registros, consultar el código VHDL completo en el [ANEXO A](#).

3.6. Módulo Puente

La arquitectura modular desarrollada hasta este punto procesa la actividad eléctrica del corazón desde una perspectiva tridimensional, manteniendo líneas de cálculo independientes para los ejes espaciales X, Y y Z. Sin embargo, para que el sistema pueda evaluar patologías o enviar datos estructurados hacia el ordenador, es necesario unificar estas tres fuentes de información en eventos de sistema únicos. Con este propósito se introduce el módulo `detection_bridge` (Módulo Puente), una unidad encargada de centralizar las detecciones de los tres ejes, aplicar un criterio de validación y calcular la duración en milisegundos de los intervalos RR y RT.

Añadir este bloque intermedio responde a dos necesidades de diseño principales:

- **Dispersión temporal y espacial:** Debido a la orientación del vector eléctrico del corazón, la energía de un latido puede registrarse con mayor amplitud en un eje (por ejemplo, el X) y de forma más débil o ligeramente desplazada en el tiempo en otro (como el Z). Si la FPGA enviara los datos directamente desde los detectores individuales, el sistema de recepción obtendría tres mediciones distintas para un mismo latido cardíaco.
- **Latencia del procesamiento digital:** El procesamiento de la señal a través de las ocho etapas del banco de filtros Wavelet y las máquinas de estados introduce un retardo en el flujo de datos. El módulo puente se encarga de ejecutar una calibración analítica para compensar estos retardos propios del hardware, asegurando que los intervalos calculados se correspondan con el tiempo real.

Para resolver estas cuestiones, el código VHDL implementa de forma secuencial cuatro funcionalidades clave:

3.6.1. Ventanas de Expansión Temporal (Pulse Stretching)

Cuando un detector localiza el pico de una onda, emite un pulso que dura un único ciclo de reloj de la FPGA. Debido al ligero desfase con el que la onda llega a los diferentes electrodos espaciales, estos pulsos no suelen coincidir en el mismo instante de reloj.

Para solucionar este problema de sincronización, el puente implementa una técnica de expansión temporal (*Pulse Stretching*) controlada por contadores internos (`cnt_qrs` y `cnt_t`). Al recibir un pulso en cualquier eje, la lógica activa una bandera de presencia (`qrs_x_str <= '1'`) y arranca un contador adaptado a la frecuencia de muestreo del sistema. Esta ventana se ha fijado en 30 muestras (equivalente a 30 milisegundos). Al mantener activo el pulso de detección durante este margen de tiempo, el hardware permite que las detecciones de los otros ejes se solapen temporalmente y puedan ser comparadas.

3.6.2. Sistema de Votación por Mayoría (Majority Voting)

Una vez que las detecciones individuales han sido expandidas, el módulo aplica una regla de votación por mayoría (`qrs_voted` y `t_voted`). La ecuación lógica determina que una onda se

considera válida si al menos dos de los tres ejes espaciales coinciden en su detección dentro de la ventana de 30 ms (es decir, si se validan las parejas X-Y, X-Z o Y-Z).

Esta funcionalidad proporciona al sistema mayor robustez frente a los falsos positivos. Si se produce una interferencia electromagnética o un artefacto de movimiento que afecte únicamente a un electrodo (por ejemplo, el eje X), el detector de ese eje podría activarse. Sin embargo, al no registrarse réplica en los ejes Y o Z, el sistema de votación descarta el evento, evitando alertas erróneas.

3.6.3. Cálculo Numérico y Conversión a Milisegundos

Para medir la distancia temporal entre las ondas, el puente mantiene dos contadores independientes: `cnt_rr`, que mide el tiempo total entre latidos sucesivos, y `cnt_t`, que cuantifica la duración de la repolarización ventricular desde el inicio del QRS (`rt_busy <= '1'`). Estos contadores incrementan su valor con cada nueva muestra validada por el módulo de adquisición (`d_valid = '1'`).

Dado que estos contadores registran el número de muestras y no unidades de tiempo estándar, el módulo realiza una conversión aritmética utilizando la frecuencia de muestreo del sistema (`FS_HZ`). Al multiplicar las muestras acumuladas por 1000 y dividir las por la frecuencia, el hardware transforma los registros directamente en valores en milisegundos (`rr_ms_i` y `rt_ms_i`). Esta operación se realiza empleando buses con signo de 24 bits, proporcionando el rango necesario para evitar desbordamientos (*overflows*) de memoria.

3.6.4. Calibración Analítica de Retardos

La última funcionalidad de este bloque consiste en la compensación de la latencia de procesamiento. A través de las simulaciones en Vivado, se determinó el retardo temporal fijo que introduce cada etapa del filtro Wavelet: la detección del QRS (en la escala 3) presenta un retardo de 83 milisegundos (`DELAY_QRS_MS`), mientras que la detección de la onda T (en la escala 8) acumula una latencia de 389 milisegundos (`DELAY_T_MS`).

El puente compensa estos retrasos dependiendo del intervalo analizado:

- **Intervalo RR:** No requiere calibración. Dado que tanto el primer pico R como el segundo sufren el mismo retardo de 83 ms en el filtrado, al medir la diferencia de tiempo entre ambos

eventos la latencia se anula matemáticamente ($83\text{ ms} - 83\text{ ms} = 0$). La medición relativa se mantiene inalterada.

- **Intervalo RT:** Requiere compensación. Puesto que la onda T se detecta con un retraso mayor que el QRS, el contador `cnt_rt` acumula un error temporal en la medición. Para corregir este desfase, el puente aplica una operación aritmética de ajuste: resta las muestras correspondientes al retardo de la onda T y suma las del retardo del QRS (`cnt_rt - SAMPLES_T_DELAY + SAMPLES_QRS_DELAY`).

Una vez compensada la latencia, el módulo unifica las salidas emitiendo los pulsos correspondientes (`qrs_unified` y `t_unified`) junto con los valores calibrados en milisegundos. Esto asegura que el siguiente bloque de evaluación de patologías reciba parámetros temporales precisos.

Para analizar las sentencias lógicas, consultar el [ANEXO A](#).

3.7. Lógica de Diagnóstico y Alarmas

Una vez unificados y calibrados los datos en el bloque puente, la etapa final del procesamiento interno en la FPGA consiste en evaluar los intervalos calculados. Esta función recae sobre el módulo `detection_alarm`, responsable de monitorizar de forma continua la duración en milisegundos de los intervalos RR y RT para activar, de forma selectiva y en tiempo real, las banderas de alerta clínica que se transmitirán al sistema de visualización.

A nivel de diseño hardware, este módulo introduce una estrategia de validación adicional: la inclusión de registros de persistencia (`tachy_persist_reg`, `brady_persist_reg`, etc.). En la práctica clínica, activar una alarma médica ante un único latido anómalo puede generar falsos positivos derivados de artefactos de movimiento o variaciones transitorias aisladas. Para evitarlo, el módulo exige mediante hardware que la anomalía se mantenga presente durante al menos dos latidos consecutivos (`v_persist >= 2`). Si se detecta un evento anómalo pero el ciclo posterior retorna a parámetros normales, el contador de persistencia se reinicia a cero, asegurando que las alarmas solo se disparen ante condiciones sostenidas.

El comportamiento del hardware para la evaluación de las cinco patologías de estudio se detalla a continuación:

3.7.1. Detección de Asistolia por Tiempo Continuo

A diferencia del resto de patologías, cuya evaluación depende de la detección de un latido, la asistolia se define por la ausencia de actividad eléctrica. Por ello, el hardware implementa un contador continuo (`asystole_cnt`) que incrementa su valor con cada muestra validada (`d_valid = '1'`).

El umbral de alerta se ha fijado en tres segundos de inactividad ($3 \times \text{FS_HZ}$). Si el contador alcanza el límite de las 3000 muestras sin reiniciarse, la FPGA activa de forma asíncrona la señal `alarm_asystole <= '1'`. Por el contrario, cada pulso unificado de un latido (`qrs_unified = '1'`) reinicia el contador a cero, manteniendo la alerta desactivada durante el ritmo normal.

3.7.2. Diagnóstico de Taquicardia y Bradicardia

Para evaluar la frecuencia cardíaca, el módulo compara el intervalo `rr_interval_ms` recibido del puente con dos constantes predefinidas en el diseño:

- **Taquicardia (`THRESHOLD_TACHY = 600`):** Un intervalo RR inferior a 600 milisegundos equivale a una frecuencia cardíaca en reposo superior a 100 latidos por minuto. La detección de dos ciclos consecutivos por debajo de este umbral activa la alarma.
- **Bradicardia (`THRESHOLD_BRADY = 1200`):** Un intervalo RR superior a 1200 milisegundos indica un ritmo inferior a 50 latidos por minuto. De igual forma, se requiere la confirmación en dos ciclos sucesivos para activar la salida lógica.

3.7.3. Algoritmo de Arritmia Adaptativa por Desplazamiento

La detección de arritmias se centra en la inestabilidad temporal entre latidos. Para cuantificar esta variabilidad en el hardware, el código calcula en cada ciclo la diferencia absoluta (`diff_rr`) entre la duración del latido actual (`rr_interval_ms`) y la del latido inmediatamente anterior (`last_rr_ms`).

Para determinar si la variación es suficiente, se implementa una regla adaptativa proporcional al ritmo del paciente: se evalúa si la diferencia supera el 25 % de la duración del último latido. Esta comprobación matemática se optimiza en la FPGA mediante un desplazamiento lógico de dos posiciones a la derecha (`shift_right(last_rr_ms, 2)`), lo que equivale a una división

exacta por 4 sin consumir recursos lógicos de división. Si la variación temporal excede este límite durante dos latidos consecutivos, se activa la alarma de arritmia.

3.7.4. Prevención de Muerte Súbita

La prevención del riesgo de muerte súbita, asociada a una prolongación anómala de la repolarización ventricular (Síndrome de QT Largo), es una de las evaluaciones más críticas del sistema. Para asegurar un diagnóstico robusto, el código implementa dos etapas de seguridad secuenciales:

- **Filtro de Validación Estructural (`v_r_since_t`):** El sistema audita la coherencia del ciclo cardíaco verificando que ocurra exactamente una contracción (onda R) antes de la repolarización (onda T). Si un artefacto de ruido rompe este patrón estructural (por ejemplo, detectando dos ondas R sucesivas sin una T intermedia), el módulo clasifica la lectura temporal como no fiable, reinicia la persistencia a cero y bloquea la evaluación para evitar un falso positivo.
- **Regla de la Mitad Clínica Adaptativa:** Con la estructura validada, el sistema evalúa la posición de la onda T. El criterio médico general establece que el intervalo de recuperación (RT) no debe exceder la mitad del ciclo cardíaco total. Para dotar al diseño de un margen preventivo de seguridad, la FPGA calcula un límite de disparo más estricto equivalente al 43.75 % del ciclo. Este porcentaje se implementa algebraicamente en VHDL restando a la mitad del latido una dieciseisava parte del mismo mediante la operación `shift_right(last_rr_ms, 1) - shift_right(last_rr_ms, 4)`.

Si el intervalo RT medido excede este umbral adaptativo durante dos ciclos consecutivos, se asume que las células miocárdicas están operando en la ventana de vulnerabilidad eléctrica. En consecuencia, la FPGA eleva la bandera `alarm_sudden_death <= '1'`, emitiendo una alerta temprana antes del posible desarrollo de un cuadro de fibrilación ventricular irreversible [14].

Para analizar el código fuente en detalle, consultar el [ANEXO A](#).

3.8. Módulo de Comunicación UART y Protocolo de Trama

Una vez que el hardware ha capturado los datos, filtrado las ondas y evaluado las posibles patologías, se alcanza la etapa final de la arquitectura en la FPGA: la exportación de la información. Para visualizar las gráficas y las alertas en tiempo real, la placa Nexys Video debe transmitir este flujo de datos hacia el ordenador. Con este propósito se ha diseñado el módulo `uart_controller`, encargado de empaquetar la información y transmitirla utilizando el protocolo serie UART (Universal Asynchronous Receiver-Transmitter).

El controlador se ha configurado con un reloj base de 100 MHz y una velocidad de transmisión de 115200 baudios (bits por segundo). Esta tasa de comunicación ofrece un equilibrio adecuado entre velocidad y estabilidad en la conexión USB-Serie, garantizando un flujo continuo de datos hacia el entorno de Python sin saturar los búferes de recepción del sistema operativo.

3.8.1. Estrategia de Throttling y Empaquetado de la Trama

El principal reto de este módulo es la gestión de las diferentes frecuencias de trabajo. El sensor ADS1293EVM genera una nueva muestra tridimensional a una tasa de 1066 Hz (`d_valid = '1'`). Transmitir la totalidad de estas muestras por la línea UART a 115200 baudios saturaría el canal físico, ya que la generación de bits supera la capacidad de transmisión del estándar.

Para solucionar esto, el diseño implementa una estrategia de submuestreo controlado (*Throttling*) en el hardware. La FPGA evalúa el estado de la transmisión mediante la bandera `sending_frame`. Si el módulo UART está ocupado enviando datos, el sistema descarta el empaquetado de las muestras intermedias. En el instante en que la línea se libera, la FPGA aprovecha el siguiente pulso `d_valid` para capturar una muestra actualizada y almacenarla en el registro de transmisión (`frame_buffer`). Esta técnica reduce la tasa de envío efectiva a unos 311 paquetes por segundo, una velocidad que la UART gestiona sin pérdidas y que es suficiente para garantizar una visualización fluida.

No obstante, esta estrategia de descarte presenta un riesgo de diseño: si la detección de un evento clínico (QRS u onda T) ocurre en una muestra que la UART descarta, la alerta no llegaría al sistema receptor. Para evitar esta pérdida de información, el código incorpora un mecanismo de retención de eventos (`qrs_aux` y `t_aux`). Cuando las máquinas de estados detectan una onda, el pulso de un ciclo de reloj se captura y almacena de forma persistente en un registro auxiliar.

El evento se mantiene retenido en el hardware hasta que la UART empaqueta la siguiente trama, momento en el cual inyecta la alerta en el paquete y reinicia el registro, asegurando la transmisión del 100 % de los eventos cardíacos.

Para estructurar la comunicación, el controlador organiza los datos en tramas fijas de 37 bytes, distribuidos de la siguiente manera:

- **Cabecera de Sincronización (Bytes 0 y 1):** Valores fijos 0xAA y 0xBB. Actúan como marcadores de inicio, permitiendo al software receptor identificar el comienzo de un paquete válido y descartar posibles bits corruptos.
- **Señales en Bruto (Bytes 2 al 10):** Datos originales del sensor para los tres canales (X , Y , Z). Al requerir 24 bits de resolución por canal, ocupan 3 bytes por eje.
- **Componentes Wavelet Filtrados (Bytes 11 al 28):** Resultados de las etapas de filtrado. Se destinan 9 bytes para la escala rápida 3 (QRS) y 9 bytes para la escala lenta 8 (onda T), permitiendo la representación paralela de las señales.
- **Intervalos Calibrados (Bytes 29 al 34):** Duración en milisegundos calculada por el módulo puente, asignando 3 bytes para el intervalo RR y 3 bytes para el RT.
- **Byte de Control y Alarmas (Byte 35):** El bit más significativo (Bit 7) funciona como bandera de validación fija en '1'. Los bits 6 y 5 transmiten las banderas retenidas de detección de ondas. Los cinco bits inferiores (Bits 4 al 0) indican el estado de las alarmas clínicas (taquicardia, bradicardia, arritmia, asistolia y riesgo de muerte súbita).
- **Fin de Trama (Byte 36):** Almacena el valor 0x0A (salto de línea en ASCII). Esta decisión optimiza el código en Python, permitiendo capturar el paquete completo mediante una única instrucción de lectura de línea.

3.8.2. Máquina de Estados de Transmisión UART

Con los 37 bytes almacenados en el búfer de transmisión, el sistema debe serializarlos, enviándolos bit a bit a través del pin de salida (tx). Para gestionar esta sincronización, el módulo emplea una máquina de estados finita compuesta por cuatro etapas:

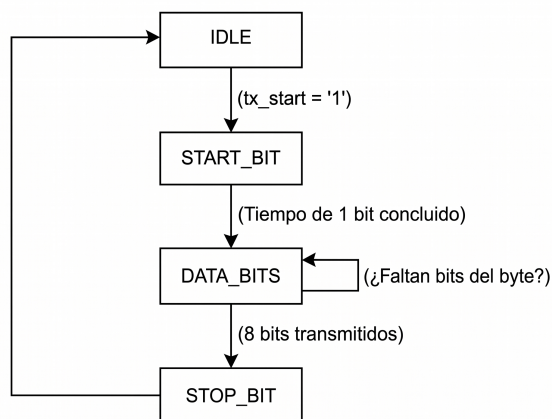


Figura 3.8. Máquina de estados síncrona para la transmisión UART.

- **IDLE (Reposo):** La línea física se mantiene en nivel lógico alto ('1'). La máquina permanece en este estado hasta que la lógica de empaquetado activa la señal `tx_start`.
- **START_BIT (Bit de Inicio):** La máquina sitúa la línea física en nivel bajo (`tx <= '0'`). Este estado se mantiene durante el tiempo definido por el contador de división de frecuencia (`CLKS_PER_BIT`), garantizando que la duración del pulso se ajuste al estándar de 115200 baudios.
- **DATA_BITS (Transmisión del Byte):** El sistema itera sobre el índice de bit (`bit_index`) y transfiere secuencialmente el contenido del byte al cable, comenzando por el bit menos significativo. El contador de reloj asegura el mantenimiento de cada bit durante el periodo temporal exigido.
- **STOP_BIT (Bit de Parada):** Tras emitir los 8 bits de datos, la línea retorna a nivel alto (`tx <= '1'`) durante el tiempo correspondiente a un bit, restableciendo el bus para el envío del siguiente carácter.

Al finalizar el bit de parada, se evalúa el contador de trama (`byte_index`). Si restan bytes por enviar, el sistema incrementa el índice y retorna a la secuencia de inicio. Una vez completado el envío del byte 37, la señal `sending_frame` se desactiva, liberando el módulo para la siguiente captura de datos.

Para más información sobre el código VHDL de transmisión, consultar el [ANEXO A](#).

3.9. Interfaz Gráfica de Telemetría (Python)

La etapa final de la arquitectura consiste en la recepción y visualización de los datos. Para traducir el flujo binario en representaciones gráficas e indicadores clínicos en tiempo real, se ha desarrollado una aplicación de telemetría en Python. El software se apoya en librerías especializadas como Matplotlib para la renderización gráfica y NumPy para el cálculo matricial eficiente de los arrays de datos [24].

La aplicación soporta dos modalidades de adquisición, compartiendo el motor de renderizado y la lógica de decodificación para asegurar la consistencia entre el entorno de simulación y las pruebas físicas de laboratorio.

3.9.1. Sincronización y Algoritmo de Decodificación

El algoritmo de recepción implementa un mecanismo de ventana deslizante para asegurar la integridad de los datos. El software analiza el flujo buscando la cabecera (0xAA y 0xBB) y verificando que la posición del byte 37 coincida con el salto de línea (0x0A). Si el canal presenta ruido y los marcadores no coinciden, el puntero avanza secuencialmente para resincronizar la lectura de forma automática.

Tras validar una trama, se procede a su decodificación:

- **Conversión con Signo:** Las señales biomédicas (Raw y Wavelet) se transmiten en formato de 24 bits. El código implementa una función de extensión de signo (`decode_24bit_signed`) que fusiona los tres bytes mediante desplazamientos lógicos y aplica aritmética de complemento a dos en caso de detectar un valor negativo ($val - 0x1000000$).
- **Conversión sin Signo:** Los intervalos de tiempo (RR y RT) se decodifican mediante `decode_24bit_unsig` concatenando los bytes en un entero absoluto.
- **Desglose de Alarmas:** Mediante operaciones de desplazamiento y enmascaramiento de bits sobre el byte 35, se aíslan los indicadores booleanos de presencia de ondas y activación de patologías.

3.9.2. Modalidades de Adquisición

El manejo de la memoria y la captura de datos se adapta al modo de ejecución seleccionado:

- **Modo Simulación (Análisis Diferido):** El script lee un archivo de texto previamente generado (`uart_prueba_3d.txt`), almacenando la totalidad de los datos en estructuras estáticas de NumPy. La animación recorre estos arrays mediante intervalos fijos, permitiendo reproducir las pruebas de laboratorio de forma acelerada. Ver Capítulo 4 para mas información.
- **Modo Real-Time (Sensor en Vivo):** Se establece comunicación serie con el hardware a través de la librería `pySerial`. Para garantizar un buen rendimiento, el puerto se configura como no bloqueante (`timeout=0`), evitando la paralización de la interfaz si se producen retrasos en el bus. Para evitar el desbordamiento de memoria tras horas de uso, los datos se almacenan en estructuras de cola circular (`deque`) de tamaño constante (`ANCHO_VENTANA = 400`). Al ingresar una nueva muestra, se descarta automáticamente la más antigua.

3.9.3. Diseño del Cuadro de Mandos (Dashboard)

La interfaz emplea un tema oscuro que mejora el contraste de las alertas clínicas. La distribución espacial se estructura mediante una matriz (`GridSpec`) dividida en tres áreas operativas:

- **Gráficos Temporales 2D:** Representan la evolución de los ejes X, Y y Z a lo largo del tiempo. Para mantener una alta tasa de refresco, los objetos gráficos no se redibujan en cada iteración; únicamente se actualiza el puntero de los arrays internos mediante la instrucción `set_data`.
- **Vectorcardiograma 3D:** Un plano de proyección tridimensional toma las coordenadas actualizadas de los tres ejes espaciales para trazar el bucle de rotación del vector eléctrico del corazón en vivo.
- **Marcadores de detección, intervalos y alarmas:** Las delineaciones de ondas QRS y T se representan mediante colecciones de líneas (`LineCollection`), una técnica que optimiza la carga de la CPU frente al dibujo de entidades lineales individuales. En el panel inferior, las mediciones de los intervalos se muestran en formato numérico y los cuadros de texto correspondientes a las patologías modifican su color de fondo a rojo (`#FF0000`) en el instante en que el bit de alarma correspondiente cambia de estado lógico.

Por último, el software integra una rutina de manejo de interrupciones (`on_close`) que fuerza el cierre seguro de la comunicación serie (COM) antes de destruir la ventana gráfica, previniendo el bloqueo del puerto en el sistema operativo.

Para consultar la implementación de la interfaz gráfica, ver el [ANEXO B](#).

4. SIMULACIÓN Y VALIDACIÓN

La definición e implementación técnica expuesta en el capítulo anterior abarca la totalidad del diseño propuesto, desde el acondicionamiento de las señales biomédicas a través del bus SPI hasta su procesamiento matemático en la FPGA y su visualización en el software de telemetría. Para confirmar la robustez de esta arquitectura y verificar que el hardware responde con precisión a las variaciones del ritmo cardíaco, es necesario someter el sistema a un entorno de pruebas controlado. En este capítulo se detalla la metodología de validación empleada, basada en simulaciones lógicas utilizando bases de datos clínicas internacionales, así como la evaluación del desempeño del equipo en el laboratorio.

4.1. Base de Datos del MIT-BIH

Una vez implementada la arquitectura del sistema, es necesario someter el diseño a un entorno de validación estructurado. Dado que probar un dispositivo de diagnóstico directamente sobre pacientes en fases iniciales de desarrollo no es viable, se ha recurrido a la Base de Datos de Arritmias del MIT-BIH.

Este repositorio, desarrollado conjuntamente por el Instituto Tecnológico de Massachusetts (MIT) y el Hospital Beth Israel, constituye un estándar internacional para la evaluación de algoritmos cardiológicos. Contiene registros electrocardiográficos reales acompañados de anotaciones clínicas precisas. Estas marcas temporales indican el instante exacto y el tipo de cada latido, proporcionando una referencia clínica validada para probar las detecciones automáticas del hardware[25].

4.1.1. Preparación y Escalado de los Datos (Python)

Para inyectar estos registros biomédicos en los bancos de pruebas (*testbenches*) de Vivado, se ha desarrollado un script de preparación de datos en Python. Utilizando la librería especializada wfdb (Waveform Database) [26], el software lee los archivos binarios nativos del repositorio y los acondiciona para la simulación digital.

El proceso de adaptación consta de dos fases. Primero, se extrae un bloque de muestras específico (por ejemplo, el registro 100 para un ritmo sinusal normal). A continuación, la señal se normaliza y se escala aritméticamente para que su amplitud máxima coincida con un valor cercano a 6.000.000. La aplicación de este factor de multiplicación es crítica para el entorno de simulación, ya que adapta la magnitud de la señal clínica original al rango dinámico de 24 bits con signo que operan los conversores analógico-digitales (ADCs) del sensor real ADS1293. Finalmente, los valores transformados se exportan como números enteros en archivos de texto plano (`.txt`), lo que permite a las librerías `textio` de VHDL inyectar la información en la FPGA simulando el comportamiento del bus SPI.

Nota: El código fuente del script de extracción y escalado se encuentra detallado en el [ANEXO B](#).

4.1.2. Adaptación de Frecuencias de Muestreo

La integración de esta base de datos presenta un desfase de diseño fundamental: los registros del MIT-BIH están digitalizados a una frecuencia fija de 360 Hz, mientras que la arquitectura de la FPGA está configurada para operar sincronizada con el sensor a 1000 Hz. Procesar la señal clínica de 360 Hz directamente a la velocidad nativa del hardware provocaría una contracción temporal; las máquinas de estados interpretarían el ritmo cardíaco como tres veces más rápido de lo real, emitiendo diagnósticos erróneos de taquicardia.

Para resolver esta incompatibilidad técnica durante las fases de simulación en Vivado, se recurre a la parametrización del diseño VHDL:

- **Ajuste por parámetros genéricos (FS_HZ):** Todos los módulos que dependen de la temporalidad (`detection_bridge` y `detection_alarm`) incorporan el parámetro genérico `FS_HZ`. En los scripts de los *testbenches*, este valor se sobrescribe explícitamente a 360. Al realizar este cambio antes de compilar la simulación, todas las ecuaciones internas que definen las ventanas lógicas (como los 40 ms del periodo refractario, el 18.75 % de espera para la onda T o las 3000 muestras para la asistolia) se recalculan automáticamente por el sintetizador, adaptando las respuestas del hardware a la base de tiempos exacta del repositorio clínico.

Esta aproximación paramétrica asegura la compatibilidad matemática de los datos y permite auditar la fiabilidad de las máquinas de estados utilizando el histórico del MIT-BIH, sin alterar la estructura central del hardware.

4.2. Simulación en Vivado

Para garantizar el correcto comportamiento del sistema antes de su síntesis e implementación física en la placa Nexys Video, se ha utilizado el simulador integrado de Vivado. Esta herramienta permite desplegar entornos de prueba virtuales (*testbenches*) para estimular las entradas de los módulos VHDL y evaluar la propagación de datos mediante cronogramas de ondas.

La estrategia de validación se ha diseñado bajo un enfoque de integración progresiva por capas. Inicialmente se validó el módulo SPI de forma aislada. A continuación, se diseñaron bancos de pruebas que inyectan los registros del MIT-BIH directamente en el filtro Wavelet, encadenando posteriormente las máquinas de detección QRS y T, el módulo de calibración de intervalos (puente) y la lógica de alarmas. Todo el código fuente utilizado para esta validación, así como los bancos de pruebas detallados en esta sección, se encuentran disponibles en el repositorio público de GitHub del proyecto (https://github.com/aghidalgo04/ECG_in_FPGA_Thesis).

4.2.1. Técnicas de Aceleración de la Simulación

Simular un dispositivo que procesa señales a 100 MHz durante varios segundos de actividad cardíaca requiere un tiempo de cómputo informático sumamente elevado. Para optimizar este proceso y reducir el tiempo de simulación de minutos a fracciones de segundo, los *testbenches* incorporan un mecanismo de aceleración temporal.

En lugar de esperar el periodo físico real que transcurriría entre dos muestras biomédicas (2777 μ s para una frecuencia de 360 Hz), que puede tardar varios minutos, el entorno de pruebas inyecta la señal `sample_valid` seguida de una pausa forzada de apenas 1 μ s (`wait for 1 us;`). La lógica del sistema es puramente síncrona a eventos porque se activa por el pulso `d_valid` y se basa en el parámetro `FS_HZ`, independientemente del tiempo de reloj absoluto. Debido a esto, la FPGA procesa la información de forma idéntica, permitiendo validar múltiples ciclos cardíacos en un lapso de simulación casi instantáneo.

Adicionalmente, el banco de pruebas principal (`tb_qrs_t_detection_module`) centraliza la evaluación de todas las anomalías. En su proceso de estímulo, incluye un listado comentado con las rutas hacia distintos archivos de texto del repositorio (ej. `ecg_tachy.txt`, `ecg_brady.txt`, `ecg_arrhyth.txt`). Esta estructura permite al desarrollador alternar rápidamente entre diferentes señales patológicas simplemente modificando la línea activa del código, agilizando la validación exhaustiva de cada alarma clínica.

4.2.2. Generación Virtual del Archivo UART

Para demostrar la interoperabilidad del hardware con la interfaz gráfica programada en Python sin depender de una conexión USB física, se diseñó el *testbench* `tb_uart_controller`. Este entorno instancia la cadena completa de procesamiento: lee los tres archivos espaciales simultáneos (X, Y, Z), los filtra, detecta las ondas, calcula los intervalos, emite las alarmas y traslada el paquete resultante al módulo de comunicación serie.

Para evitar la necesidad de una captura externa de hardware, el simulador incorpora un proceso concurrente (`UART_RX_VIRTUAL`). Este subprograma actúa como un receptor serie integrado; monitoriza pasivamente las caídas de tensión en el pin de salida `tx`, sincroniza la lectura aplicando el tiempo de bit exacto para 115200 baudios (`BIT_PERIOD`) y ensambla lógicamente el flujo de bits en bytes de 8 bits. Una vez reconstruido el paquete, el proceso vuelca la trama generada por la FPGA en un nuevo archivo de texto (`uart_healthy.txt`).

Este archivo de salida representa una copia binaria idéntica a lo que viajaría por el cable de cobre físico, permitiendo alimentar el panel de telemetría de Python en su modo diferido para validar el renderizado del vectorcardiograma y la visualización de las alarmas médicas.

4.3. Unidad de Filtrado Wavelet 3D

Este test representa la primera etapa de la cadena de procesamiento digital. El banco de pruebas se encarga de leer secuencialmente el archivo de texto con las muestras tridimensionales de la base de datos del MIT-BIH y entregárselas al filtro cada vez que se simula la llegada de un dato válido.



Figura 4.1. Simulación de la Unidad de Filtrado Wavelet 3D.

Como se aprecia en la captura de la simulación, el cronograma muestra una selección de las señales para facilitar su análisis visual. En concreto, se representan las entradas en bruto de los ejes X e Y (rx y ry), junto con las salidas filtradas correspondientes al eje X en sus dos escalas operativas: la escala 3 (dx3) y la escala 8 (dx8). En la señal dx3 se observa cómo el filtro aísla los componentes de alta frecuencia, acentuando el complejo QRS y formando el pico positivo y su posterior rebote negativo. Por su parte, la señal dx8 muestra la salida tras la octava etapa matemática, donde los componentes rápidos se han atenuado para perfilar de forma suave la morfología de la onda T.

Además, el cronograma enseña las señales de control lógico que rigen la sincronización del módulo. La entrada `sample_valid` representa el pulso de disparo que notifica al filtro la llegada de una nueva muestra digitalizada. Como respuesta al flujo de procesamiento, el módulo emite las banderas `rdy3` y `rdy8`. Estos pulsos conmutan a nivel alto para indicar al resto de la arquitectura de la FPGA que los cálculos de las etapas 3 y 8 han finalizado, confirmando que los datos en los buses dx3 y dx8 son válidos y están listos para ser evaluados por las máquinas de estados de detección.

4.4. Máquina de Estados del Detector QRS

En este apartado se realiza la primera prueba de integración del sistema, donde la salida de la escala 3 de la unidad Wavelet (alimentada previamente por los datos del MIT-BIH) se conecta directamente a la entrada del detector de latidos.

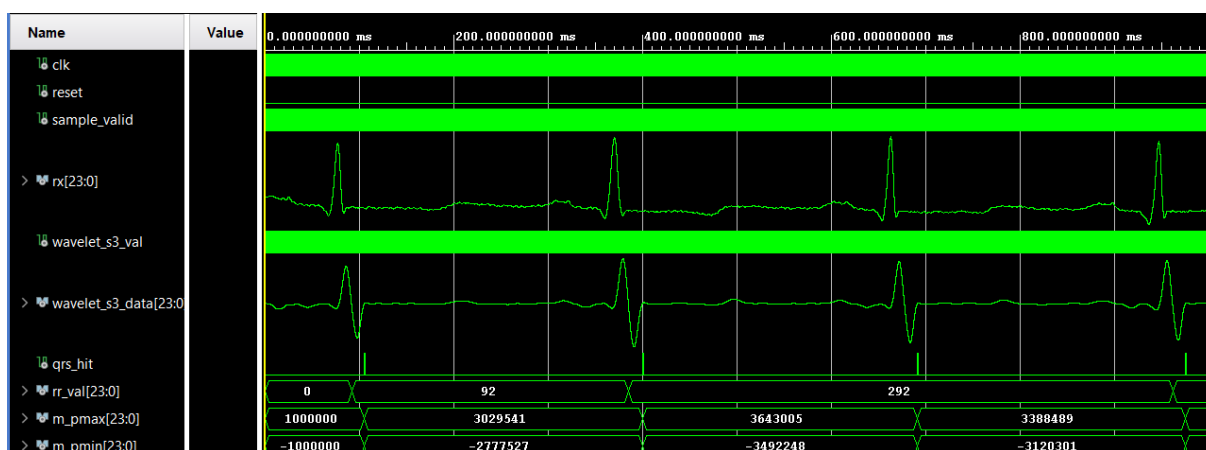


Figura 4.2. Simulación de la Máquina de Estados del Detector QRS.

Como se observa en la captura de la simulación, el cronograma ilustra el comportamiento de las señales internas durante la detección del complejo QRS. Se representa la señal cardíaca en

bruto (rx) junto con su correspondiente filtrado en la escala 3 (wavelet_s3_data), donde el latido adquiere una morfología bifásica nítida. En la parte inferior, se constata la evolución de los umbrales dinámicos positivo (m_pmax) y negativo (m_pmin), cuyos valores numéricos se reajustan ciclo a ciclo para adaptarse a la amplitud de la onda analizada. En el instante exacto en que finaliza la evaluación del latido, el módulo emite un pulso de confirmación a través de la señal qrs_hit. De forma simultánea a este disparo, el registro rr_val actualiza su valor numérico, cerrando la medición y almacenando la duración del intervalo detectado.

4.5. Máquina de Estados del Detector T

Este módulo añade una capa más de complejidad a la simulación, ya que su funcionamiento requiere la interacción coordinada de dos bloques anteriores de forma simultánea: necesita recibir la escala lenta 8 de la unidad Wavelet y, al mismo tiempo, el pulso de disparo que emite el detector QRS del apartado anterior.

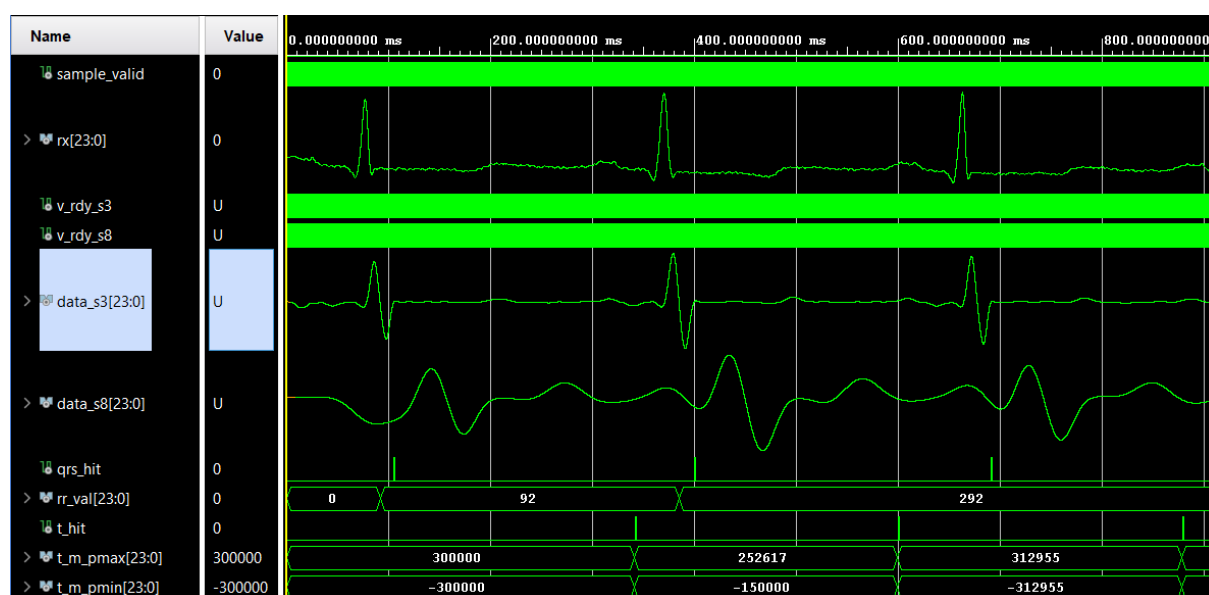


Figura 4.3. Simulación de la Máquina de Estados del Detector de Onda T.

En la simulación correspondiente a este módulo, el cronograma expone la coordinación secuencial entre las detecciones del complejo QRS y la onda T. Se visualiza la señal cardíaca original (rx) descompuesta simultáneamente en la escala rápida 3 (data_s3) y la escala lenta 8 (data_s8). Tras la identificación del latido, evidenciada por el pulso qrs_hit y la actualización del registro rr_val, el sistema procede a evaluar la repolarización ventricular sobre la curva suavizada data_s8. En los registros inferiores se puede observar cómo los umbrales dinámicos (t_m_pmax y t_m_pmin) adaptan sus valores numéricos para acotar la amplitud de

esta onda de baja frecuencia. Una vez que la señal `data_s8` concluye su morfología y se estabiliza de nuevo en la línea de base, el módulo emite el pulso de validación `t_hit`, marcando la localización exacta del final de la onda T.

4.6. Módulo Puente

El bloque puente se sitúa de forma estratégica para centralizar y unificar las detecciones independientes que provienen de los tres ejes espaciales (X, Y, Z) de las etapas anteriores. Su simulación permite comprobar tanto la robustez frente al ruido como la precisión de la calibración temporal del hardware.

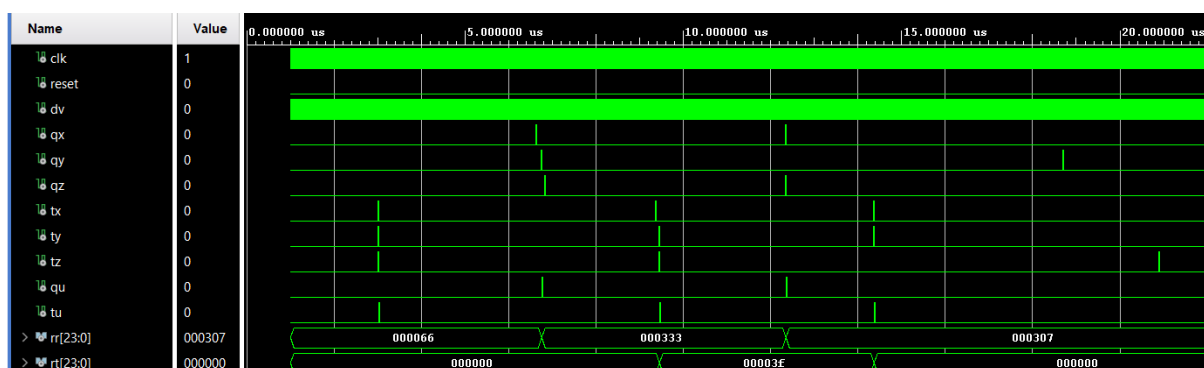


Figura 4.4. Simulación del Módulo de Sincronización e Intervalos.

En esta simulación, el cronograma muestra el funcionamiento del sistema de arbitraje y votación por mayoría. Se representan los pulsos de detección independientes provenientes de los tres ejes espaciales, tanto para el complejo QRS (`qx`, `qy`, `qz`) como para la onda T (`tx`, `ty`, `tz`). El módulo evalúa la concurrencia temporal de estos estímulos y genera los pulsos de salida unificados (`qu` y `tu`) exclusivamente cuando detecta coincidencia en al menos dos de los canales de entrada. En el instante en que se emiten estas validaciones, los buses `rr` y `rt` actualizan sus registros numéricos. Estos valores de salida representan la duración de los intervalos entre latidos, ejecutando simultáneamente la conversión a milisegundos y la compensación matemática de las latencias introducidas previamente.

4.7. Lógica de Diagnóstico y Alarmas

Este conjunto de pruebas simula el comportamiento del evaluador clínico del sistema (`detection_alarm`). Su propósito es verificar que el hardware recibe los intervalos de tiempo en milisegundos ya uni-

ficados por el bloque puente y aplica correctamente las reglas de detección de enfermedades.

Para corroborar la efectividad de los filtros de persistencia y la precisión de los umbrales matemáticos, se han ejecutado diversos bancos de pruebas inyectando señales correspondientes a diferentes estados clínicos. A continuación, se detallan los resultados obtenidos para un ritmo sinusal normal y para cada una de las cinco patologías contempladas en el diseño.

4.7.1. Ritmo Sinusal Normal (Sano)

En esta simulación se inyecta un registro cardíaco estructuralmente estable y con una frecuencia estándar.

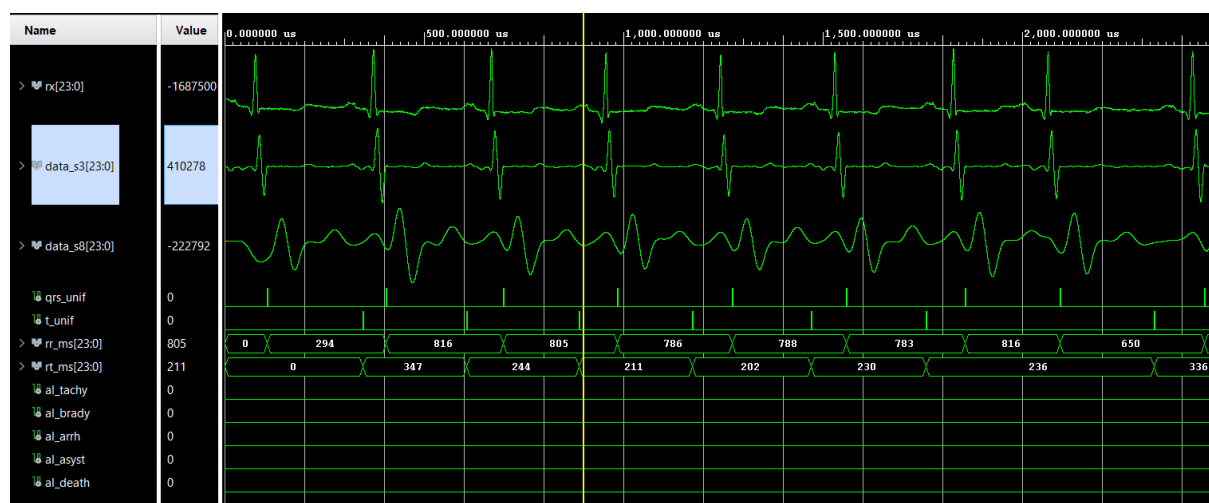


Figura 4.5. Simulación de la lógica de diagnóstico para un ritmo cardíaco sano.

Como se observa en el cronograma, los intervalos temporales cuantificados en los buses `rr_ms` y `rt_ms` se mantienen dentro de los rangos clínicos seguros: el intervalo RR fluctúa de forma natural entre los 600 ms y 1200 ms, y el intervalo de repolarización RT no excede en ningún caso el 43.75 % del ciclo total. En consecuencia, el módulo supervisor no detecta anomalías y todas las banderas de salida (`al_tachy`, `al_brady`, `al_arrh`, `al_asyst` y `al_death`) permanecen estables en nivel lógico bajo ('0').

4.7.2. Detección de Taquicardia

La simulación de este escenario evidencia un acortamiento drástico del ciclo cardíaco.

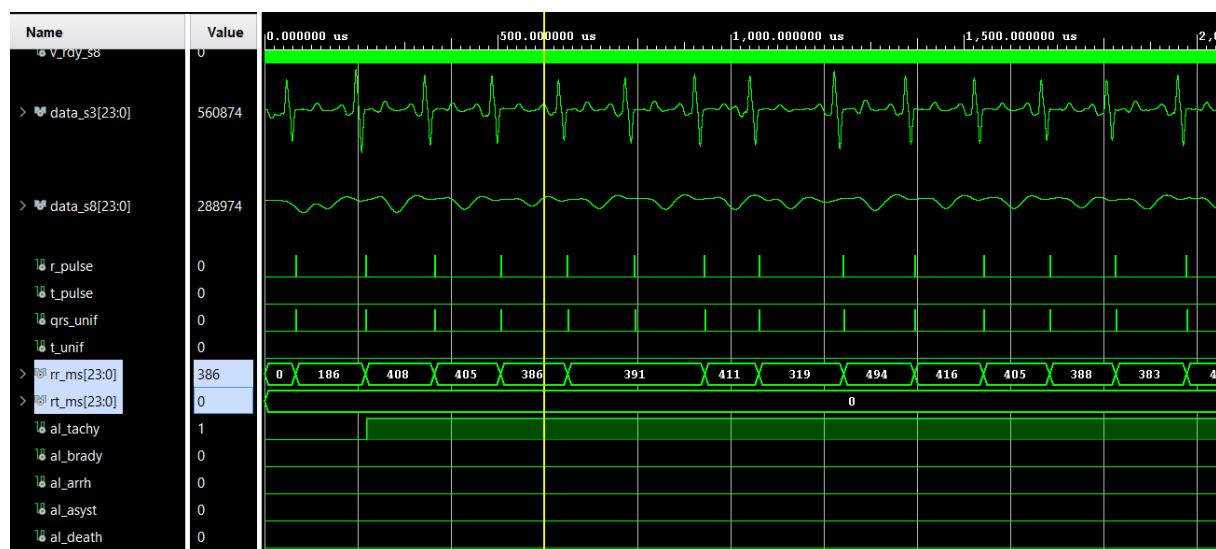


Figura 4.6. Simulación de la lógica de diagnóstico ante un cuadro de taquicardia.

Se aprecia en el bus `rr_ms` cómo el intervalo de tiempo entre latidos sucesivos decae a valores numéricos netamente inferiores al umbral predefinido de 600 ms (registrando valores de 386 ms, 405 ms, 391 ms, etc.). El sistema no emite la alerta en el primer latido corto; únicamente tras registrarse dos ciclos consecutivos por debajo de este límite inferior, el registro de persistencia interno cumple la condición (`v_persist >= 2`) y la señal `al_tachy` conmuta a nivel lógico alto ('1'). Esto demuestra el correcto funcionamiento del filtro para descartar aceleraciones aisladas.

4.7.3. Detección de Bradicardia

En contraposición al caso anterior, esta prueba inyecta una señal caracterizada por una baja frecuencia.



Figura 4.7. Simulación de la lógica de diagnóstico ante un cuadro de bradicardia.

El bus `rr_ms` cuantifica intervalos prolongados que superan el límite superior de 1200 ms. Al igual que en el cuadro de taquicardia, la bandera `al_brady` se activa exclusivamente cuando el hardware confirma la presencia de esta elongación temporal durante dos latidos sucesivos, manteniendo el estado de alerta activo mientras la condición de lentitud perdure en la señal entrante.

4.7.4. Detección de Arritmia

Este escenario clínico evalúa la inestabilidad y variabilidad del ritmo, independientemente de la frecuencia media del paciente.

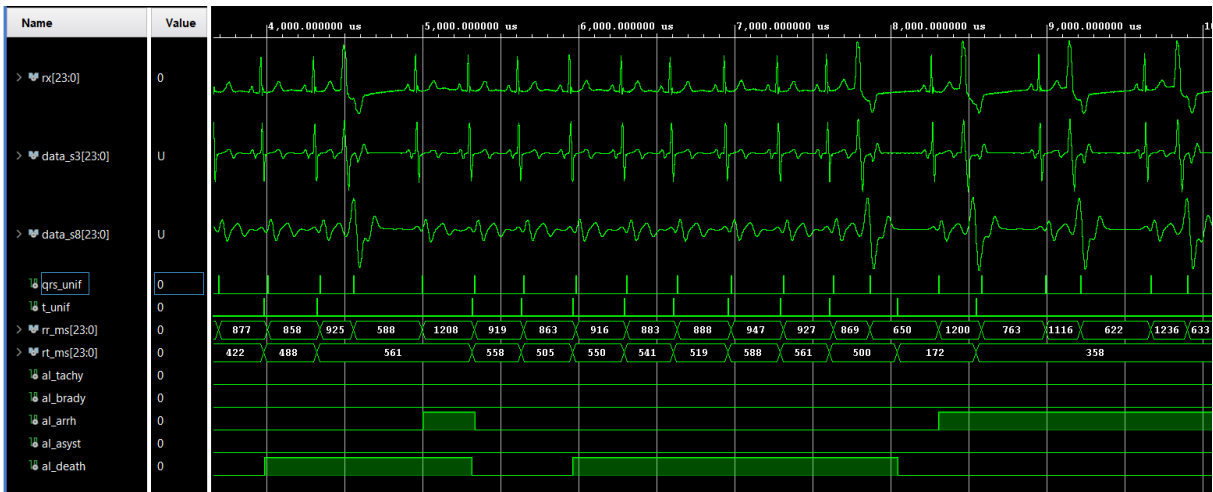


Figura 4.8. Simulación de la lógica de diagnóstico ante un cuadro de arritmia.

En el cronograma se analiza una señal caracterizada por la presencia de extrasístoles ventriculares (contracciones ventriculares prematuras). Este tipo de evento ectópico se manifiesta mediante un latido anticipado seguido de una pausa compensatoria, provocando fluctuaciones severas en la duración de los ciclos. El bus `rr_ms` cuantifica estas variaciones abruptas (reducciones a 588 ms seguidas de incrementos a 1208 ms). La lógica aritmética calcula la diferencia absoluta entre el intervalo actual y el previo. Al confirmar que esta diferencia excede el umbral adaptativo del 25 % (evaluado mediante desplazamientos binarios lógicos) durante dos iteraciones, el hardware activa la señal `al_arrh`, validando la detección de la asincronía.

Adicionalmente, se observa la activación concurrente de la señal `al_death` (riesgo de muerte súbita). Este comportamiento es clínicamente esperado dadas las características de la señal inyectada: las extrasístoles ventriculares presentan una morfología anómala que incluye una fase de repolarización (onda T) significativamente ancha y prolongada. Al medir este ensanchamiento fisiológico, el hardware detecta que el intervalo RT excede el límite de seguridad del 43.75 % respecto al ciclo temporal acordado previo, activando la regla de la mitad clínica. A pesar de este solapamiento de alarmas, el sistema demuestra su eficacia funcional al identificar y notificar correctamente la inestabilidad principal del ritmo mediante la bandera de arritmia.

4.7.5. Prevención de Muerte Súbita (Síndrome de QT Largo)

En la simulación de esta patología se pone a prueba la evaluación estructural y la regla de la mitad clínica adaptativa previamente explicada.

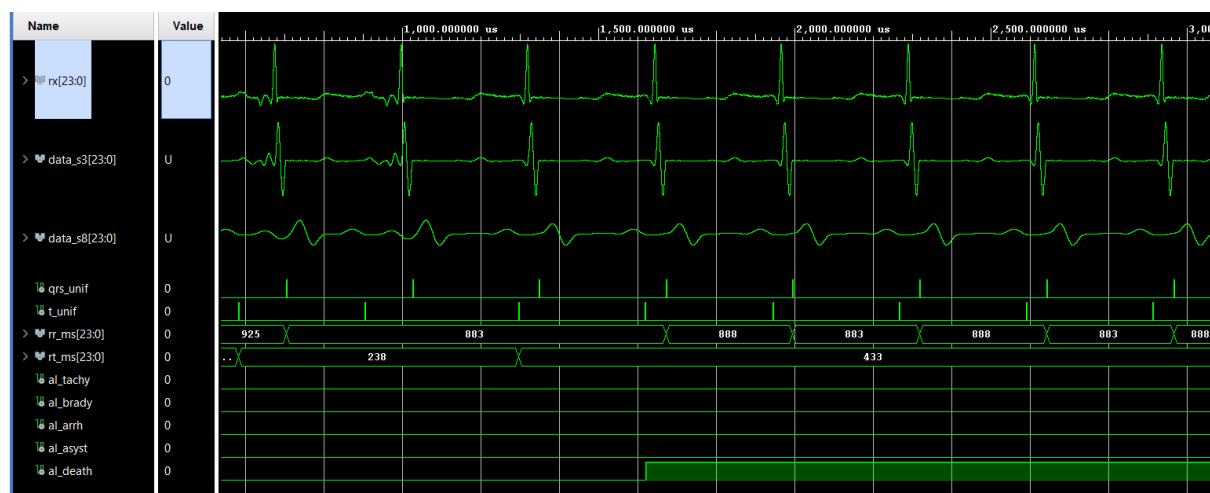


Figura 4.9. Simulación de la lógica de diagnóstico preventivo de muerte súbita.

En el cronograma se aprecia cómo la onda T sufre un retraso fisiológico prolongado respecto a su complejo QRS de origen. Esto provoca que el valor volcado en el bus `rt_ms` crezca hasta superar el 43.75 % del latido completo. Tras confirmar estructuralmente que existe una única onda R antes de la repolarización (descartando así posibles ruidos) y validar este patrón en dos ciclos continuos, el módulo dispara la señal `al_death` ('1') para emitir una alerta temprana ante el riesgo de degeneración en fibrilación.

4.7.6. Detección de Asistolia

La prueba final simula una interrupción abrupta y completa de la actividad eléctrica cardíaca.

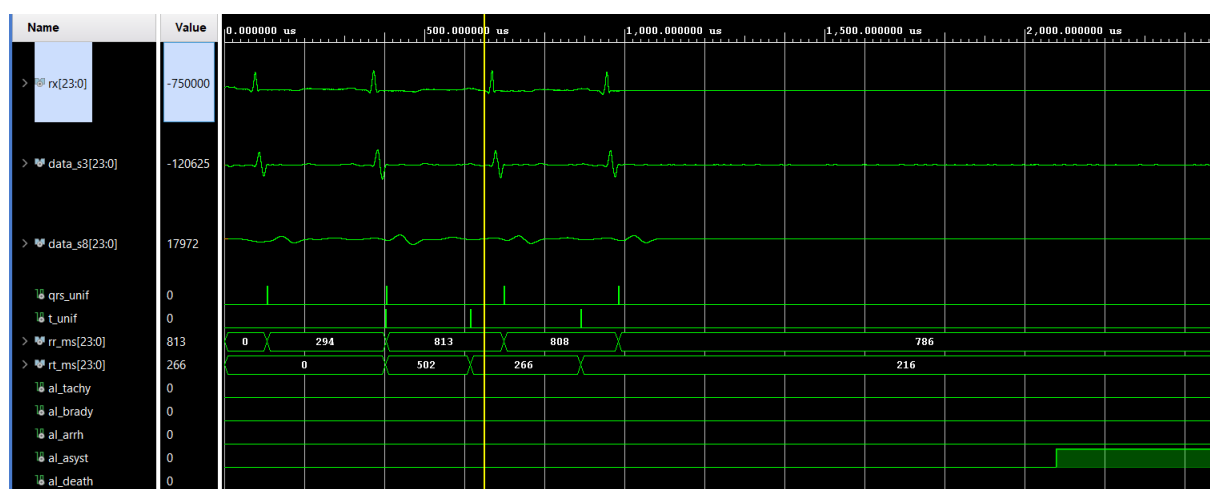


Figura 4.10. Simulación de la lógica de diagnóstico ante un cuadro de asistolia.

A diferencia de las alarmas anteriores, esta evaluación no requiere la llegada de un latido para dispararse. Se observa que los pulsos de validación `qrs_unif` cesan por completo en la entrada. Ante la falta de refresco, el contador de inactividad de la FPGA incrementa su valor progresivamente de forma síncrona. En el instante matemático en el que el contador supera el umbral predefinido de 3 segundos continuos sin actividad eléctrica, la señal `al_asyst` se activa de forma asíncrona a los latidos, notificando la condición crítica de paro cardíaco.

4.8. Módulo de Comunicación UART

El último eslabón de la cadena lúdica dentro de la FPGA es el controlador UART, el cual se encarga de recolectar toda la información generada por los bloques anteriores (señales crudas, wavelets, intervalos y alarmas de diagnóstico) y transformarla en un flujo serie.

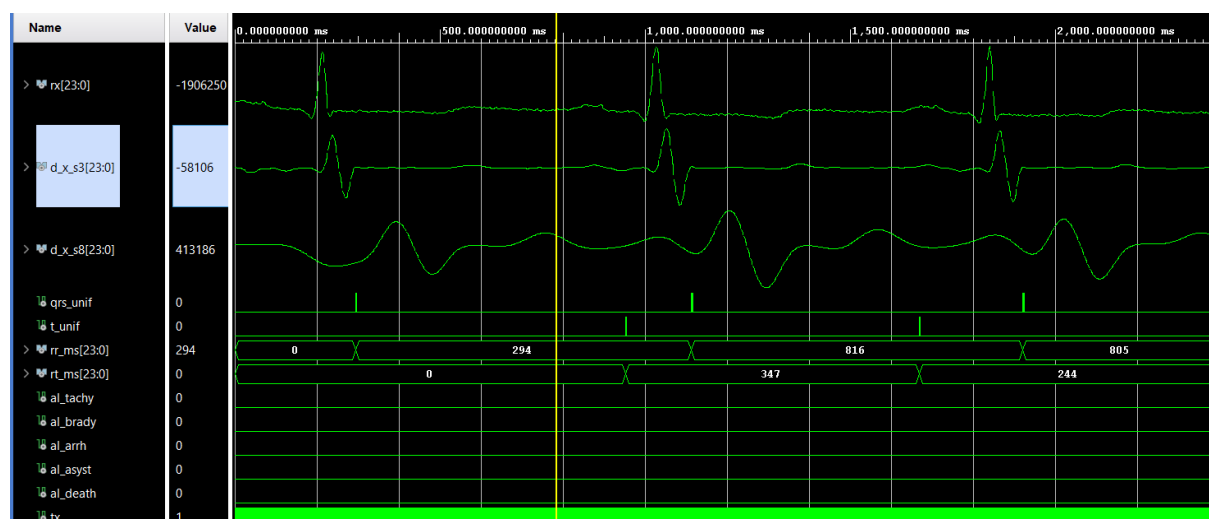


Figura 4.11. Simulación del Módulo de Comunicación UART.

En la simulación del módulo de comunicación UART, el cronograma ilustra la consolidación de todas las variables del sistema previas a su exportación. Se visualizan simultáneamente todas las señales biomédicas y de filtrado, los pulsos de eventos unificados, los registros temporales y el estado lógico de las alarmas explicados previamente. En la parte inferior, la señal de salida `tx` evidencia la actividad de transmisión serie. Debido a la amplia escala temporal necesaria para abarcar varios ciclos cardíacos completos, la emisión secuencial de los 37 bytes de la trama se representa visualmente como una banda densa y continua de transiciones lógicas. Este comportamiento confirma que la máquina de estados del controlador captura exitosamente el conjunto de datos en el instante de validación y los serializa hacia el exterior a la velocidad configurada de 115200 baudios, operando en paralelo sin interrumpir el flujo de captura de la FPGA.

4.9. Pruebas de la Interfaz Gráfica

A diferencia de los apartados anteriores ejecutados en Vivado, este test valida el comportamiento del extremo receptor del proyecto, analizando la capacidad del software programado en Python para interpretar y renderizar el flujo de datos exportado por la UART.

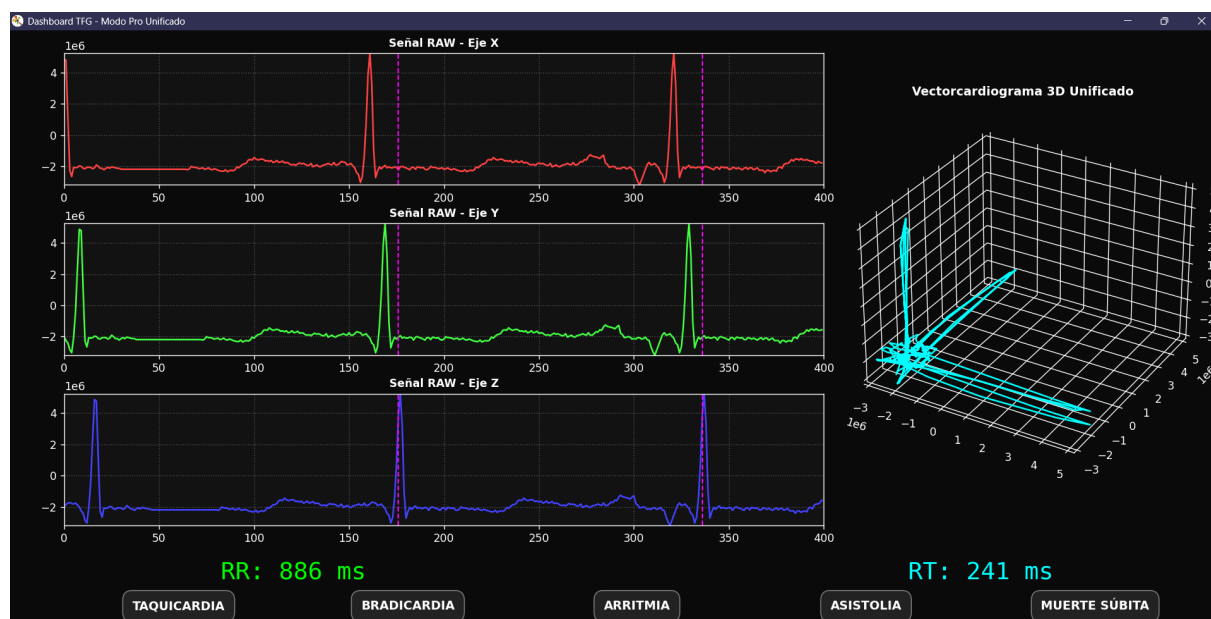


Figura 4.12. Interfaz gráfica de telemetría renderizando los datos recibidos.

La captura de pantalla evidencia el correcto desempaqueado de la trama serie. En la sección izquierda del cuadro de mandos se representan las señales biomédicas en bruto correspondientes a los tres ejes espaciales (X en rojo, Y en verde y Z en azul), reconstruidas a partir de los valores de 24 bits en complemento a dos. Sobre estas curvas temporales se visualizan líneas verticales discontinuas de color morado; estas marcas son inyectadas por el entorno gráfico al detectar los bits de validación de onda (t_{unif} o qrs_{unif}) dentro del byte de control, confirmando la sincronía visual entre el relieve de la onda y el diagnóstico de la FPGA.

Simultáneamente, el panel derecho integra las tres coordenadas espaciales para trazar el bucle continuo del vectorcardiograma tridimensional en color cian. Por último, la franja inferior del panel extrae y muestra los cálculos numéricos finales operados por el hardware (RR: 886 ms y RT: 241 ms). Dado que los tiempos de este latido específico se encuentran dentro de los parámetros fisiológicos normales, el algoritmo mantiene los cinco paneles indicadores de patología en su estado neutro oscuro, demostrando una lectura coherente y estable del byte de alarmas.

4.10. Pruebas del Módulo SPI

El controlador SPI fue el último componente de la arquitectura sometido a simulación en el entorno virtual. A diferencia de los bloques de procesamiento descritos anteriormente, la lógica de este módulo no puede ser evaluada utilizando los registros pregrabados de la base de datos del MIT-BIH, ya que su propósito exclusivo es actuar como pasarela de comunicación a nivel físico. Por consiguiente, aunque la simulación permite auditar su comportamiento teórico, la validación definitiva de su funcionamiento requiere la conexión y prueba directa con el sensor real ADS1293EVM en el laboratorio.

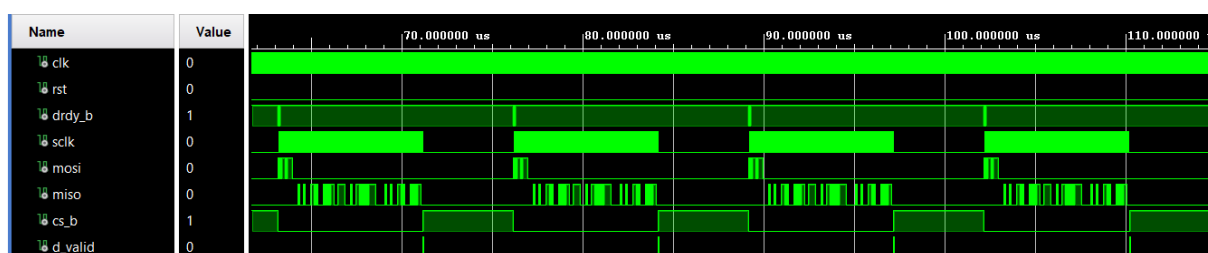


Figura 4.13. Simulación del Módulo SPI.

A nivel de cronograma, el banco de pruebas permite verificar la correcta secuenciación de las señales de control durante la fase de lectura continua (*Loop Readback*). La captura ilustra cómo el sistema reacciona ante la caída a cero de la señal de interrupción del sensor (*drdy_b*). Inmediatamente, la máquina de estados activa la línea de selección de chip (*cs_b* a nivel bajo) y arranca la generación de la señal de reloj (*sclk*). Mientras el pin *mosi* transmite el comando de inicio de lectura, el bus *miso* refleja la recepción ininterrumpida de los bytes correspondientes a la muestra tridimensional. Una vez finalizada la transferencia, el controlador desactiva la selección de chip y eleva la bandera *d_valid* durante un ciclo de reloj, indicando a la Transformada Wavelet que hay datos disponibles para procesar.

5. PRUEBAS FÍSICAS, GESTIÓN Y CONCLUSIONES

5.0.1. Resultados de Síntesis

Uno de los objetivos críticos del diseño ha sido maximizar la eficiencia algorítmica para asegurar que la arquitectura pueda sintetizarse en matrices de bajo coste y consumo. Al haber sustituido las operaciones de multiplicación complejas del filtro Wavelet por operaciones de desplazamiento binario rápido (*bit-shifting*), se ha evitado por completo la dependencia de los bloques físicos DSP del chip.

Tras ejecutar los procesos de síntesis e implementación en el entorno de Vivado para la FPGA Artix-7 (modelo XC7A200T), se generó el reporte de utilización de recursos. Los datos confirman la extrema ligereza del diseño lógico definitivo (sin incluir los núcleos de depuración ILA), tal y como se detalla a continuación:

Tabla 5.1. Resumen de utilización de recursos físicos de la FPGA Artix-7 tras la implementación.

Recurso Físico	Utilizado	Disponible	Ocupación (%)
LUT (Look-Up Tables)	8713	133800	6.51 %
FF (Flip-Flops)	7786	267600	2.91 %
BRAM (Block RAM)	0	365	0.00 %
DSP (Multiplicadores)	0	740	0.00 %

A nivel energético, la herramienta de análisis de Vivado (*Power Report*) estima que la potencia total consumida por el circuito (*Total On-Chip Power*) es de tan solo **0.225 W**. De este total, la potencia estática propia del dispositivo (*Device Static*) representa 0.139 W (62 %), mientras que la potencia dinámica asociada a las transiciones lógicas y de reloj de nuestro diseño (*Dynamic Power*) apenas supone 0.086 W (38 %).

Estos estrechos márgenes y la nula dependencia de memoria BRAM certifican que la solución propuesta es ideal para su futura integración en microchips de menor gama y su uso en disposi-

tivos portátiles ambulatorios alimentados por baterías convencionales.

5.1. Pruebas Físicas con la Placa y el Sensor Real

Una vez validado el funcionamiento de las simulaciones en Vivado, se procedió a la fase práctica en el laboratorio empleando los componentes físicos. Para ello, se ejecutaron los procesos de síntesis e implementación, generando exitosamente el archivo de programación (*bitstream*). La configuración de la FPGA Artix-7 alojada en la Nexys Video se completó sin reportar incidencias técnicas.

5.1.1. Cableado y Conexión del Sistema

Para el montaje del prototipo, se estableció una conexión física entre la placa Nexys Video y la placa de evaluación del sensor analógico ADS1293EVM mediante cableado Dupont, utilizando los puertos de expansión de ambas tarjetas. Con el objetivo de garantizar la integridad del hardware, se definió un mapa de conexiones estricto como guía de ensamblaje.

Tabla 5.2. Mapeo físico de pines para la integración del sistema.

Sensor	I/O	Nombre del pin	Señal
-	-	R4	clk
CSB	JA4	AB18	cs_b
DRDYB	JA7	Y21	drdy_b
SDO	JA3	AB20	miso
SDI	JA2	AB21	mosi
-	BTNC	B22	reset
SCLK	JA1	AB22	sclk
-	J13-UART_TX_IN	V18	tx

Tras completar el cableado y cargar el *bitstream* en la FPGA, se procedió a la primera prueba de encendido. No obstante, el sistema no reaccionó: la interfaz de visualización no registraba datos entrantes y la lógica de la arquitectura permanecía en estado de espera permanente.

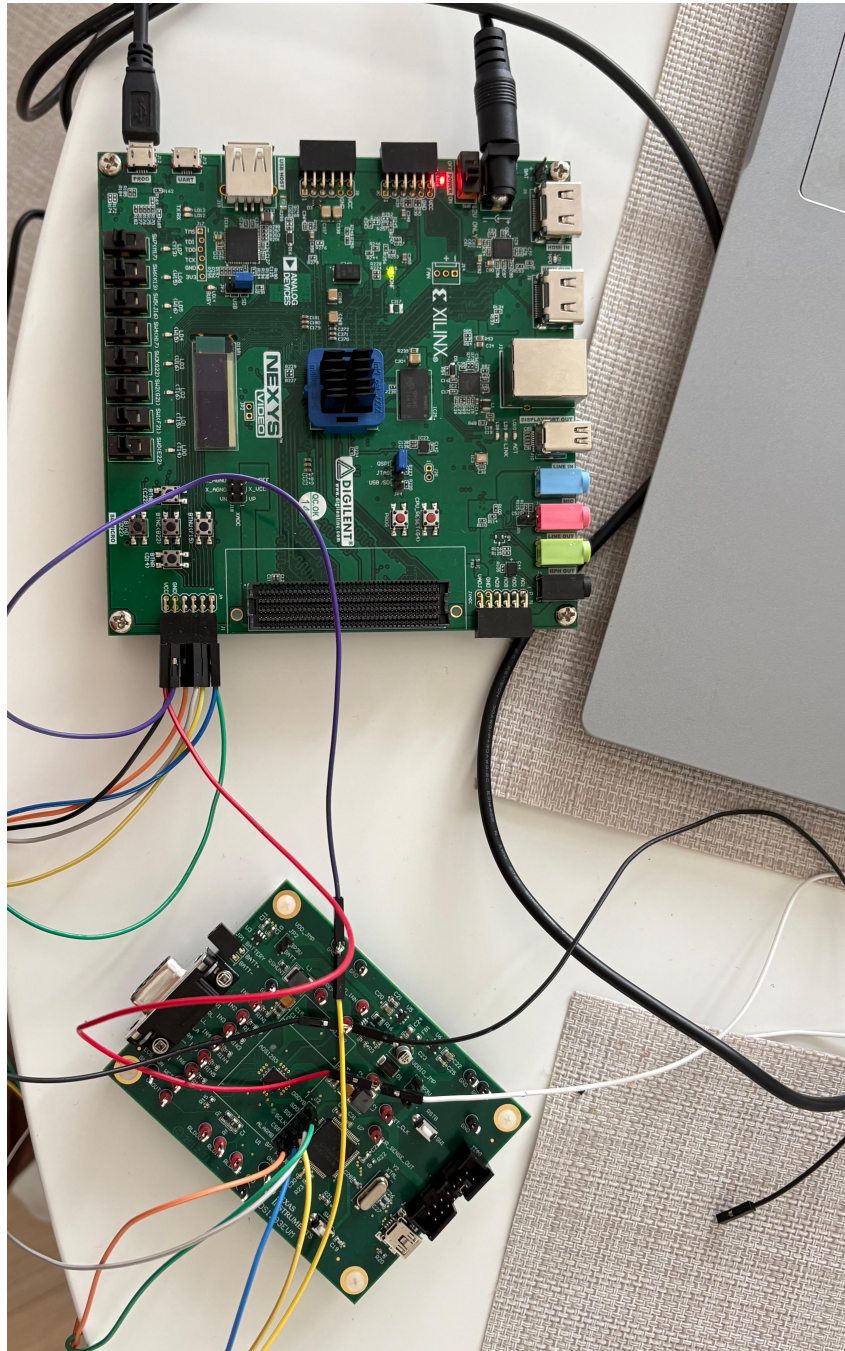


Figura 5.1. Montaje físico de los componentes.

5.1.2. Depuración en el Hardware mediante el Módulo ILA

Ante la ausencia de actividad en el sistema y la imposibilidad de monitorizar los registros internos físicamente, se integró un módulo de depuración lógica denominado ILA (*Integrated Logic Analyzer*). Este componente permite sondear y capturar las señales internas de la FPGA en tiem-

po real mientras el circuito opera en el laboratorio, volcando los cronogramas resultantes en la interfaz de Vivado.

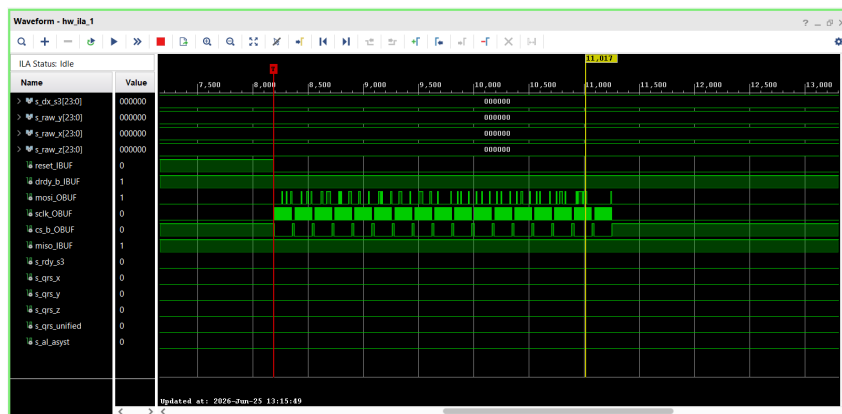


Figura 5.2. Captura de las señales internas del bus SPI mediante el módulo ILA.

El análisis del cronograma capturado por el ILA permitió aislar la anomalía. Como se observa en la gráfica, el controlador SPI de la FPGA opera con total normalidad: el hardware sitúa la señal de selección de chip (`cs_b`) en nivel bajo, activa el reloj de sincronización (`sclk`) y transmite secuencialmente por la línea `miso` la trama completa con los 17 registros de configuración. La falla radica en la ausencia de respuesta por parte del sensor analógico; la línea de recepción `miso` no presenta variaciones y se mantiene estática en nivel lógico bajo de forma ininterrumpida.

Para intentar solventar esta incidencia en el laboratorio, se verificó el cableado físico exhaustivamente, confirmando que las conexiones se correspondían con exactitud al esquema diseñado. Adicionalmente, se modificó el código fuente de la FPGA para inyectar diversas variaciones en la trama de configuración SPI, alterando los tiempos de retención y los comandos de inicio. Pese a las múltiples iteraciones de depuración física y lógica, la placa de evaluación del sensor no logró inicializarse.

5.1.3. Impacto en el Sistema y Conclusión de las Pruebas

Este problema con el sensor bloqueó por completo el funcionamiento del hardware debido a la forma en que está diseñado el código. El sistema funciona mediante una lógica guiada por eventos: cada módulo de la FPGA (los filtros Wavelet, los detectores y el puente) necesita recibir un dato válido (`d_valid`) para poder despertar y procesar la información. Como este pulso lo genera el controlador SPI únicamente cuando el sensor le entrega una muestra nueva, al no recibir información de ningún tipo del sensor, no se genera el pulso `d_valid` y todo el sistema se queda congelado en el estado de espera inicial.

A pesar de que el chip del sensor analógico permaneció inactivo en la placa real, el funcionamiento de nuestro diseño se ha podido verificar por completo. Todos los módulos digitales de procesamiento de datos, las máquinas de estados de detección de ondas, el cálculo de intervalos en milisegundos y el envío por UART han sido probados minuciosamente en la fase de simulación utilizando las señales de la base de datos del MIT. Esto demuestra que la lógica programada es totalmente válida, exacta y funciona a la perfección con los datos reales de control médicos.

5.2. Planificación Temporal

El desarrollo de este Trabajo de Fin de Grado se ha ejecutado a lo largo de un periodo de 5 meses (con una estimación de 300 horas de carga de trabajo), siguiendo una metodología de integración progresiva. Las tareas del proyecto se han distribuido en las siguientes cinco fases principales:

- **Fase 1: Investigación y Estado de la Cuestión (3 semanas):** Revisión de la literatura médica contemporánea, estudio matemático de la Transformada Wavelet y análisis de las especificaciones técnicas de la placa de evaluación ADS1293EVM.
- **Fase 2: Diseño de Hardware VHDL (5 semanas):** Programación del controlador de adquisición SPI, desarrollo del banco de filtros Wavelet 3D paramétrico, máquinas de estados síncronas para detección de ondas y módulo supervisor de alarmas clínicas.
- **Fase 3: Entorno de Pruebas y Simulación (4 semanas):** Programación del script de extracción de datos en Python para la base de datos del MIT-BIH, creación de *testbenches* exhaustivos y validación lógica secuencial en Vivado.
- **Fase 4: Interfaz Gráfica y Comunicación (3 semanas):** Programación del módulo de transmisión UART en hardware y desarrollo del cuadro de mandos (*dashboard*) interactivo de telemetría en Python.
- **Fase 5: Pruebas Físicas y Documentación (5 semanas):** Montaje de los componentes en el laboratorio, depuración lógica en tiempo real mediante el módulo ILA, extracción de métricas de rendimiento y redacción final de la memoria del proyecto.

5.3. Presupuesto del Proyecto

Para evaluar la viabilidad económica de la arquitectura propuesta, se ha desglosado el coste integral del proyecto en tres categorías fundamentales: recursos de hardware, licencias de software

y costes de personal (ingeniería). Para este último concepto, se ha estimado una tarifa estándar de 20 €/hora correspondiente a un perfil de Ingeniero Junior de Computadores.

Tabla 5.3. Presupuesto estimado del proyecto.

Concepto	Cantidad	Coste Unitario (€)	Total (€)
1. Recursos Hardware			
Placa FPGA Digilent Nexys Video	1	450.00	450.00
Módulo Frontal Analógico ADS1293EVM	1	115.00	115.00
Componentes, cableado Dupont y electrodos	1	15.00	15.00
2. Recursos Software			
Licencia AMD Vivado WebPACK	1	0.00	0.00
Entorno Python y librerías científicas	1	0.00	0.00
3. Costes de Personal			
Horas de Ingeniería (Diseño y Desarrollo)	300 h	20.00	6000.00
Subtotal			6580.00
IVA (21 %)			1381.80
PRESUPUESTO TOTAL			7931.80

5.4. Conclusión

Este Trabajo de Fin de Grado resulta en el diseño y validación de una arquitectura digital modular orientada al procesamiento en tiempo real de señales de vectocardiografía tridimensional mediante una matriz de puertas lógicas Artix-7. A nivel de hardware, se ha implementado una cadena de datos síncrona que abarca desde la Transformada Wavelet 3D para el aislamiento frecuencial del ruido, hasta la delineación adaptativa de ondas y la evaluación combinacional de patologías cardíacas. Asimismo, la integración del protocolo de comunicación UART ha permitido la exportación estructurada de estos parámetros biomédicos hacia un entorno de telemetría y visualización desarrollado en Python.

Durante la fase de validación mediante simulación lógica, el sistema demostró un comportamiento estable y conforme a las especificaciones métricas de diseño. El empleo de los registros clínicos estandarizados de la base de datos del MIT-BIH permitió certificar que las máquinas de estados, la regla de votación por mayoría espacial y el mecanismo de compensación analítica de latencias operan correctamente. No obstante, durante las pruebas de integración física

en el laboratorio se identificó una limitación en el frontal analógico: la placa de evaluación ADS1293EVM no respondió a las secuencias de inicialización del bus SPI, impidiendo la adquisición de biopotenciales físicos en vivo.

Pese a esta restricción en la etapa de interconexión analógica, el núcleo matemático y algorítmico del proyecto ha quedado íntegramente verificado. El desarrollo confirma la viabilidad técnica de emplear arquitecturas FPGA para la monitorización cardíaca continua, aprovechando su capacidad de procesamiento paralelo para calcular intervalos críticos, como el segmento RT, de forma determinista y sin las latencias inherentes a los microcontroladores secuenciales tradicionales.

5.5. Posibles Mejoras

A partir del análisis de la arquitectura desplegada y de los resultados de validación obtenidos, se proponen las siguientes líneas de desarrollo continuo para optimizar y expandir las capacidades del prototipo:

- **Depuración de la adquisición física:** La prioridad técnica inmediata reside en la resolución de la falla de inicialización del sensor ADS1293EVM a través del bus SPI. Restablecer la conectividad en la línea `miso` permitirá migrar el sistema desde los entornos de simulación en Vivado hacia la monitorización física de pacientes mediante electrodos reales.
- **Alertas hardware autónomas:** En la versión actual, la notificación de los eventos patológicos depende de la interfaz gráfica en Python. Se propone integrar el control de los periféricos nativos de la placa Nexys Video (diodos LED y emisores acústicos) para emitir avisos de emergencia directamente desde el hardware. Esto dotaría al dispositivo de autonomía operativa en caso de pérdida de comunicación con el ordenador principal.
- **Almacenamiento no volátil:** Para habilitar el uso del sistema como un monitor tipo Holter de larga duración, resultaría necesario implementar un controlador de memoria masiva (como soporte para tarjetas SD o gestión de la memoria DDR integrada). Esta mejora facilitaría el registro continuo de la actividad vectocardiográfica y del estado de las alarmas a lo largo de varias jornadas, permitiendo su posterior volcado para el análisis clínico diferido.
- **Detección de isquemia miocárdica:** La capacidad analítica de la arquitectura podría escalar en el futuro mediante la incorporación de un módulo dedicado al análisis morfológico del segmento ST. Al habilitar la detección de infartos agudos de miocardio, el dispositivo aumentaría su valor diagnóstico. Sin embargo, esta expansión requerirá el diseño de lógicas

de reconocimiento de patrones más avanzadas y una fase exhaustiva de validación empleando bases de datos especializadas en cardiopatías isquémicas.

BIBLIOGRAFÍA

- [1] ResearchGate. (s.f.). *Representación del potencial de acción de una célula miocárdica*. https://www.researchgate.net/figure/Representacion-del-potencial-de-accion-de-una-celula-miocardica-fig2_330811984
- [2] Mayo Clinic. (s.f.). *The conduction system*. <https://www.mayoclinic.org/es/tests-procedures/pacemaker/multimedia/the-conduction-system/img-20008516>
- [3] Wikipedia. (s.f.). *Electrocardiograma*. <https://es.wikipedia.org/wiki/Electrocardiograma>
- [4] Schiller. (s.f.). *Vector ECG*. <https://www.schiller.ch/es-es/productos/ecg-vector-p654>
- [5] INM Perú. (s.f.). *Electrocardiograma*. <https://www.inmperu.com/electrocardiograma>
- [6] EcuRed. (s.f.). *Síndrome de QT largo*. https://www.ecured.cu/S%C3%ADndrome_de_QT_largo
- [7] Built In. (s.f.). *Wavelet Transform*. <https://builtin.com/data-science/wavelet-transform>
- [8] AmeliCA. (s.f.). *Descomposición por escalas de señales*. <https://portal.amelica.org/ameli/jatsRepo/150/150673005/150673005.pdf>
- [9] TME. (s.f.). *Nexys Video Development Board*. <https://www.tme.eu/en/details/410-316/development-kits-others/digilent/nexys-video/>
- [10] Texas Instruments. (s.f.). *ADS1293EVM*. <https://www.ti.com/tool/es-mx/ADS1293EVM>
- [11] García Limón, J. A., Martínez-Suárez, F., y Alvarado-Serrano, C. (2023). Implementation of Wavelet-Transform-Based Algorithms in an FPGA for Heart Rate and RT Interval Automatic Measurements in Real Time. *Micromachines*, 14(9), 1748. <https://doi.org/10.3390/>

mi14091748

- [12] Digilent Inc. (2024). *Nexys Video Board Reference Manual*. Pullman, WA. <https://digilent.com/reference/programmable-logic/nexys-video/reference-manual>
- [13] Pan, J., y Tompkins, W. J. (1985). A Real-Time QRS Detection Algorithm. *IEEE Transactions on Biomedical Engineering*, BME-32(3), 230–236. <https://doi.org/10.1109/TBME.1985.325532>
- [14] Wikipedia. (s.f.). *Intervalo QT*. https://es.wikipedia.org/wiki/Intervalo_QT
- [15] Wikipedia. (s.f.). *Frecuencia cardíaca*. https://es.wikipedia.org/wiki/Frecuencia_card%C3%ADaca
- [16] Wikipedia. (s.f.). *Sistema de conducción eléctrica del corazón*. https://es.wikipedia.org/wiki/Sistema_de_conducci%C3%B3n_el%C3%A9ctrica_del_coraz%C3%B3n
- [17] World Health Organization (WHO). (2021). *Cardiovascular diseases (CVDs)*. [https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds))
- [18] Wikipedia. (s.f.). *Torsades de pointes*. https://es.wikipedia.org/wiki/Torsades_de_pointes
- [19] Wikipedia. (s.f.). *Transformada wavelet discreta*. https://es.wikipedia.org/wiki/Transformada_wavelet_discreta
- [20] Wikipedia. (s.f.). *Wavelet transform*. https://en.wikipedia.org/wiki/Wavelet_transform
- [21] Martínez, J. P., et al. (2004). A wavelet-based ECG delineator: evaluation on standard databases. *IEEE Transactions on Biomedical Engineering*, 51(4), 570–581. <https://doi.org/10.1109/TBME.2003.821031>
- [22] MathWorks. (2024). *R-Wave Detection in the ECG*. MathWorks Wavelet Toolbox Documentation. <https://www.mathworks.com/help/wavelet/ug/r-wave-detection-in-the-ecg.html>

- [23] Wikipedia. (s.f.). *Digital signal processing*. https://en.wikipedia.org/wiki/Digital_signal_processing
- [24] Hunter, J. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- [25] Moody, G. B., y Mark, R. G. (2001). The impact of the MIT-BIH Arrhythmia Database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3), 45–50. <https://doi.org/10.1109/51.932724>
- [26] Silva, C. A., y Moody, G. B. (2019). WFDB-Python: A software package for physiological signal processing and visualization in Python. *PhysioNet Open Source Software Papers*, 4(1), 12–18. <https://doi.org/10.13026/g72j-v508>
- [27] Guevara Johnson, J. F. (s.f.). *Vectorcardiografía*. <https://www.gorgas.gob.pa/wp-content/uploads/external/BiblioGorgas/pdf/Vectorcardiografia.%20J.F%20guevara%20Johnson..pdf>

A.

CÓDIGO FUENTE VHDL

En este anexo se incluye el código fuente íntegro en lenguaje VHDL correspondiente a todos los módulos de hardware diseñados e implementados en la FPGA para el procesamiento, filtrado y diagnóstico de la señal vectocardiográfica.

A.0.1. Módulo Principal (Top Module)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity top_module is
generic (
CLK_FREQ  : integer := 100000000;
BAUD_RATE : integer := 115200;
CLK_DIV   : integer := 10;
FS_HZ     : integer := 1000
);
port (
clk      : in  std_logic;
reset    : in  std_logic;
drdy_b   : in  std_logic;
miso     : in  std_logic;

sclk     : out std_logic;
mosi     : out std_logic;
cs_b     : out std_logic;
tx       : out std_logic
);
end top_module;

architecture Structural of top_module is

component spi_controller is
generic ( CLK_DIV : integer := 10 );
port (
clk      : in  std_logic;
rst      : in  std_logic;
drdy_b   : in  std_logic;
sclk     : out std_logic;
mosi     : out std_logic;
miso     : in  std_logic;
cs_b     : out std_logic;
raw_x    : out std_logic_vector(23 downto 0);
```

```

raw_y      : out std_logic_vector(23 downto 0);
raw_z      : out std_logic_vector(23 downto 0);
d_valid    : out std_logic
);
end component;

component wavelet_3d_transform is
port (
clk          : in  std_logic;
reset        : in  std_logic;
sample_valid_in : in  std_logic;
raw_x        : in  std_logic_vector(23 downto 0);
raw_y        : in  std_logic_vector(23 downto 0);
raw_z        : in  std_logic_vector(23 downto 0);
wavelet_x    : out std_logic_vector(23 downto 0);
wavelet_y    : out std_logic_vector(23 downto 0);
wavelet_z    : out std_logic_vector(23 downto 0);
vector_ready_s3 : out std_logic;
vector_ready_s8 : out std_logic;
d_x_s3       : out signed(23 downto 0);
d_y_s3       : out signed(23 downto 0);
d_z_s3       : out signed(23 downto 0);
d_x_s8       : out signed(23 downto 0);
d_y_s8       : out signed(23 downto 0);
d_z_s8       : out signed(23 downto 0)
);
end component;

component qrs_detector is
port (
clk          : in  std_logic;
reset        : in  std_logic;
d_valid      : in  std_logic;
d_wavelet    : in  signed(23 downto 0);
qrs_detected : out std_logic;
polarity     : out std_logic;
time_rr      : out signed(23 downto 0);
current_mem_pmax : out signed(23 downto 0);
current_mem_pmin : out signed(23 downto 0)
);
end component;

component t_detector is
port (
clk          : in  std_logic;
reset        : in  std_logic;
d_valid      : in  std_logic;
d_wavelet    : in  signed(23 downto 0);
start_trigger : in  std_logic;
rr_interval   : in  signed(23 downto 0);
qrs_mem_pmax  : in  signed(23 downto 0);
qrs_mem_pmin  : in  signed(23 downto 0);
t_detected    : out std_logic;
t_mem_pmax    : out signed(23 downto 0);
t_mem_pmin    : out signed(23 downto 0)
);
end component;

```



```

component detection_bridge is
generic ( FS_HZ : integer := 1000 );
port (
clk          : in  std_logic;
reset        : in  std_logic;
d_valid      : in  std_logic;
qrs_x, qrs_y, qrs_z : in std_logic;
t_x, t_y, t_z   : in std_logic;
qrs_unified   : out std_logic;
t_unified     : out std_logic;
rr_interval_ms : out signed(23 downto 0);
rt_interval_ms : out signed(23 downto 0)
);
end component;

component detection_alarm is
generic ( FS_HZ : integer := 1000 );
port (
clk          : in  std_logic;
reset        : in  std_logic;
d_valid      : in  std_logic;
qrs_unified   : in  std_logic;
t_unified     : in  std_logic;
rr_interval_ms : in  signed(23 downto 0);
rt_interval_ms : in  signed(23 downto 0);
alarm_tachycardia : out std_logic;
alarm_bradycardia : out std_logic;
alarm_arrhythmia  : out std_logic;
alarm_asystole    : out std_logic;
alarm_sudden_death : out std_logic
);
end component;

component uart_controller is
generic (
CLK_FREQ : integer := 100000000;
BAUD_RATE : integer := 115200
);
port (
clk          : in  std_logic;
reset        : in  std_logic;
d_valid      : in  std_logic;
raw_x        : in  std_logic_vector(23 downto 0);
raw_y        : in  std_logic_vector(23 downto 0);
raw_z        : in  std_logic_vector(23 downto 0);
s3_x, s3_y, s3_z : in  signed(23 downto 0);
s8_x, s8_y, s8_z : in  signed(23 downto 0);
rr_interval_ms : in  signed(23 downto 0);
rt_interval_ms : in  signed(23 downto 0);
qrs_unified   : in  std_logic;
t_unified     : in  std_logic;
al_tachy      : in  std_logic;
al_brady      : in  std_logic;
al_arrh       : in  std_logic;
al_asyst      : in  std_logic;
al_death      : in  std_logic;

```

```

tx          : out std_logic
);
end component;

signal s_raw_x, s_raw_y, s_raw_z : std_logic_vector(23 downto 0);
signal s_spi_valid               : std_logic;

signal s_w_x, s_w_y, s_w_z      : std_logic_vector(23 downto 0);
signal s_rdy_s3, s_rdy_s8      : std_logic;

signal s_dx_s3, s_dy_s3, s_dz_s3 : signed(23 downto 0);
signal s_dx_s8, s_dy_s8, s_dz_s8 : signed(23 downto 0);

signal s_qrs_x, s_qrs_y, s_qrs_z : std_logic;
signal s_t_x, s_t_y, s_t_z       : std_logic;

signal s_pol_x, s_pol_y, s_pol_z : std_logic;
signal s_trr_x, s_trr_y, s_trr_z : signed(23 downto 0);

signal s_pmax_x, s_pmax_y, s_pmax_z : signed(23 downto 0);
signal s_pmin_x, s_pmin_y, s_pmin_z : signed(23 downto 0);

signal s_tpmax_x, s_tpmax_y, s_tpmax_z : signed(23 downto 0);
signal s_tpmin_x, s_tpmin_y, s_tpmin_z : signed(23 downto 0);

signal s_qrs_unified, s_t_unified : std_logic;
signal s_rr_ms, s_rt_ms           : signed(23 downto 0);

signal s_al_tachy, s_al_brady, s_al_arrh, s_al_asyt, s_al_death : std_logic;

begin

SPI_INST: spi_controller
generic map ( CLK_DIV => CLK_DIV )
port map (
clk      => clk,
rst      => reset,
drdy_b   => drdy_b,
sclk     => sclk,
mosi     => mosi,
miso     => miso,
cs_b     => cs_b,
raw_x    => s_raw_x,
raw_y    => s_raw_y,
raw_z    => s_raw_z,
d_valid  => s_spi_valid
);

WAVELET_INST: wavelet_3d_transform
port map (
clk          => clk,
reset        => reset,
sample_valid_in => s_spi_valid,
raw_x        => s_raw_x,
raw_y        => s_raw_y,
raw_z        => s_raw_z,
wavelet_x    => s_w_x,

```

```

wavelet_y      => s_w_y,
wavelet_z      => s_w_z,
vector_ready_s3 => s_rdy_s3,
vector_ready_s8 => s_rdy_s8,
d_x_s3         => s_dx_s3,
d_y_s3         => s_dy_s3,
d_z_s3         => s_dz_s3,
d_x_s8         => s_dx_s8,
d_y_s8         => s_dy_s8,
d_z_s8         => s_dz_s8
);

QRS_DET_X: qrs_detector
port map (
clk          => clk,
reset        => reset,
d_valid      => s_rdy_s3,
d_wavelet    => s_dx_s3,
qrs_detected => s_qrs_x,
polarity     => s_pol_x,
time_rr      => s_trr_x,
current_mem_pmax => s_pmax_x,
current_mem_pmin => s_pmin_x
);

QRS_DET_Y: qrs_detector
port map (
clk          => clk,
reset        => reset,
d_valid      => s_rdy_s3,
d_wavelet    => s_dy_s3,
qrs_detected => s_qrs_y,
polarity     => s_pol_y,
time_rr      => s_trr_y,
current_mem_pmax => s_pmax_y,
current_mem_pmin => s_pmin_y
);

QRS_DET_Z: qrs_detector
port map (
clk          => clk,
reset        => reset,
d_valid      => s_rdy_s3,
d_wavelet    => s_dz_s3,
qrs_detected => s_qrs_z,
polarity     => s_pol_z,
time_rr      => s_trr_z,
current_mem_pmax => s_pmax_z,
current_mem_pmin => s_pmin_z
);

T_DET_X: t_detector
port map (
clk          => clk,
reset        => reset,
d_valid      => s_rdy_s8,
d_wavelet    => s_dx_s8,

```

```

start_trigger    => s_qrs_x,
rr_interval      => s_trr_x,
qrs_mem_pmax     => s_pmax_x,
qrs_mem_pmin     => s_pmin_x,
t_detected       => s_t_x,
t_mem_pmax       => s_tpmax_x,
t_mem_pmin       => s_tpmix_x
);

```

```

T_DET_Y: t_detector
port map (
clk           => clk,
reset         => reset,
d_valid       => s_rdy_s8,
d_wavelet     => s_dy_s8,
start_trigger => s_qrs_y,
rr_interval   => s_trr_y,
qrs_mem_pmax  => s_pmax_y,
qrs_mem_pmin  => s_pmin_y,
t_detected    => s_t_y,
t_mem_pmax    => s_tpmax_y,
t_mem_pmin    => s_tpmix_y
);

```

```

T_DET_Z: t_detector
port map (
clk           => clk,
reset         => reset,
d_valid       => s_rdy_s8,
d_wavelet     => s_dz_s8,
start_trigger => s_qrs_z,
rr_interval   => s_trr_z,
qrs_mem_pmax  => s_pmax_z,
qrs_mem_pmin  => s_pmin_z,
t_detected    => s_t_z,
t_mem_pmax    => s_tpmax_z,
t_mem_pmin    => s_tpmix_z
);

```

```

BRIDGE_INST: detection_bridge
generic map ( FS_HZ => FS_HZ )
port map (
clk           => clk,
reset         => reset,
d_valid       => s_spi_valid,
qrs_x         => s_qrs_x,
qrs_y         => s_qrs_y,
qrs_z         => s_qrs_z,
t_x           => s_t_x,
t_y           => s_t_y,
t_z           => s_t_z,
qrs_unified   => s_qrs_unified,
t_unified     => s_t_unified,
rr_interval_ms => s_rr_ms,
rt_interval_ms => s_rt_ms
);

```

```

ALARM_INST: detection_alarm
generic map ( FS_HZ => FS_HZ )
port map (
  clk           => clk,
  reset         => reset,
  d_valid       => s_spi_valid,
  qrs_unified   => s_qrs_unified,
  t_unified     => s_t_unified,
  rr_interval_ms => s_rr_ms,
  rt_interval_ms => s_rt_ms,
  alarm_tachycardia => s_al_tachy,
  alarm_bradycardia => s_al_brady,
  alarm_arrhythmia  => s_al_arrh,
  alarm_asystole    => s_al_asyst,
  alarm_sudden_death => s_al_death
);

UART_INST: uart_controller
generic map (
  CLK_FREQ => CLK_FREQ,
  BAUD_RATE => BAUD_RATE
)
port map (
  clk           => clk,
  reset         => reset,
  d_valid       => s_spi_valid,
  raw_x         => s_raw_x,
  raw_y         => s_raw_y,
  raw_z         => s_raw_z,
  s3_x          => s_dx_s3,
  s3_y          => s_dy_s3,
  s3_z          => s_dz_s3,
  s8_x          => s_dx_s8,
  s8_y          => s_dy_s8,
  s8_z          => s_dz_s8,
  rr_interval_ms => s_rr_ms,
  rt_interval_ms => s_rt_ms,
  qrs_unified   => s_qrs_unified,
  t_unified     => s_t_unified,
  al_tachy      => s_al_tachy,
  al_brady      => s_al_brady,
  al_arrh       => s_al_arrh,
  al_asyst      => s_al_asyst,
  al_death      => s_al_death,
  tx            => tx
);
end Structural;

```

A.0.2. Controlador SPI (SPI Controller)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

use IEEE.NUMERIC_STD.ALL;

entity spi_controller is
generic (
CLK_DIV : integer := 100 -- Spi a 100 MHz
);
port (
-- Sistema
clk      : in  std_logic;
rst      : in  std_logic;

-- Control Sensor
drdy_b   : in  std_logic;

-- Bus SPI
sclk     : out std_logic;
mosi     : out std_logic;
miso     : in  std_logic;
cs_b     : out std_logic;

-- Salidas de datos (Canales X, Y, Z)
raw_x    : out std_logic_vector(23 downto 0);
raw_y    : out std_logic_vector(23 downto 0);
raw_z    : out std_logic_vector(23 downto 0);
d_valid  : out std_logic
);
end spi_controller;

architecture Behavioral of spi_controller is

-- Estados de la FSM
type state_type is (STARTUP, STARTUP_HOLD, IDLE, TRANSFER, DONE, WAIT_DRDY_HIGH);
signal state : state_type := STARTUP;

-- Contadores de tiempo y bits
signal clk_cnt : integer range 0 to 255 := 0;
signal bit_cnt : integer range 0 to 7 := 0;
signal byte_cnt : integer range 0 to 16 := 0;

-- Registros de desplazamiento y reloj interno
signal shift_out : std_logic_vector(7 downto 0) := x"00";
signal shift_in  : std_logic_vector(7 downto 0) := x"00";
signal sclk_reg  : std_logic := '0';

-- Buffers de recepción de datos
signal rx_buf_x : std_logic_vector(23 downto 0) := x"000000";
signal rx_buf_y : std_logic_vector(23 downto 0) := x"000000";
signal rx_buf_z : std_logic_vector(23 downto 0) := x"000000";

-- Array de configuración
type config_array is array (0 to 33) of std_logic_vector(7 downto 0);
constant CONFIG_DATA : config_array := (
x"01", x"11", -- FLEX_CH1_CN
x"02", x"19", -- FLEX_CH2_CN
x"03", x"1E", -- FLEX_CH3_CN
x"0A", x"07", -- CMDET_EN
x"0C", x"04", -- RLD_CN

```

```

-- Activacion de WCT
x"0D", x"01", -- WILSON_EN1
x"0E", x"02", -- WILSON_EN2
x"0F", x"03", -- WILSON_EN3
x"10", x"01", -- WILSON_CN

x"12", x"04", -- OSC_CN
x"21", x"02", -- R2_RATE
x"22", x"02", -- R3_RATE_CH1
x"23", x"02", -- R3_RATE_CH2
x"24", x"02", -- R3_RATE_CH3
x"27", x"20", -- DRDYB_SRC
x"2F", x"70", -- CH_CNFG
x"00", x"01" -- CONFIG
);
signal cfg_ptr : integer range 0 to 17 := 0;

begin
sclk <= sclk_reg;

process(clk)
begin
if rising_edge(clk) then
if rst = '1' then
state <= STARTUP;
cs_b <= '1';
sclk_reg <= '0';
mosi <= '0';
d_valid <= '0';
clk_cnt <= 0;
bit_cnt <= 7;
byte_cnt <= 0;
cfg_ptr <= 0;
raw_x <= x"000000";
raw_y <= x"000000";
raw_z <= x"000000";
shift_out <= x"01";
shift_in <= x"00";
else
case state is

-- Configuración inicial de los registros del sensor
when STARTUP =>
d_valid <= '0';
if cfg_ptr < 17 then
cs_b <= '0';

-- Carga del primer bit
if clk_cnt = 0 and bit_cnt = 7 and byte_cnt = 0 then
shift_out <= CONFIG_DATA(cfg_ptr * 2);
mosi <= CONFIG_DATA(cfg_ptr * 2)(7);
end if;

-- Generación de SCLK
if clk_cnt = (CLK_DIV / 2) - 1 then
sclk_reg <= '1';

```

```

clk_cnt <= clk_cnt + 1;
elsif clk_cnt = CLK_DIV - 1 then
sclk_reg <= '0';
clk_cnt <= 0;

if bit_cnt = 0 then
bit_cnt <= 7;
if byte_cnt = 0 then
byte_cnt <= 1;
shift_out <= CONFIG_DATA((cfg_ptr * 2) + 1);
mosi <= CONFIG_DATA((cfg_ptr * 2) + 1)(7);
else
byte_cnt <= 0;
clk_cnt <= 0;
state <= STARTUP_HOLD;
end if;
else
bit_cnt <= bit_cnt - 1;
mosi <= shift_out(bit_cnt - 1);
end if;
else
clk_cnt <= clk_cnt + 1;
end if;
else
cs_b <= '1';
sclk_reg <= '0';
mosi <= '0';
state <= IDLE;
end if;

-- Retención de CS_B
when STARTUP_HOLD =>
cs_b <= '1';
sclk_reg <= '0';
mosi <= '0';
if clk_cnt = 20 then
clk_cnt <= 0;
cfg_ptr <= cfg_ptr + 1;
state <= STARTUP;
else
clk_cnt <= clk_cnt + 1;
end if;

-- Espera la respuesta del sensor
when IDLE =>
d_valid <= '0';
cs_b <= '1';
sclk_reg <= '0';
clk_cnt <= 0;
bit_cnt <= 7;
byte_cnt <= 0;
if drdy_b = '0' then
cs_b <= '0';
shift_out <= x"B7";
mosi <= '1';
state <= TRANSFER;
end if;

```



```

-- Lectura SPI
when TRANSFER =>
  if clk_cnt = (CLK_DIV / 2) - 1 then
    sclk_reg <= '1';
    shift_in <= shift_in(6 downto 0) & miso;
    clk_cnt <= clk_cnt + 1;
  elsif clk_cnt = CLK_DIV - 1 then
    sclk_reg <= '0';
    clk_cnt <= 0;

    if bit_cnt = 0 then
      bit_cnt <= 7;
      byte_cnt <= byte_cnt + 1;
      shift_out <= x"00";
      mosi <= '0';

      case byte_cnt is
        when 1 => rx_buf_x(23 downto 16) <= shift_in;
        when 2 => rx_buf_x(15 downto 8) <= shift_in;
        when 3 => rx_buf_x(7 downto 0) <= shift_in;
        when 4 => rx_buf_y(23 downto 16) <= shift_in;
        when 5 => rx_buf_y(15 downto 8) <= shift_in;
        when 6 => rx_buf_y(7 downto 0) <= shift_in;
        when 7 => rx_buf_z(23 downto 16) <= shift_in;
        when 8 => rx_buf_z(15 downto 8) <= shift_in;
        when 9 =>
          rx_buf_z(7 downto 0) <= shift_in;
          state <= DONE;
        when others => null;
      end case;
    else
      bit_cnt <= bit_cnt - 1;
      mosi <= shift_out(bit_cnt - 1);
    end if;
  else
    clk_cnt <= clk_cnt + 1;
  end if;

-- Fin de transmisión
when DONE =>
  cs_b <= '1';
  raw_x <= rx_buf_x;
  raw_y <= rx_buf_y;
  raw_z <= rx_buf_z;
  d_valid <= '1';
  state <= WAIT_DRDY_HIGH;

-- Espera DRDY_B nivel alto
when WAIT_DRDY_HIGH =>
  d_valid <= '0';
  if drdy_b = '1' then
    state <= IDLE;
  end if;
end case;
end if;
end if;

```

```

end process;

end Behavioral;

```

A.0.3. Filtro Wavelet Base (Wavelet Phase)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity wavelet_phase is
  Generic (
    m : integer := 1 -- Numero de escala
  );
  Port (
    clk      : in  STD_LOGIC;
    reset    : in  STD_LOGIC;
    -- Entradas
    d_in_valid : in  STD_LOGIC;
    d_in      : in  SIGNED(23 downto 0);
    -- Salidas
    d_out_valid : out STD_LOGIC;
    y          : out SIGNED(23 downto 0);
    d          : out SIGNED(23 downto 0);
  );
end wavelet_phase;

architecture Behavioral of wavelet_phase is
  -- Buffer de 33 datos para escala 8 (4 * 8 = 32)
  type delay_line is array (0 to 32) of signed(23 downto 0);
  signal regs : delay_line := (others => (others => '0'));

begin
  process(clk)
    variable sum_h : signed(28 downto 0);
  begin
    if rising_edge(clk) then
      if reset = '1' then
        regs <= (others => (others => '0'));
        y <= (others => '0');
        d <= (others => '0');
        d_out_valid <= '0';
      else
        d_out_valid <= '0';

        if d_in_valid = '1' then
          regs(1 to 32) <= regs(0 to 31);
          regs(0) <= d_in;

          -- Filtro B-Spline [1, 4, 6, 4, 1]
          sum_h := shift_left(resize(regs(0), 29), 0) + -- 1
            shift_left(resize(regs(1*m), 29), 2) + -- 4

```

```

(shift_left(resize(regs(2*m), 29), 2) + shift_left(resize(regs(2*m), 29), 1)) + -- 6
  (4+2)
shift_left(resize(regs(3*m), 29), 2) + -- 4
shift_left(resize(regs(4*m), 29), 0); -- 1

-- Salida dividida por 16 (aproximación)
y <= sum_h(27 downto 4);

-- Aplicación de la escala (m)
d <= regs(0) - regs(4*m);

d_out_valid <= '1';
end if;
end if;
end if;
end process;
end Behavioral;

```

A.0.4. Banco de Filtros 1D (Wavelet 1Dimension)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity wavelet_1dimension is
Port (
clk          : in  STD_LOGIC;
reset        : in  STD_LOGIC;

-- Entradas (modulo SPI)
sample_valid : in  STD_LOGIC;
raw_data     : in  STD_LOGIC_VECTOR(23 downto 0);

-- Salidas (Detección)
y_wavelet_s3 : out SIGNED(23 downto 0);
d_wavelet_s3 : out SIGNED(23 downto 0);
d_wavelet_s8 : out SIGNED(23 downto 0);
wavelet_ready_s3 : out STD_LOGIC;
wavelet_ready_s8 : out STD_LOGIC
);
end wavelet_1dimension;

architecture Behavioral of wavelet_1dimension is

component wavelet_phase is
generic ( m : integer );
Port (
clk : in STD_LOGIC;
reset : in STD_LOGIC;
d_in_valid : in STD_LOGIC;
d_in : in SIGNED(23 downto 0);
d_out_valid : out STD_LOGIC;

```

```

y : out SIGNED(23 downto 0);
d : out SIGNED(23 downto 0)
);
end component;

signal val_1, val_2, val_3, val_4, val_5, val_6, val_7, val_8 : std_logic;
signal y1, y2, y3, y4, y5, y6, y7, y8 : signed(23 downto 0);
signal d1, d2, d3, d4, d5, d6, d7, d8 : signed(23 downto 0);

signal raw_signed : signed(23 downto 0);

begin
raw_signed <= signed(raw_data);

-- Escalado
STAGE_1: wavelet_phase generic map(m => 1)
port map (
clk => clk,
reset => reset,
d_in_valid => sample_valid,
d_in => raw_signed,
d_out_valid => val_1,
y => y1,
d => d1
);

STAGE_2: wavelet_phase generic map(m => 2)
port map (
clk => clk,
reset => reset,
d_in_valid => val_1,
d_in => y1,
d_out_valid => val_2,
y => y2,
d => d2
);
-- Fin de escalado para qrs
STAGE_3: wavelet_phase generic map(m => 3)
port map (
clk => clk,
reset => reset,
d_in_valid => val_2,
d_in => y2,
d_out_valid => val_3,
y => y3,
d => d3
);

STAGE_4: wavelet_phase generic map(m => 4)
port map (
clk => clk,
reset => reset,
d_in_valid => val_3,
d_in => y3,
d_out_valid => val_4,
y => y4,
d => d4

```

```
);

STAGE_5: wavelet_phase generic map(m => 5)
port map (
  clk => clk,
  reset => reset,
  d_in_valid => val_4,
  d_in => y4,
  d_out_valid => val_5,
  y => y5,
  d => d5
);

STAGE_6: wavelet_phase generic map(m => 6)
port map (
  clk => clk,
  reset => reset,
  d_in_valid => val_5,
  d_in => y5,
  d_out_valid => val_6,
  y => y6,
  d => d6
);

STAGE_7: wavelet_phase generic map(m => 7)
port map (
  clk => clk,
  reset => reset,
  d_in_valid => val_6,
  d_in => y6,
  d_out_valid => val_7,
  y => y7,
  d => d7
);

-- Fin de escalado para t
STAGE_8: wavelet_phase generic map(m => 8)
port map (
  clk => clk,
  reset => reset,
  d_in_valid => val_7,
  d_in => y7,
  d_out_valid => wavelet_ready_s8,
  y => y8,
  d => d_wavelet_s8
);

y_wavelet_s3 <= y3;
d_wavelet_s3 <= d3;
wavelet_ready_s3 <= val_3;

end Behavioral;
```

A.0.5. Transformada Wavelet 3D (Wavelet 3D Transform)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity wavelet_3d_transform is
Port (
clk           : in  STD_LOGIC;
reset         : in  STD_LOGIC;

-- Entradas (SPI)
sample_valid_in : in  STD_LOGIC;
raw_x           : in  STD_LOGIC_VECTOR(23 downto 0);
raw_y           : in  STD_LOGIC_VECTOR(23 downto 0);
raw_z           : in  STD_LOGIC_VECTOR(23 downto 0);

-- Salidas
-- Wavelet escala 3
wavelet_x       : out STD_LOGIC_VECTOR(23 downto 0);
wavelet_y       : out STD_LOGIC_VECTOR(23 downto 0);
wavelet_z       : out STD_LOGIC_VECTOR(23 downto 0);

-- Ready Flags
vector_ready_s3 : out STD_LOGIC;
vector_ready_s8 : out STD_LOGIC;

-- Escala 3 (QRS_Detector)
d_x_s3          : out SIGNED(23 downto 0);
d_y_s3          : out SIGNED(23 downto 0);
d_z_s3          : out SIGNED(23 downto 0);

-- Escala 8 (T_Detector)
d_x_s8          : out SIGNED(23 downto 0);
d_y_s8          : out SIGNED(23 downto 0);
d_z_s8          : out SIGNED(23 downto 0);
);
end wavelet_3d_transform;

architecture Behavioral of wavelet_3d_transform is

component wavelet_1dimension is
port (
clk           : in  STD_LOGIC;
reset         : in  STD_LOGIC;
sample_valid  : in  STD_LOGIC;
raw_data      : in  STD_LOGIC_VECTOR(23 downto 0);
y_wavelet_s3  : out SIGNED(23 downto 0);
d_wavelet_s3  : out SIGNED(23 downto 0);
d_wavelet_s8  : out SIGNED(23 downto 0);
wavelet_ready_s3 : out STD_LOGIC;
wavelet_ready_s8 : out STD_LOGIC
);
end component;

```

```

signal yx3, yy3, yz3 : SIGNED(23 downto 0);
signal dx3, dy3, dz3 : SIGNED(23 downto 0);
signal dx8, dy8, dz8 : SIGNED(23 downto 0);

signal rdyx_s3, rdy_y_s3, rdyz_s3 : STD_LOGIC;
signal rdyx_s8, rdy_y_s8, rdyz_s8 : STD_LOGIC;

begin
EJE_X: wavelet_1dimension
port map (
clk => clk,
reset => reset,
sample_valid => sample_valid_in,
raw_data => raw_x,
y_wavelet_s3 => yx3,
d_wavelet_s3 => dx3,
d_wavelet_s8 => dx8,
wavelet_ready_s3 => rdyx_s3,
wavelet_ready_s8 => rdyx_s8
);

EJE_Y: wavelet_1dimension
port map (
clk => clk,
reset => reset,
sample_valid => sample_valid_in,
raw_data => raw_y,
y_wavelet_s3 => yy3,
d_wavelet_s3 => dy3,
d_wavelet_s8 => dy8,
wavelet_ready_s3 => rdy_y_s3,
wavelet_ready_s8 => rdy_y_s8
);

EJE_Z: wavelet_1dimension
port map (
clk => clk,
reset => reset,
sample_valid => sample_valid_in,
raw_data => raw_z,
y_wavelet_s3 => yz3,
d_wavelet_s3 => dz3,
d_wavelet_s8 => dz8,
wavelet_ready_s3 => rdyz_s3,
wavelet_ready_s8 => rdyz_s8
);

process(clk)
begin
if rising_edge(clk) then
if reset = '1' then
vector_ready_s3 <= '0';
vector_ready_s8 <= '0';

d_x_s3 <= (others => '0');
d_y_s3 <= (others => '0');
d_z_s3 <= (others => '0');

```

```

d_x_s8 <= (others => '0');
d_y_s8 <= (others => '0');
d_z_s8 <= (others => '0');

wavelet_x <= (others => '0');
wavelet_y <= (others => '0');
wavelet_z <= (others => '0');
else
vector_ready_s3 <= '0';
vector_ready_s8 <= '0';

-- Procesado de Escala 3
if rdyz_s3 = '1' then
d_x_s3 <= dx3;
d_y_s3 <= dy3;
d_z_s3 <= dz3;

wavelet_x <= std_logic_vector(yx3);
wavelet_y <= std_logic_vector(yy3);
wavelet_z <= std_logic_vector(yz3);

vector_ready_s3 <= '1';
end if;

-- Procesado Escala 8
if rdyz_s8 = '1' then
d_x_s8 <= dx8;
d_y_s8 <= dy8;
d_z_s8 <= dz8;

vector_ready_s8 <= '1';
end if;
end if;
end process;
end Behavioral;

```

A.0.6. Detector del Complejo QRS (QRS Detector)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity qrs_detector is
Port (
clk           : in  STD_LOGIC;
reset         : in  STD_LOGIC;

-- Entradas(Wavelet)
d_valid       : in  STD_LOGIC;
d_wavelet     : in  SIGNED(23 downto 0);

```



```

-- Salidas (Bridge)
qrs_detected      : out STD_LOGIC := '0';
polarity          : out STD_LOGIC := '0';
time_rr           : out SIGNED(23 downto 0) := (others => '0');
current_mem_pmax  : out SIGNED(23 downto 0) := (others => '0');
current_mem_pmin  : out SIGNED(23 downto 0) := (others => '0');
);
end qrs_detector;

architecture Behavioral of qrs_detector is

-- Maquina de 5 estados
type state_type is (ETAPA_1, ETAPA_2, ETAPA_3, ETAPA_4, ETAPA_5);
signal state : state_type := ETAPA_1;

-- Umbrales predefinidos para evitar falsos positivos
signal mem_pmax : signed(23 downto 0) := to_signed(1000000, 24);
signal mem_pmin : signed(23 downto 0) := to_signed(-1000000, 24);

signal hold_pmax : signed(23 downto 0) := (others => '0');
signal hold_pmin : signed(23 downto 0) := (others => '0');
signal qrs_n : std_logic := '0';
constant TIME_40MS : integer := 40;
signal cnt_40ms : integer := 0;
constant TIME_3S : integer := 3000;
signal cnt_rr : integer := 0;

-- Constante para "contar como 0" un baremo amplio porque nunca va a ser 0 absoluto.
constant VIRTUAL_ZERO : signed(23 downto 0) := to_signed(100000, 24);

begin
current_mem_pmax <= mem_pmax;
current_mem_pmin <= mem_pmin;
polarity <= qrs_n;

process(clk)
variable abs_pmin : signed(23 downto 0);
begin
if rising_edge(clk) then
if reset = '1' then
state <= ETAPA_1;
mem_pmax <= to_signed(1000000, 24);
mem_pmin <= to_signed(-1000000, 24);
qrs_detected <= '0';
cnt_rr <= 0;
cnt_40ms <= 0;
hold_pmax <= (others => '0');
hold_pmin <= (others => '0');
else
qrs_detected <= '0';

if d_valid = '1' then

-- Ajustar umbrales para seguir detectando aunque la senal decrezca
abs_pmin := abs(mem_pmin);
if mem_pmax > shift_left(abs_pmin, 1) and abs_pmin > 0 then

```

```

mem_pmin <= -mem_pmax;
elsif abs_pmin > shift_left(mem_pmax, 1) and mem_pmax > 0 then
mem_pmax <= abs_pmin;
end if;

-- Contador para bajar el umbral en caso de no detectar latidos.
if cnt_rr < TIME_3S then
cnt_rr <= cnt_rr + 1;
else
cnt_rr <= 0;
mem_pmax <= shift_right(mem_pmax, 1);
mem_pmin <= shift_right(mem_pmin, 1);
state <= ETAPA_1;
end if;

case state is
-- ETAPA 1: Superar el umbral (50% del anterior)
when ETAPA_1 =>
hold_pmax <= (others => '0');
hold_pmin <= (others => '0');
if d_wavelet < mem_pmin then
qrs_n <= '1';
hold_pmin <= d_wavelet;
state <= ETAPA_2;
elsif d_wavelet > mem_pmax then
qrs_n <= '0';
hold_pmax <= d_wavelet;
state <= ETAPA_2;
end if;

-- ETAPA 2: Monitorizar picos y cruce por 0
when ETAPA_2 =>
if d_wavelet > hold_pmax then hold_pmax <= d_wavelet; end if;
if d_wavelet < hold_pmin then hold_pmin <= d_wavelet; end if;

if (qrs_n = '0' and d_wavelet <= VIRTUAL_ZERO) or
(qrs_n = '1' and d_wavelet >= -VIRTUAL_ZERO) then
time_rr <= to_signed(cnt_rr, 24);
cnt_rr <= 0;
state <= ETAPA_3;
end if;

-- ETAPA 3: Buscar un pico similar en el espectro opuesto.
when ETAPA_3 =>
if d_wavelet > hold_pmax then hold_pmax <= d_wavelet; end if;
if d_wavelet < hold_pmin then hold_pmin <= d_wavelet; end if;

if (qrs_n = '0' and d_wavelet < mem_pmin) or
(qrs_n = '1' and d_wavelet > mem_pmax) then
state <= ETAPA_4;
end if;

-- ETAPA 4: Actualizacion de umbrales (Al 50%)
when ETAPA_4 =>
if d_wavelet > hold_pmax then hold_pmax <= d_wavelet; end if;
if d_wavelet < hold_pmin then hold_pmin <= d_wavelet; end if;

```

```

if (qrs_n = '0' and d_wavelet >= -VIRTUAL_ZERO) or
(qrs_n = '1' and d_wavelet <= VIRTUAL_ZERO) then

mem_pmax <= shift_right(hold_pmax, 1);
mem_pmin <= shift_right(hold_pmin, 1);

qrs_detected <= '1';
state <= ETAPA_5;
end if;

-- ETAPA 5: Refractario (Espera de seguridad)
when ETAPA_5 =>
if cnt_40ms < TIME_40MS then
cnt_40ms <= cnt_40ms + 1;
else
cnt_40ms <= 0;
state <= ETAPA_1;
end if;
end case;
end if;
end if;
end if;
end process;
end Behavioral;

```

A.0.7. Detector de la Onda T (T Detector)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity t_detector is
Port (
clk           : in  STD_LOGIC;
reset         : in  STD_LOGIC;

--Entradas
--Senal S8 (Wavelet)
d_valid       : in  STD_LOGIC;
d_wavelet     : in  SIGNED(23 downto 0);
start_trigger : in  STD_LOGIC;

-- Intervalo RR y umbrales (QRS_Detector)
rr_interval   : in  SIGNED(23 downto 0);
qrs_mem_pmax  : in  SIGNED(23 downto 0);
qrs_mem_pmin  : in  SIGNED(23 downto 0);

-- Salidas (Bridge)
t_detected    : out STD_LOGIC := '0';
t_mem_pmax    : out SIGNED(23 downto 0) := (others => '0');
t_mem_pmin    : out SIGNED(23 downto 0) := (others => '0')
);

```

```

end t_detector;

architecture Behavioral of t_detector is
type state_type is (ETAPA_1, ETAPA_2, ETAPA_3, ETAPA_4, ETAPA_5);
signal state : state_type := ETAPA_1;

-- Umbrales predefinidos para evitar falsos positivos
signal mem_pmax : signed(23 downto 0) := to_signed(300000, 24);
signal mem_pmin : signed(23 downto 0) := to_signed(-300000, 24);

signal hold_pmax : signed(23 downto 0) := (others => '0');
signal hold_pmin : signed(23 downto 0) := (others => '0');

signal t_n : std_logic := '0';
signal cnt_window : integer := 0;
signal limit_window : integer := 0;

constant VIRTUAL_ZERO : signed(23 downto 0) := to_signed(100000, 24);

begin
t_mem_pmax <= mem_pmax;
t_mem_pmin <= mem_pmin;

process(clk)
variable abs_pmin : signed(23 downto 0);
variable calc_win : integer;
begin
if rising_edge(clk) then
if reset = '1' then
state <= ETAPA_1;
mem_pmax <= to_signed(300000, 24);
mem_pmin <= to_signed(-300000, 24);
t_detected <= '0';
cnt_window <= 0;
limit_window <= 170;
else
t_detected <= '0';

-- Ajustar umbrales para seguir detectando aunque la senal decrezca
abs_pmin := abs(mem_pmin);
if mem_pmax > shift_left(abs_pmin, 1) and abs_pmin > 0 then
mem_pmin <= -mem_pmax;
elsif abs_pmin > shift_left(mem_pmax, 1) and mem_pmax > 0 then
mem_pmax <= abs_pmin;
end if;

-- Calculo de periodo de espera
if start_trigger = '1' then

-- RR_ms/8 + RR_ms/16 equivale al ~52% del latido.
calc_win := to_integer(shift_right(rr_interval, 3)) + to_integer(shift_right(
rr_interval, 4));

-- Minimo de seguridad
if calc_win < 100 then
limit_window <= 170;
else

```

```

limit_window <= calc_win;
end if;

cnt_window <= 1;
state <= ETAPA_1;
end if;

if d_valid = '1' then
case state is
-- ETAPA 1: Espera tras latido
when ETAPA_1 =>
hold_pmax <= (others => '0');
hold_pmin <= (others => '0');
if cnt_window > 0 then
if cnt_window >= limit_window then
cnt_window <= 0;
state <= ETAPA_2;
else
cnt_window <= cnt_window + 1;
end if;
end if;

-- ETAPA 2: Buscar primer pico T (Entrada al 50% de la memoria)
when ETAPA_2 =>
if d_wavelet > hold_pmax then hold_pmax <= d_wavelet; end if;
if d_wavelet < hold_pmin then hold_pmin <= d_wavelet; end if;

if d_wavelet > shift_right(mem_pmax, 1) then
t_n <= '0';
elsif d_wavelet < shift_right(mem_pmin, 1) then
t_n <= '1';
end if;

if (abs(d_wavelet) <= VIRTUAL_ZERO) and
(hold_pmax > 80000 or hold_pmin < -80000) then
state <= ETAPA_3;
end if;

-- ETAPA 3: Buscar espejo negativo
when ETAPA_3 =>
if d_wavelet > hold_pmax then hold_pmax <= d_wavelet; end if;
if d_wavelet < hold_pmin then hold_pmin <= d_wavelet; end if;

if (t_n = '0' and d_wavelet < shift_right(mem_pmin, 2)) or
(t_n = '1' and d_wavelet > shift_right(mem_pmax, 2)) then
state <= ETAPA_4;
end if;

-- ETAPA 4: Final de onda T y actualizacion de umbrales
when ETAPA_4 =>
if d_wavelet > hold_pmax then hold_pmax <= d_wavelet; end if;
if d_wavelet < hold_pmin then hold_pmin <= d_wavelet; end if;

if abs(d_wavelet) <= VIRTUAL_ZERO then

-- Minimo de seguridad (150.000)
if shift_right(hold_pmax, 1) < to_signed(150000, 24) then

```

```

mem_pmax <= to_signed(150000, 24);
else
mem_pmax <= shift_right(hold_pmax, 1);
end if;

if shift_right(hold_pmin, 1) > to_signed(-150000, 24) then
mem_pmin <= to_signed(-150000, 24);
else
mem_pmin <= shift_right(hold_pmin, 1);
end if;

state <= ETAPA_5;
end if;

-- ETAPA 5: Pulso de salida
when ETAPA_5 =>
if mem_pmax > shift_right(qrs_mem_pmax, 1) then
mem_pmax <= shift_right(qrs_mem_pmax, 2);
end if;
t_detected <= '1';
state <= ETAPA_1;
end case;
end if;
end if;
end if;
end process;
end Behavioral;

```

A.0.8. Puente de Unificación y Calibración (Detection Bridge)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity detection_bridge is
generic (
FS_HZ : integer := 1000
);
Port (
clk           : in  STD_LOGIC;
reset         : in  STD_LOGIC;
d_valid       : in  STD_LOGIC;

-- Entradas (QRS y T detector)
qrs_x, qrs_y, qrs_z : in  STD_LOGIC;
t_x, t_y, t_z       : in  STD_LOGIC;

-- Salidas (Detection Alarm y UART)
qrs_unified       : out STD_LOGIC := '0';
t_unified         : out STD_LOGIC := '0';

```

```

rr_interval_ms : out SIGNED(23 downto 0) := (others => '0');
rt_interval_ms : out SIGNED(23 downto 0) := (others => '0')
);
end detection_bridge;

architecture Behavioral of detection_bridge is
constant WIN_SAMPLES : integer := (30 * FS_HZ) / 1000;

-- Retraso en la deteccion de las ondas (basado en simulación)
constant DELAY_QRS_MS : integer := 83;
constant DELAY_T_MS   : integer := 389;

-- Cálculo de espera segun frecuencia necesitada (para acelerar simulación)
constant SAMPLES_QRS_DELAY : integer := (DELAY_QRS_MS * FS_HZ) / 1000;
constant SAMPLES_T_DELAY   : integer := (DELAY_T_MS * FS_HZ) / 1000;

signal qrs_x_str, qrs_y_str, qrs_z_str : std_logic := '0';
signal t_x_str, t_y_str, t_z_str      : std_logic := '0';
signal cnt_qrs_x, cnt_qrs_y, cnt_qrs_z : integer := 0;
signal cnt_t_x, cnt_t_y, cnt_t_z      : integer := 0;

signal qrs_voted, t_voted : std_logic := '0';
signal qrs_voted_prev, t_voted_prev : std_logic := '0';

signal cnt_rr, cnt_rt      : integer := 0;
signal rt_busy            : std_logic := '0';

signal qrs_unif_i, t_unif_i : std_logic := '0';
signal rr_ms_i, rt_ms_i    : signed(23 downto 0) := (others => '0');

begin
qrs_unified    <= qrs_unif_i;
t_unified      <= t_unif_i;
rr_interval_ms <= rr_ms_i;
rt_interval_ms <= rt_ms_i;

-- Generador de periodo de tiempo de detección
process(clk) begin
if rising_edge(clk) then
if reset = '1' then
qrs_x_str <= '0'; qrs_y_str <= '0'; qrs_z_str <= '0';
t_x_str <= '0'; t_y_str <= '0'; t_z_str <= '0';
cnt_qrs_x <= 0; cnt_qrs_y <= 0; cnt_qrs_z <= 0;
cnt_t_x <= 0; cnt_t_y <= 0; cnt_t_z <= 0;
else
-- Eje X (QRS)
if qrs_x = '1' then
qrs_x_str <= '1'; cnt_qrs_x <= WIN_SAMPLES;
elsif d_valid = '1' then
if cnt_qrs_x > 0 then cnt_qrs_x <= cnt_qrs_x - 1;
else qrs_x_str <= '0'; end if;
end if;

-- Eje Y (QRS)
if qrs_y = '1' then
qrs_y_str <= '1'; cnt_qrs_y <= WIN_SAMPLES;
elsif d_valid = '1' then

```

```

if cnt_qrs_y > 0 then cnt_qrs_y <= cnt_qrs_y - 1;
else qrs_y_str <= '0'; end if;
end if;

-- Eje Z (QRS)
if qrs_z = '1' then
qrs_z_str <= '1'; cnt_qrs_z <= WIN_SAMPLES;
elsif d_valid = '1' then
if cnt_qrs_z > 0 then cnt_qrs_z <= cnt_qrs_z - 1;
else qrs_z_str <= '0'; end if;
end if;

-- Eje X (T)
if t_x = '1' then
t_x_str <= '1'; cnt_t_x <= WIN_SAMPLES;
elsif d_valid = '1' then
if cnt_t_x > 0 then cnt_t_x <= cnt_t_x - 1;
else t_x_str <= '0'; end if;
end if;

-- Eje Y (T)
if t_y = '1' then
t_y_str <= '1'; cnt_t_y <= WIN_SAMPLES;
elsif d_valid = '1' then
if cnt_t_y > 0 then cnt_t_y <= cnt_t_y - 1;
else t_y_str <= '0'; end if;
end if;

-- Eje Z (T)
if t_z = '1' then
t_z_str <= '1'; cnt_t_z <= WIN_SAMPLES;
elsif d_valid = '1' then
if cnt_t_z > 0 then cnt_t_z <= cnt_t_z - 1;
else t_z_str <= '0'; end if;
end if;
end if;
end if;
end process;

qrs_voted <= (qrs_x_str and qrs_y_str) or (qrs_x_str and qrs_z_str) or (qrs_y_str and
qrs_z_str);
t_voted <= (t_x_str and t_y_str) or (t_x_str and t_z_str) or (t_y_str and t_z_str);

-- Calculo de intervalos
process(clk)
variable rt_smp : integer;
variable ms_tmp : integer;
begin
if rising_edge(clk) then
if reset = '1' then
qrs_unif_i <= '0'; t_unif_i <= '0';
cnt_rr <= 0; cnt_rt <= 0; rt_busy <= '0';
rr_ms_i <= (others => '0');
rt_ms_i <= (others => '0');
qrs_voted_prev <= '0'; t_voted_prev <= '0';
elsif d_valid = '1' then
qrs_voted_prev <= qrs_voted;

```

```

t_voted_prev <= t_voted;

cnt_rr <= cnt_rr + 1;
if rt_busy = '1' then cnt_rt <= cnt_rt + 1; end if;

qrs_unif_i <= '0'; t_unif_i <= '0';

if qrs_voted = '1' and qrs_voted_prev = '0' then
qrs_unif_i <= '1';

-- El intervalo RR no necesita compensación porque el retraso se anula al medir R vs R
ms_tmp := (cnt_rr * 1000) / FS_HZ;
rr_ms_i <= to_signed(ms_tmp, 24);

cnt_rr <= 0;
cnt_rt <= 0;
rt_busy <= '1';
end if;

if t_voted = '1' and t_voted_prev = '0' and rt_busy = '1' then
t_unif_i <= '1';

-- Para corregir el intervalo RT, le restamos el retraso de la onda T y le sumamos el
  de la QRS
rt_smp := cnt_rt - SAMPLES_T_DELAY + SAMPLES_QRS_DELAY;

if rt_smp < 0 then
rt_ms_i <= (others => '0');
else
ms_tmp := (rt_smp * 1000) / FS_HZ;
rt_ms_i <= to_signed(ms_tmp, 24);
end if;

rt_busy <= '0';
end if;
end if;
end if;
end process;
end Behavioral;

```

A.0.9. Evaluador de Patologías (Detection Alarm)

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity detection_alarm is
generic (
FS_HZ : integer := 1000
);
Port (
clk
: in STD_LOGIC;

```

```

reset                : in  STD_LOGIC;

-- Entradas (bridge)
d_valid              : in  STD_LOGIC;
qrs_unified          : in  STD_LOGIC;
t_unified            : in  STD_LOGIC;
rr_interval_ms       : in  SIGNED(23 downto 0);
rt_interval_ms       : in  SIGNED(23 downto 0);

-- Salidas (Alarmas UART)
alarm_tachycardia    : out STD_LOGIC := '0';
alarm_bradycardia    : out STD_LOGIC := '0';
alarm_arrhythmia     : out STD_LOGIC := '0';
alarm_asystole       : out STD_LOGIC := '0';
alarm_sudden_death   : out STD_LOGIC := '0'
);
end detection_alarm;

architecture Behavioral of detection_alarm is

-- Umbrales clínicos
constant THRESHOLD_TACHY      : integer := 600;
constant THRESHOLD_BRADY     : integer := 1200;

-- 3 segundos para asistolia
constant THRESHOLD_ASYSTOLE   : integer := 3 * FS_HZ;

signal last_rr_ms           : SIGNED(23 downto 0) := (others => '0');
signal asystole_cnt         : integer := 0;

-- Registros de persistencia (Para activar una alarma cuando varios latidos confirman
-- la patología)
signal tachy_persist_reg : integer := 0;
signal brady_persist_reg : integer := 0;
signal arrh_persist_reg  : integer := 0;
signal death_persist_reg : integer := 0;

begin

process(clk)
-- Variables actualizar valores
variable v_tachy_persist : integer := 0;
variable v_brady_persist : integer := 0;
variable v_arrh_persist  : integer := 0;
variable v_death_persist : integer := 0;
variable diff_rr         : SIGNED(23 downto 0);
variable v_r_since_t     : integer := 0;
begin
if rising_edge(clk) then
if reset = '1' then
alarm_tachycardia <= '0';
alarm_bradycardia <= '0';
alarm_arrhythmia  <= '0';
alarm_asystole    <= '0';
alarm_sudden_death <= '0';

v_tachy_persist := 0;

```

```

v_brady_persist := 0;
v_arrh_persist  := 0;
v_death_persist := 0;
v_r_since_t     := 0;

tachy_persist_reg <= 0;
brady_persist_reg <= 0;
arrh_persist_reg  <= 0;
death_persist_reg <= 0;

asystole_cnt <= 0;
last_rr_ms   <= (others => '0');

elsif d_valid = '1' then

    -- Activador Asistolia
    if asystole_cnt >= THRESHOLD_ASYSTOLE then
        alarm_asystole <= '1';
    else
        asystole_cnt <= asystole_cnt + 1;
        alarm_asystole <= '0';
    end if;

    if qrs_unified = '1' then
        asystole_cnt <= 0;

    -- Registro de onda R
    v_r_since_t := v_r_since_t + 1;

    -- Persistencia de Taquicardia
    if rr_interval_ms < THRESHOLD_TACHY and rr_interval_ms > 0 then
        if v_tachy_persist < 2 then
            v_tachy_persist := v_tachy_persist + 1;
        end if;
    else
        v_tachy_persist := 0;
    end if;

    -- Persistencia de Bradicardia
    if rr_interval_ms > THRESHOLD_BRADY then
        if v_brady_persist < 2 then
            v_brady_persist := v_brady_persist + 1;
        end if;
    else
        v_brady_persist := 0;
    end if;

    -- Persistencia de arritmia
    diff_rr := abs(rr_interval_ms - last_rr_ms);
    if diff_rr > shift_right(last_rr_ms, 2) and last_rr_ms > 0 then
        if v_arrh_persist < 2 then
            v_arrh_persist := v_arrh_persist + 1;
        end if;
    else
        v_arrh_persist := 0;
    end if;

```

```

-- Activacion de alarmas
if v_tachy_persist >= 2 then alarm_tachycardia <= '1'; else alarm_tachycardia <= '0';
  end if;
if v_brady_persist >= 2 then alarm_bradycardia <= '1'; else alarm_bradycardia <= '0';
  end if;
if v_arrh_persist >= 2 then alarm_arrhythmia <= '1'; else alarm_arrhythmia <= '0';
  end if;

last_rr_ms <= rr_interval_ms;
tachy_persist_reg <= v_tachy_persist;
brady_persist_reg <= v_brady_persist;
arrh_persist_reg <= v_arrh_persist;
end if;

-- Deteccion de muerte subita
if t_unified = '1' then

-- Validacion de unicamente una R antes de la T para asegurarnos de no hacer un falso
  positivo por fallo de deteccion
  if v_r_since_t = 1 then

-- Regla de la mitad clinica. Adaptacion del tiempo maximo antes de hacer saltar la
    alarma segun ritmo cardiaco.
    if rt_interval_ms > shift_right(last_rr_ms, 1) - shift_right(last_rr_ms, 4) then
      if v_death_persist < 2 then
        v_death_persist := v_death_persist + 1;
      end if;
    else
      v_death_persist := 0;
    end if;

  else
    v_death_persist := 0;
  end if;

-- Activacion de la alarma
  if v_death_persist >= 2 then
    alarm_sudden_death <= '1';
  else
    alarm_sudden_death <= '0';
  end if;

  death_persist_reg <= v_death_persist;

  v_r_since_t := 0;
end if;
end if;
end if;
end process;

end Behavioral;

```

B. CÓDIGO FUENTE PYTHON

En este anexo se incluye el código fuente de la interfaz gráfica y de telemetría desarrollada en Python. Se divide en tres scripts: uno para la monitorización en vivo conectando el sensor a través del puerto serie, otro para el análisis en diferido utilizando las simulaciones exportadas desde Vivado y el último para empaquetar los datos de la base de datos del MIT.

B.0.1. Script de Monitorización en Tiempo Real (Puerto Serie)

```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.animation as animation
from matplotlib.collections import LineCollection
import serial
import numpy as np
from collections import deque
import sys

def decode_24bit_signed(b1, b2, b3):
    val = (b1 << 16) | (b2 << 8) | b3
    return val - 0x1000000 if val & 0x800000 else val

def decode_24bit_unsigned(b1, b2, b3):
    return (b1 << 16) | (b2 << 8) | b3

# conexion fisica a la fpga
PUERTO = 'COM4'
BAUD_RATE = 115200

try:
    # timeout cero evita bloqueo grafico
    puerto_serie = serial.Serial(PUERTO, BAUD_RATE, timeout=0)
    print(f"Conectado a la FPGA en {PUERTO} a {BAUD_RATE} bps.")
except Exception as e:
    print(f"Error abriendo el puerto serie: {e}")
sys.exit()

# configuracion de la interfaz grafica
plt.style.use('dark_background')
fig = plt.figure(figsize=(15, 9), facecolor='#0a0a0a')
fig.canvas.manager.set_window_title("Dashboard TFG - MODO REAL-TIME SENSOR")
```

```

gs = gridspec.GridSpec(4, 3, height_ratios=[1, 1, 1, 0.4])

# ejes 2d para señales raw
ax_x = fig.add_subplot(gs[0, 0:2])
ax_y = fig.add_subplot(gs[1, 0:2])
ax_z = fig.add_subplot(gs[2, 0:2])
axes_1d = [ax_x, ax_y, ax_z]

titles = ["Señal RAW - Eje X", "Señal RAW - Eje Y", "Señal RAW - Eje Z"]
colors = ["#ff4444", "#44ff44", "#4444ff"]
lines_1d = []

# colecciones para marcas de qrs y t
qrs_collections = []
t_collections = []

for idx, ax in enumerate(axes_1d):
    ax.set_facecolor('#121212')
    ax.set_title(titles[idx], fontweight='bold', fontsize=10)
    ax.grid(True, linestyle=':', alpha=0.3)
    line, = ax.plot([], [], color=colors[idx], linewidth=1.5)
    lines_1d.append(line)

qrs_col = LineCollection([], colors='#ffaa00', linewidths=1.2, linestyle='--')
t_col = LineCollection([], colors='#ff00ff', linewidths=1.2, linestyle='--')
ax.add_collection(qrs_col)
ax.add_collection(t_col)
qrs_collections.append(qrs_col)
t_collections.append(t_col)

# grafico 3d unificado
ax_3d = fig.add_subplot(gs[0:3, 2], projection='3d')
ax_3d.set_facecolor('#0a0a0a')
ax_3d.set_title("Vectorcardiograma 3D en Vivo", fontsize=11, fontweight='bold')
ax_3d.xaxis.set_pane_color((0.0, 0.0, 0.0, 0.0))
ax_3d.yaxis.set_pane_color((0.0, 0.0, 0.0, 0.0))
ax_3d.zaxis.set_pane_color((0.0, 0.0, 0.0, 0.0))
line_3d, = ax_3d.plot([], [], [], color='cyan', linewidth=1.5)

# panel inferior de estados y tiempos
ax_dash = fig.add_subplot(gs[3, :])
ax_dash.set_facecolor('#050505')
ax_dash.axis('off')

txt_rr = ax_dash.text(0.20, 0.7, "RR: --- ms", fontsize=22, color='#00FF00',
    fontfamily='monospace', ha='center')
txt_rt = ax_dash.text(0.80, 0.7, "RT: --- ms", fontsize=22, color='#00FFFF',
    fontfamily='monospace', ha='center')

labels_alarmas = ['TAQUICARDIA', 'BRADICARDIA', 'ARRITMIA', 'ASISTOLIA', 'MUERTE
    SÚBITA']
botones = []
for idx, lab in enumerate(labels_alarmas):
    x_pos = 0.1 + (idx * 0.2)
    btn = ax_dash.text(x_pos, 0.2, lab, ha='center', va='center', fontsize=11, color='
        white', fontweight='bold',
        bbox=dict(facecolor='#222222', edgecolor='#555555', boxstyle='round,pad=0.8'))

```

```

botones.append(btn)

# memoria dinamica de alta velocidad
ANCHO_VENTANA = 400
t_axis = deque(maxlen=ANCHO_VENTANA)
data_rx = deque(maxlen=ANCHO_VENTANA)
data_ry = deque(maxlen=ANCHO_VENTANA)
data_rz = deque(maxlen=ANCHO_VENTANA)
data_qrs = deque(maxlen=ANCHO_VENTANA)
data_t = deque(maxlen=ANCHO_VENTANA)

buffer_recepcion = bytearray()
frame_global = 0

ultimo_rr = 0
ultimo_rt = 0
ultimo_alarm_byte = 0

print("Escuchando datos... Cierra la ventana de Matplotlib para detener.")

# motor de actualizacion en tiempo real
def update(frame_anim):
    global frame_global, buffer_recepcion
    global ultimo_rr, ultimo_rt, ultimo_alarm_byte

    tramas_procesadas_ahora = 0

    # lectura de buffer usb sin bloqueo
    if puerto_serie.in_waiting > 0:
        buffer_recepcion.extend(puerto_serie.read(puerto_serie.in_waiting))

    # parseo y extraccion de tramas
    i = 0
    while i <= len(buffer_recepcion) - 37:
        if buffer_recepcion[i] == 0xAA and buffer_recepcion[i+1] == 0xBB and buffer_recepcion[
            i+36] == 0x0A:
            # extraccion de ejes raw
            rx = decode_24bit_signed(buffer_recepcion[i+2], buffer_recepcion[i+3],
                buffer_recepcion[i+4])
            ry = decode_24bit_signed(buffer_recepcion[i+5], buffer_recepcion[i+6],
                buffer_recepcion[i+7])
            rz = decode_24bit_signed(buffer_recepcion[i+8], buffer_recepcion[i+9],
                buffer_recepcion[i+10])

            # extraccion de tiempos
            ultimo_rr = decode_24bit_unsigned(buffer_recepcion[i+29], buffer_recepcion[i+30],
                buffer_recepcion[i+31])
            ultimo_rt = decode_24bit_unsigned(buffer_recepcion[i+32], buffer_recepcion[i+33],
                buffer_recepcion[i+34])

            # extraccion de alarmas
            ultimo_alarm_byte = buffer_recepcion[i+35]
            qrs_detected = (ultimo_alarm_byte >> 5) & 1
            t_detected = (ultimo_alarm_byte >> 6) & 1

            # almacenamiento en memoria
            t_axis.append(frame_global)

```

```

data_rx.append(rx)
data_ry.append(ry)
data_rz.append(rz)
data_qrs.append(qrs_detected)
data_t.append(t_detected)

frame_global += 1
i += 37
tramas_procesadas_ahora += 1
else:
i += 1 # resincronizacion por ruido

# liberacion de memoria ram
del buffer_recepcion[:i]

# actualizacion visual por datos nuevos
if tramas_procesadas_ahora > 0 and len(t_axis) > 0:
# conversion a numpy para velocidad
arr_rx = np.array(data_rx)
arr_ry = np.array(data_ry)
arr_rz = np.array(data_rz)
arr_t = np.array(t_axis)

# actualizacion de trayectorias 2d y 3d
lines_1d[0].set_data(arr_t, arr_rx)
lines_1d[1].set_data(arr_t, arr_ry)
lines_1d[2].set_data(arr_t, arr_rz)

line_3d.set_data(arr_rx, arr_ry)
line_3d.set_3d_properties(arr_rz)

# autoescalado dinamico
margin = 500
lim_x = (np.min(arr_rx) - margin, np.max(arr_rx) + margin)
lim_y = (np.min(arr_ry) - margin, np.max(arr_ry) + margin)

# proteccion contra z plana
min_z, max_z = np.min(arr_rz), np.max(arr_rz)
lim_z = (min_z - 1000, max_z + 1000) if min_z == max_z else (min_z - margin, max_z +
    margin)

# aplicacion de limites
min_time = arr_t[0]
max_time = arr_t[-1]
for ax in axes_1d:
ax.set_xlim(min_time, max(min_time + ANCHO_VENTANA, max_time))

ax_x.set_ylim(lim_x)
ax_y.set_ylim(lim_y)
ax_z.set_ylim(lim_z)

ax_3d.set_xlim(lim_x)
ax_3d.set_ylim(lim_y)
ax_3d.set_zlim(lim_z)

# actualizacion de marcadores qrs y t
qrs_idx = [t for t, q in zip(arr_t, data_qrs) if q == 1]

```

```

t_idx = [t for t, dt in zip(arr_t, data_t) if dt == 1]

qrs_collections[0].set_segments([[x, lim_x[0]], (x, lim_x[1])] for x in qrs_idx])
qrs_collections[1].set_segments([[x, lim_y[0]], (x, lim_y[1])] for x in qrs_idx])
qrs_collections[2].set_segments([[x, lim_z[0]], (x, lim_z[1])] for x in qrs_idx])

t_collections[0].set_segments([[x, lim_x[0]], (x, lim_x[1])] for x in t_idx])
t_collections[1].set_segments([[x, lim_y[0]], (x, lim_y[1])] for x in t_idx])
t_collections[2].set_segments([[x, lim_z[0]], (x, lim_z[1])] for x in t_idx])

# actualizacion de textos y alarmas
txt_rr.set_text(f"RR: {ultimo_rr} ms")
txt_rt.set_text(f"RT: {ultimo_rt} ms")

f = ultimo_alarm_byte
al_states = [(f>>4)&1, (f>>3)&1, (f>>2)&1, (f>>1)&1, f&1]
for j, state in enumerate(al_states):
    color_fondo = '#FF0000' if state else '#222222'
    botones[j].set_bbox(dict(facecolor=color_fondo, edgecolor='#777777', boxstyle='round',
                             pad=0.8))

# retorno de elementos a actualizar
elementos = lines_1d + [line_3d, txt_rr, txt_rt] + botones
return elementos

# ejecucion de la animacion
ani = animation.FuncAnimation(fig, update, interval=30, blit=False, cache_frame_data=False)

# cierre seguro del puerto com
def on_close(event):
    puerto_serie.close()
    print("Puerto serie cerrado con seguridad. Hasta la próxima.")
    fig.canvas.mpl_connect('close_event', on_close)

plt.tight_layout()
plt.show()

```

B.0.2. Script de Simulación Diferida (Lectura de Archivo)

```

import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import matplotlib.animation as animation
from matplotlib.collections import LineCollection
import numpy as np

def decode_24bit_signed(b1, b2, b3):
    val = (b1 << 16) | (b2 << 8) | b3
    return val - 0x1000000 if val & 0x800000 else val

def decode_24bit_unsigned(b1, b2, b3):
    return (b1 << 16) | (b2 << 8) | b3

```

```

# carga y parseo de datos
try:
    with open("uart_prueba_3d.txt", "r") as f:
        bytes_totales = [int(line.strip()) for line in f if line.strip() != ""]
    except Exception as e:
        exit(f"Error cargando archivo: {e}")

frames = []
i = 0
while i <= len(bytes_totales) - 37:
    if bytes_totales[i:i+2] == [0xAA, 0xBB] and bytes_totales[i+36] == 0x0A:
        # extraccion de datos raw
        rx = decode_24bit_signed(bytes_totales[i+2], bytes_totales[i+3], bytes_totales[i+4])
        ry = decode_24bit_signed(bytes_totales[i+5], bytes_totales[i+6], bytes_totales[i+7])
        rz = decode_24bit_signed(bytes_totales[i+8], bytes_totales[i+9], bytes_totales[i+10])

        # extraccion de tiempos
        rr = decode_24bit_unsigned(bytes_totales[i+29], bytes_totales[i+30], bytes_totales[i+31])
        rt = decode_24bit_unsigned(bytes_totales[i+32], bytes_totales[i+33], bytes_totales[i+34])

        # registro de alarmas
        alarms = bytes_totales[i+35]

        # deteccion de qrs y onda t
        qrs_detected = (alarms >> 5) & 1
        t_detected = (alarms >> 6) & 1

        frames.append((rx, ry, rz, rr, rt, alarms, qrs_detected, t_detected))
        i += 37
    else:
        i += 1

if not frames:
    exit("Error: No se encontraron tramas válidas en el archivo.")

# conversion a arrays numpy
data_rx = np.array([f[0] for f in frames])
data_ry = np.array([f[1] for f in frames])
data_rz = np.array([f[2] for f in frames])
data_rr = np.array([f[3] for f in frames])
data_rt = np.array([f[4] for f in frames])
data_al = np.array([f[5] for f in frames])
data_qrs = np.array([f[6] for f in frames])
data_t = np.array([f[7] for f in frames])

# configuracion de la interfaz grafica
plt.style.use('dark_background')
fig = plt.figure(figsize=(15, 9), facecolor='#0a0a0a')
fig.canvas.manager.set_window_title("Dashboard TFG - Modo Pro Unificado")

gs = gridspec.GridSpec(4, 3, height_ratios=[1, 1, 1, 0.4])

# ejes 2d para señales raw
ax_x = fig.add_subplot(gs[0, 0:2])

```

```

ax_y = fig.add_subplot(gs[1, 0:2])
ax_z = fig.add_subplot(gs[2, 0:2])
axes_1d = [ax_x, ax_y, ax_z]

titles = ["Señal RAW - Eje X", "Señal RAW - Eje Y", "Señal RAW - Eje Z"]
colors = ["#ff4444", "#44ff44", "#4444ff"]
lines_1d = []

# colecciones para qrs y t
qrs_collections = []
t_collections = []

for idx, ax in enumerate(axes_1d):
    ax.set_facecolor('#121212')
    ax.set_title(titles[idx], fontweight='bold', fontsize=10)
    ax.grid(True, linestyle=':', alpha=0.3)
    line, = ax.plot([], [], color=colors[idx], linewidth=1.5)
    lines_1d.append(line)

# lineas verticales para qrs y t
qrs_col = LineCollection([], colors='#ffaa00', linewidths=1.2, linestyle='--')
t_col = LineCollection([], colors='#ff00ff', linewidths=1.2, linestyle='--')

ax.add_collection(qrs_col)
ax.add_collection(t_col)
qrs_collections.append(qrs_col)
t_collections.append(t_col)

# grafico 3d unificado
ax_3d = fig.add_subplot(gs[0:3, 2], projection='3d')
ax_3d.set_facecolor('#0a0a0a')
ax_3d.set_title("Vectorcardiograma 3D Unificado", fontsize=11, fontweight='bold')
ax_3d.xaxis.set_pane_color((0.0, 0.0, 0.0, 0.0))
ax_3d.yaxis.set_pane_color((0.0, 0.0, 0.0, 0.0))
ax_3d.zaxis.set_pane_color((0.0, 0.0, 0.0, 0.0))
line_3d, = ax_3d.plot([], [], [], color='cyan', linewidth=1.5)

# panel de estados y tiempos
ax_dash = fig.add_subplot(gs[3, :])
ax_dash.set_facecolor('#050505')
ax_dash.axis('off')

txt_rr = ax_dash.text(0.20, 0.7, "RR: --- ms", fontsize=22, color='#00FF00',
    fontfamily='monospace', ha='center')
txt_rt = ax_dash.text(0.80, 0.7, "RT: --- ms", fontsize=22, color='#00FFFF',
    fontfamily='monospace', ha='center')

labels_alarmas = ['TAQUICARDIA', 'BRADICARDIA', 'ARRITMIA', 'ASISTOLIA', 'MUERTE
    SÚBITA']
botones = []
for idx, lab in enumerate(labels_alarmas):
    x_pos = 0.1 + (idx * 0.2)
    btn = ax_dash.text(x_pos, 0.2, lab, ha='center', va='center', fontsize=11, color='
        white', fontweight='bold',
        bbox=dict(facecolor='#222222', edgecolor='#555555', boxstyle='round,pad=0.8'))
    botones.append(btn)

```

```

# configuracion de la animacion
ANCHO = 400
STEP = 4

margin = 500
lim_x = (np.min(data_rx) - margin, np.max(data_rx) + margin)
lim_y = (np.min(data_ry) - margin, np.max(data_ry) + margin)
lim_z = (np.min(data_rz) - margin, np.max(data_rz) - 100 if np.min(data_rz) == np.max(
    data_rz) else np.max(data_rz) + margin)

if lim_z[0] == lim_z[1]:
    lim_z = (-1000, 1000)

def init():
    for ax in axes_1d:
        ax.set_xlim(0, ANCHO)
        ax_x.set_ylim(lim_x)
        ax_y.set_ylim(lim_y)
        ax_z.set_ylim(lim_z)

    ax_3d.set_xlim(lim_x)
    ax_3d.set_ylim(lim_y)
    ax_3d.set_zlim(lim_z)
    return lines_1d + [line_3d, txt_rr, txt_rt] + botones

def update(frame_step):
    idx = frame_step * STEP
    if idx >= len(data_rx):
        idx = len(data_rx) - 1

    start = max(0, idx - ANCHO)
    window_length = idx - start

    if window_length > 0:
        # actualizacion de señales 2d
        t_axis = np.arange(window_length)
        lines_1d[0].set_data(t_axis, data_rx[start:idx])
        lines_1d[1].set_data(t_axis, data_ry[start:idx])
        lines_1d[2].set_data(t_axis, data_rz[start:idx])

    # actualizacion de lineas para qrs y t
    window_qrs = data_qrs[start:idx]
    window_t = data_t[start:idx]

    qrs_x_coords = np.where(window_qrs == 1)[0]
    t_x_coords = np.where(window_t == 1)[0]

    qrs_collections[0].set_segments([[ (x, lim_x[0]), (x, lim_x[1]) ] for x in qrs_x_coords
    ])
    qrs_collections[1].set_segments([[ (x, lim_y[0]), (x, lim_y[1]) ] for x in qrs_x_coords
    ])
    qrs_collections[2].set_segments([[ (x, lim_z[0]), (x, lim_z[1]) ] for x in qrs_x_coords
    ])

    t_collections[0].set_segments([[ (x, lim_x[0]), (x, lim_x[1]) ] for x in t_x_coords])
    t_collections[1].set_segments([[ (x, lim_y[0]), (x, lim_y[1]) ] for x in t_x_coords])
    t_collections[2].set_segments([[ (x, lim_z[0]), (x, lim_z[1]) ] for x in t_x_coords])

```

```

# actualizacion de señal 3d
line_3d.set_data(data_rx[start:idx], data_ry[start:idx])
line_3d.set_3d_properties(data_rz[start:idx])

# textos de intervalos
txt_rr.set_text(f"RR: {data_rr[idx]} ms")
txt_rt.set_text(f"RT: {data_rt[idx]} ms")

# mapeo de alarmas
f = data_al[idx]
al_states = [(f>>4)&1, (f>>3)&1, (f>>2)&1, (f>>1)&1, f&1]

for j, state in enumerate(al_states):
    color_fondo = '#FF0000' if state else '#222222'
    botones[j].set_bbox(dict(facecolor=color_fondo, edgecolor='#777777', boxstyle='round',
                             pad=0.8))

return lines_1d + [line_3d, txt_rr, txt_rt] + botones

# ejecucion de la animacion a 50hz
total_frames = len(data_rx) // STEP
ani = animation.FuncAnimation(fig, update, frames=total_frames, init_func=init, blit=
    False, interval=20)

plt.tight_layout()
plt.show()

```

B.0.3. Script para la obtención de los datos de la base de datos del MIT

```

import wfdb
import numpy as np

# === CONFIGURACIÓN ===
FILENAME = "ecg_healthy_raw.txt"
SAMPLES = 5000
AMPLITUD_MAX = 6000000

try:
    record = wfdb.rdrecord('100', pn_dir='mitdb', sampfrom=0, sampto=SAMPLES)
    raw_signal = record.p_signal[:, 0]
except Exception as e:
    print(f"Error: {e}")
    exit()

# Escalado a 24 bits
max_val = np.max(np.abs(raw_signal))
int_signal = (raw_signal / max_val * AMPLITUD_MAX).astype(int)

np.savetxt(FILENAME, int_signal, fmt='%d')

```


C.

IMPACTO SOCIAL Y MEDIOAMBIENTAL

Cumpliendo con la normativa de la Universidad Politécnica de Madrid, a continuación se detalla el impacto que tiene el desarrollo de este proyecto tanto en la sociedad como en la conservación del entorno.

Impacto Social

Este proyecto busca hacer más accesible la tecnología médica avanzada. Al pasar de equipos de hospital grandes y caros a un dispositivo portátil y económico, facilitamos que cualquier persona pueda monitorizar su corazón de forma continua. Esto es vital para detectar a tiempo problemas cardíacos graves, como el Síndrome de QT Largo, y prevenir la muerte súbita desde casa de una manera no invasiva, mejorando la calidad de vida del paciente.

Impacto Medioambiental

El uso de un chip programable (FPGA) reduce de forma drástica el consumo de energía si lo comparamos con el uso de un ordenador convencional trabajando constantemente para procesar las señales médicas. Además, el código se ha diseñado específicamente para que los circuitos solo se "despierten" en el instante exacto en que llega un dato nuevo del sensor. Al evitar que el chip procese información vacía o se quede esperando activamente, logramos minimizar el calor que genera la placa y reducimos notablemente la huella de carbono del dispositivo a lo largo de toda su vida útil.

Índice de términos

Glosario

Arritmia Pérdida de la regularidad constante y la sincronía en el intervalo de tiempo transcurrido entre latidos consecutivos del corazón

Asistolia Ausencia completa de actividad eléctrica en el miocardio, lo que se traduce clínicamente en un paro cardíaco

Bitstream Archivo de programación binario generado tras la síntesis e implementación, encargado de configurar físicamente las puertas lógicas de la FPGA

Bradicardia Alteración del ritmo cardíaco donde la frecuencia en reposo desciende a un nivel anormalmente bajo, por debajo de los 60 latidos por minuto

Complejo QRS Conjunto de ondas de alta frecuencia en el registro cardíaco que representan la contracción y despolarización de los ventrículos

Diástole Fase del ciclo cardíaco encargada de la relajación muscular y destinada al llenado de las cámaras cardíacas

Muerte Súbita Cardíaca Parada inesperada y repentina del corazón, frecuentemente desencadenada por una fibrilación originada durante el periodo vulnerable de repolarización

Onda T Curva de baja frecuencia en el electrocardiograma que muestra la repolarización y recuperación eléctrica de las células miocárdicas

Resampleo (Remuestreo) Técnica de procesamiento digital mediante software que adapta y transforma una señal capturada a una frecuencia de muestreo diferente

Spline cúbico Función polinómica utilizada en la etapa de filtrado para lograr un suavizado matemático continuo de las señales biomédicas

Síndrome de QT Largo Alteración cardíaca peligrosa caracterizada por un retraso anómalo en la recuperación eléctrica de los ventrículos (intervalo RT)

Sístole Fase del ciclo cardíaco en la que el músculo ventricular se contrae para expulsar la sangre hacia el sistema circulatorio

Taquicardia Trastorno del ritmo cardíaco caracterizado por una frecuencia en reposo acelerada, superior a los 100 latidos por minuto

Transformada Wavelet Herramienta matemática diseñada para analizar señales no estacionarias, descomponiéndolas en diferentes frecuencias y ubicándolas en el tiempo

Votación por Mayoría Algoritmo lógico de decisión democrática que requiere que al menos dos de tres componentes espaciales coincidan para confirmar la validez de un latido

Siglas

CLB Configurable Logic Block (Bloque Lógico Configurable)

DSP Digital Signal Processor (Procesador de Señales Digitales)

ECG Electrocardiograma

FPGA Field Programmable Gate Array (Matriz de Puertas Lógicas Programable)

FSM Finite State Machine (Máquina de Estados Síncrona)

ILA Integrated Logic Analyzer (Analizador Lógico Integrado)

LUT Look-Up Table (Tabla de Consulta)

MIT Massachusetts Institute of Technology

SPI Serial Peripheral Interface (Interfaz Periférica Serie)

UART Universal Asynchronous Receiver-Transmitter

VCG Vectocardiograma

VHDL VHSIC Hardware Description Language

WCT Wilson Central Terminal (Terminal Central de Wilson)

